The Backend Engineer's Library

Messaging, Streaming, and Event Systems

Volume 10

Contents

Messaging Fundamentals: Queues, Logs, Pub/Sub

Delivery Semantics: At-Most, At-Least, Exactly-Once

Apache Kafka Architecture

Stream Processing

Event Sourcing and CQRS

The Outbox Pattern and the Dual-Write Problem

Backpressure and Flow Control

Dead Letters, Retries, Poison Messages

Chapter 1 — Messaging Fundamentals: Queues, Logs, Pub/Sub

What this chapter covers. Every interesting backend is a distributed system, and distributed systems communicate through messages. Whether you call it an RPC, an event, a task, or a notification, the question is the same: how does information move between services that fail independently, deploy independently, and scale independently? This chapter builds the taxonomy that the rest of Volume 10 depends on. Three abstractions — **queues**, **logs**, and **publish/subscribe** — cover essentially every messaging system you will encounter, from Amazon SQS to Apache Kafka to Google Pub/Sub to RabbitMQ. They differ in their data model (destructive consume vs. retained log), their consumption model (competing vs. broadcast vs. partitioned), and their ordering and durability guarantees. Get these distinctions right and every broker becomes a variation on a theme; get them wrong and you will build ordering assumptions on a system that never promised them.

Learning goals — after this chapter you should be able to:

- Distinguish queue, log, and pub/sub semantics on five axes: consumption model, retention, ordering scope, delivery topology, and replay capability.
- Explain the distributed-systems reason messaging exists — temporal, spatial, and failure decoupling — and when synchronous RPC is the wrong choice.
- Describe how a point-to-point queue works end to end: enqueue, competing consumers, acknowledgment, visibility timeout, and poison-message handling.
- Describe how a partitioned log works: append-only segments, offsets, sequential I/O, retention, replay, and partitioned parallelism.
- Describe how pub/sub fan-out works: topics, subscriptions, filtering, and the push-vs.-pull delivery choice.

- Map real systems (SQS, RabbitMQ, Kafka, Kinesis, Redis Streams, NATS, Google Pub/Sub) onto the taxonomy and explain where hybrids blur the lines.
- Choose correctly among queue, log, and pub/sub for a given workload using a concrete decision framework.

Why messaging: decoupling in a distributed system

A monolith calls a function. A distributed system sends a message. The difference is not syntax — it is the failure model. A function call either happens or it does not; the caller and callee share fate. A message crosses a network between processes that fail independently, and the sender cannot know whether the message was processed without an explicit acknowledgment that itself can be lost (see Vol 6, Ch 1 — the Two Generals problem).

Messaging infrastructure exists to manage that reality. It provides three kinds of decoupling:

Decoupling	Without messaging	With messaging
Temporal	Producer and consumer must be alive at the same instant (synchronous RPC)	Producer writes now, consumer reads later — hours or days later if needed
Spatial	Producer must know consumer addresses and cardinality	Producer knows a queue/topic name; the broker handles routing and scaling
Failure	A slow or crashed consumer blocks or fails the producer	The broker buffers durably; producer and consumer fail and recover independently

This decoupling is not free. It trades latency for durability, ordering simplicity for throughput, and request/response clarity for eventual processing. Understanding when the trade is worthwhile is the first design judgment in any messaging architecture.

> **When not to use messaging.** If the caller needs the answer before it can proceed ("what is this user's current balance?"), a synchronous request — REST, gRPC, or a read from a replicated store — is simpler and faster. Messaging shines when the producer can proceed without waiting:

> "an order was placed," "resize this image," "reindex this document." The heuristic: if you can name the event in the past tense and the producer does not need the result, a message is likely the right shape.

```
flowchart LR
    subgraph Sync["Synchronous RPC — coupled"]
        A1[Service A] -- "request / response\nboth must be live" -->
B1[Service B]
        B1 -- "success or failure\nimmediate backpressure" --> A1
    end
    subgraph Async["Asynchronous messaging — decoupled"]
        A2[Service A] -- "produce\nfire and proceed" -->
Broker[(Broker\nbuffer + route)]
        Broker -- "deliver\nwhen ready" --> B2[Service B]
        Broker -- "deliver\nwhen ready" --> C2[Service C]
    end
    style Broker fill:#e3f2fd
```

Figure 1-1: Synchronous coupling versus asynchronous decoupling. The broker turns a temporal rendezvous into a durable handoff, at the cost of an extra hop and eventual delivery.

The three models at a glance

Most confusion about messaging comes from treating every broker as "a queue." In fact there are three distinct data models, each with different guarantees. Real products often implement more than one, but the models themselves are cleanly separable.

Axis	Queue (point-to-point)	Log (partitioned, append-only)	Pub/Sub (fan-out)
Core verb	*Send* to a queue, *receive* from it	*Append* to a log, *read* from an offset	*Publish* to a topic, *subscribe* to it
Consumption	Competing consumers; each message to **one** consumer	Partitioned readers; each partition to **one** consumer in a group, but many groups can read independently	Broadcast; each message to **every** subscription

Axis	Queue (point-to-point)	Log (partitioned, append-only)	Pub/Sub (fan-out)
After delivery	Removed (or hidden until ack) — destructive consume	Retained by retention policy — immutable, replayable	Depends on subscription — often acked and removed per-subscription
Ordering	Best-effort FIFO per queue (often no global order)	Total order **within a partition**; no order across partitions	Usually unordered or best-effort per-publisher
Replay	No — once acked, gone	Yes — seek to any offset, re-read any window	Sometimes — subscription seek/replay if the broker retains
Scaling axis	More consumers drain faster, but single queue is the bottleneck	More partitions = more parallel readers and writers	More subscriptions = more fan-out; each subscription scales independently
Canonical examples	Amazon SQS Standard/FIFO, RabbitMQ classic queue, ActiveMQ queue	Apache Kafka, Amazon Kinesis, RabbitMQ Streams, Redpanda	Google Pub/Sub, Amazon SNS, NATS core, Redis Pub/Sub

The mental model: a **queue** is a mailbox — mail is removed when collected. A **log** is a commit history — entries are never removed by reading, only by retention, and you read by moving a cursor. **Pub/sub** is a mailing list — one post reaches every subscriber's copy.

Figure 1-2: The three consumption topologies. Queue drains to one winner; log fans out by partition to groups that track their own offsets; pub/sub replicates to every subscription.

> **Boundary note.** This chapter classifies the *broker data model* — what happens to a message after it is written and who can read it. Delivery guarantees (at-most/at-least/effectively-once, redelivery, deduplication, and transactional consumption) are a separate concern, covered in Chapter 2. Exactly-once processing theory (the end-to-end argument, idempotency keys, and the atomic check-and-record pattern) is developed

> formally in Vol 6, Chapter 9 — Idempotency, Deduplication, and Exactly-Once; Chapter 2 here focuses on what the *broker* can and cannot promise.

Queues: point-to-point and competing consumers

The contract

A queue provides **point-to-point** semantics: a producer enqueues a message; exactly one consumer dequeues it. If multiple consumers are attached, they compete — the broker delivers each message to one consumer, typically round-robin or based on prefetch and readiness.

The lifecycle of a queue message:

1. **Enqueue** — producer sends; broker persists (durably or in memory, per configuration) and acknowledges the write.
2. **Deliver** — broker selects a consumer and pushes or lets it pull the message. The message becomes *invisible* or *unacked* but is not yet deleted.
3. **Acknowledge** — consumer signals success (`ack`). The broker deletes the message.
4. **Negative-acknowledge or timeout** — consumer signals failure (`nack` / `reject`) or crashes before acking; the visibility timeout expires; the broker makes the message visible again for redelivery, often with a redelivery count.
5. **Dead-letter** — after N failed deliveries, the broker routes the message to a dead-letter queue (DLQ) for inspection. See Chapter 8 for DLQ design.

```
stateDiagram-v2
    [*] --> Enqueued: producer send
    Enqueued --> Invisible: broker delivers\nto consumer
    Invisible --> Acked: consumer ack\n→ deleted
    Invisible --> Visible: nack / reject\nor visibility timeout
    Visible --> Invisible: redelivered\nto next consumer
    Visible --> DeadLetter: max deliveries\nexceeded → DLQ
    Acked --> [*]
    DeadLetter --> [*]
```

Figure 1-3: Queue message lifecycle. The invisible state is the at-least-once window — a crash between delivery and ack causes redelivery.

Ordering and scaling limits

A single queue with a single consumer gives total FIFO order — trivially, because there is only one reader. The moment you add competing consumers, global ordering is lost: message 2 may be acked before message 1 if consumer B is faster than consumer A. Systems that promise FIFO queues (SQS FIFO, RabbitMQ quorum queues with single active consumer) do so by constraining concurrency — one active consumer per queue or per message group — and paying the throughput price.

A single queue is also a scaling bottleneck. Throughput is bounded by the broker node owning the queue and by the serial ack/delete path. The standard scale-out is **more queues**: sharding by tenant, entity, or hash, each with its own consumer set.

Prefetch, backpressure, and fairness

Without flow control, a fast broker will overwhelm a slow consumer. Queues implement **prefetch** (RabbitMQ `basic.qos`, SQS long polling + max messages, Kafka `max.poll.records` analogue for queue-like use):

```
# RabbitMQ 3.13 — competing consumers with fair dispatch (amqp 5.x,
Python 3.11)
# pip install pika==1.3.2
import pika

conn = pika.BlockingConnection(pika.ConnectionParameters("rabbitmq.inte
rnal"))
ch = conn.channel()

# Durable classic queue — survives broker restart
ch.queue_declare(queue="orders.process", durable=True, arguments={
    "x-dead-letter-exchange": "orders.dlx",
    "x-dead-letter-routing-key": "orders.failed",
})

# Fair dispatch: only 10 unacked messages per consumer at a time
ch.basic_qos(prefetch_count=10)

def on_message(ch, method, props, body):
    try:
        process_order(body)          # may raise
        ch.basic_ack(delivery_tag=method.delivery_tag)
    except TransientError:
        # requeue — will be redelivered, redelivery_count increments
        ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)
    except PermanentError:
        # reject without requeue — broker routes to DLX → DLQ
        ch.basic_nack(delivery_tag=method.delivery_tag, requeue=False)

ch.basic_consume(queue="orders.process",
on_message_callback=on_message)
ch.start_consuming()
```

```
# Amazon SQS — queue with DLQ via AWS CLI / CloudFormation snippet
(SQS, 2024)
# Key parameters that affect queue semantics:
Resources:
  OrdersQueue:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: orders-process
      VisibilityTimeout: 60          # seconds invisible after
delivery; must exceed handler time
      MessageRetentionPeriod: 1209600  # 14 days — after this, message
is dropped
      RedrivePolicy:
        deadLetterTargetArn: !GetAtt OrdersDLQ.Arn
        maxReceiveCount: 5           # after 5 failed receives → DLQ
      # For FIFO:
      # FifoQueue: true

# ContentBasedDeduplication: false  # or true; see Ch02 for dedup
window

  OrdersDLQ:
    Type: AWS::SQS::Queue
    Properties:
      QueueName: orders-process-dlq
      MessageRetentionPeriod: 1209600
```

The distributed-systems lens: prefetch is **consumer-side backpressure made explicit**. Without it, the broker buffers unboundedly in consumer memory (RabbitMQ) or the consumer OOMs. With it, a slow consumer naturally receives fewer messages and the broker retains the backlog durably — which is exactly the decoupling you wanted.

Logs: the append-only, replayable foundation

The contract

A log is an **append-only, totally ordered, replayable** sequence. Producers append; consumers read from an offset. Reading does not remove anything — retention (time or size) does. A consumer tracks its position as an **offset** (a monotonic index per partition), and it can seek backward to re-read.

A single log with a single writer gives total order and trivial reasoning — but a single log is also a single bottleneck and a single failure domain.

Every production log system therefore **partitions** the log: the topic is split into N ordered sub-logs (partitions), each independently ordered and replicated. Ordering is guaranteed *within* a partition; across partitions, there is no order. Producers choose a partition (by key hash, round-robin, or explicit assignment); consumers in a group are assigned partitions exclusively.

This is the core idea behind Kafka, Kinesis, Pulsar, and RabbitMQ Streams. It is worth internalizing:

> **A partitioned log is not a queue with extra features. It is a different data structure.** Queues model *tasks to be done once*; logs model *facts that happened*, retained for any number of readers to consume at their own pace.

Why logs scale

Sequential append to a file is the fastest durable write a disk can do — no random seeks, no B-tree updates, just `write()` to the end. Modern logs exploit this ruthlessly: Kafka and Redpanda write batches sequentially to segment files and rely on the OS page cache and `sendfile(2)` for zero-copy reads. A single partition can sustain hundreds of MB/s on commodity SSDs; adding partitions scales throughput linearly (until the broker or network saturates).

Retention is the other scaling lever. Because reads do not delete, multiple consumer groups can read the same data independently and at different speeds — an ETL group reading from the beginning for a backfill while a serving group reads the tail at near-real-time. The log becomes the **system of record** for events, not just a transport.

Retention, compaction, and the replay superpower

Two retention modes cover most use cases:

- **Time/size retention** — keep N days or N bytes, then delete oldest segments. The default for event streams.
- **Compaction** — for each key, keep only the latest value. Turns the log into a durable, replayable key-value snapshot (Kafka compacted topics, used for changelog and configuration topics).

Replay is the superpower that justifies the complexity. New consumer? Start from offset 0. Bug in yesterday's deploy that corrupted downstream

state? Seek back to yesterday's offset and reprocess. Schema migration? Re-read the log through the new code. No queue can do this without an out-of-band store.

```
flowchart LR
    Prod[Producer\nkey=user-42] --> Part0[Partition 0\n0: {k:A,v:1}\n1:
{k:B,v:1}\n2: {k:A,v:2} ← latest A]
    Prod --> Part1[Partition 1\n0: {k:C,v:1}\n1: {k:D,v:1}]

    Part0 --> SegA[Segment 00000000.log\n+ index + timeindex]
    Part1 --> SegB[Segment 00000000.log]

    SegA --> RetTime[Retention:\n7 days / 10 GB]
    SegA --> RetCompact[Compaction:\nkeep latest per key\nA:1 → A:2
tombstones A:1]

    G1[Consumer Group: serving\nreads tail, offset 3] -.-> Part0
    G2[Consumer Group: backfill\nreads from offset 0] -.-> Part0
    G2 -.-> Part1

    style Part0 fill:#e8f5e9
    style Part1 fill:#e8f5e9
```

Figure 1-4: Partitioned log anatomy — segments, retention, compaction, and independent consumer groups reading the same partitions at different offsets.

```
# Kafka 3.7 — producing to a partitioned log (confluent-kafka 2.4,
Python 3.11)
# pip install confluent-kafka==2.4.0
from confluent_kafka import Producer

conf = {
    "bootstrap.servers": "kafka-1.internal:9092,kafka-2.internal:9092",
    "client.id": "orders-producer",
    "acks": "all",                # wait for ISR ack — see Ch03 for
ISR
    "enable.idempotence": True,
# exactly-once per partition (broker dedup window)
    "compression.type": "lz4",
    "linger.ms": 10,              # micro-batch for throughput
    "batch.size": 65536,
}
p = Producer(conf)

def delivery(err, msg):
    if err:
        print(f"failed: {err}")

# Key determines partition — same key → same partition → total order
for that key
p.produce("orders", key="user-42", value=b'{"order_id": 123, "amount":
5000}', on_delivery=delivery)
p.flush(10)
```

Ordering is per-partition — design for it

The single most common log bug is assuming global order. There is none. If order matters for a group of messages (all events for one order, one user, one device), they must share a **partition key** so they land on the same partition. Random or round-robin partitioning maximizes throughput but destroys any cross-message ordering. Choose deliberately:

Partitioning strategy	Ordering guarantee	Throughput	Use when
Key hash (`hash(key) % N`)	All messages for a key ordered	Skewed if keys are hot	Order matters per entity (user, order, device)
Round-robin / sticky	None	Even, maximal	Order irrelevant, pure fan-out or ingestion

Partitioning strategy	Ordering guarantee	Throughput	Use when
Manual partition	Caller controls	Caller controls	Low-level replication or migration tooling

Hot keys are the distributed-systems catch: one celebrity user or one noisy device can saturate a single partition while others idle. Monitor partition produce rate and consumer lag per partition, not just per topic.

Publish/subscribe: fan-out and filtering

The contract

Pub/sub decouples producers (publishers) from consumers (subscribers) through a **topic** (or exchange + binding). A publisher publishes to a topic without knowing who subscribes; each subscription receives its own copy. One message, many deliveries — **fan-out**.

Two delivery styles:

- **Push** — the broker actively delivers to subscribers (SNS → SQS/HTTP, RabbitMQ push to consumers).
- **Pull** — subscribers poll or long-poll for messages (SQS, Google Pub/Sub pull, Kafka poll loop).

Most managed pub/sub leans push for latency and pull for flow control. Many systems support both (Google Pub/Sub subscriptions can be push or pull; RabbitMQ is push with prefetch as pull-like backpressure).

Topics, subscriptions, and filtering

A topic is a named channel. Subscriptions bind to topics, optionally with **filters**:

- **Subject/routing-key filtering** — NATS subject wildcards (`orders.*.created`), RabbitMQ topic exchange (`orders.#`), SNS filter policies.
- **Attribute filtering** — Google Pub/Sub filter expressions, SNS filter policies on message attributes, EventBridge event patterns.

- **Content filtering** — less common at the broker; usually done in a stream processor (see Ch 4).

Filtering at the broker avoids delivering every message to every subscriber and then discarding — critical when fan-out is large.

```
flowchart LR
    Pub[Publisher] --> Ex[(Topic / Exchange\norders.events)]

    Ex --> S1[Subscription A\nfilter: order.created\n→ fulfillment
service]
    Ex --> S2[Subscription B\nfilter: order.*\n→ analytics pipeline]
    Ex --> S3[Subscription C\nno filter\n→ audit log]

    S1 --> Q1[(Queue / backlog\nper subscription)]
    S2 --> Q2[(Queue / backlog\nper subscription)]
    S3 --> Q3[(Queue / backlog\nper subscription)]

    Q1 --> C1[Consumer A]
    Q2 --> C2[Consumer B]
    Q3 --> C3[Consumer C]

    style Ex fill:#e3f2fd
    style Q1 fill:#fff3e0
    style Q2 fill:#fff3e0
    style Q3 fill:#fff3e0
```

Figure 1-5: Pub/sub fan-out. The topic is the rendezvous; each subscription has its own backlog and ack state, so a slow subscriber does not block others.

The per-subscription queue is the key insight: pub/sub is internally **one queue per subscription** sharing a topic. That is why a slow subscriber does not affect a fast one — their backlogs are independent. It is also why cost scales with subscriptions: each subscription stores a copy (or a reference + separate ack state) of every matching message.

```
# Google Cloud Pub/Sub — publisher and pull subscriber (google-cloud-
pubsub 2.21, Python 3.11)
# pip install google-cloud-pubsub==2.21.0
from google.cloud import pubsub_v1

publisher = pubsub_v1.PublisherClient()
subscriber = pubsub_v1.SubscriberClient()
topic = publisher.topic_path("my-project", "orders.events")
sub   = subscriber.subscription_path("my-project", "orders-
fulfillment")

# Publish with attributes for broker-side filtering
future = publisher.publish(topic, b'{"order_id": 123}', type="order.cre
ated", region="us-east")
msg_id = future.result(timeout=10)

# Pull subscription — long poll, explicit ack (at-least-once)
def callback(msg):
    try:
        handle(msg.data, msg.attributes)
        msg.ack()          # removes from this subscription only
    except TransientError:
        msg.nack()         # redelivers
    except PermanentError:
        msg.ack()          # avoid poison loop — forward to DLQ topic
explicitly
        publish_to_dlq(msg)

streaming_pull = subscriber.subscribe(sub, callback=callback)
streaming_pull.result()  # blocks
```

```
# Redis Streams — log-like pub/sub with consumer groups (redis-py 5.0,
Python 3.11)
# pip install redis==5.0.4
import redis
r = redis.Redis(host="redis.internal", port=6379)

# Append (log) — each entry gets an auto-generated ID (timestamp-seq)
r.xadd("orders", {"order_id": "123", "status": "created"}, maxlen=10000
0)

# Consumer group — queue-like competing consumption over a log, with
replay
r.xgroup_create("orders", "fulfillment", id="0", mkstream=True)

# Read — block 5s, at most 10 entries, claim pending on restart via
XPENDING/XCLAIM
msgs = r.xreadgroup("fulfillment", "worker-1", {"orders": ">"}, count=1
0, block=5000)
for stream, entries in msgs:
    for msg_id, fields in entries:
        try:
            handle(fields)
            r.xack("orders", "fulfillment", msg_id)
        except Exception:
            pass
# left pending — visibility via XPENDING, reclaim via XCLAIM
```

Hybrids and the real world

Textbook taxonomies are clean; production brokers are not. Most systems blend models:

System	Primary model	Hybrid capability
RabbitMQ	Queue (classic, quorum)	Streams (log), topic/fanout exchanges (pub/sub), quorum queues add log-like replication
Apache Kafka	Log	Consumer groups act as queues over the log; no per-message ack, only offset commits
Amazon SQS	Queue	FIFO queues add ordering + dedup; no log replay

System	Primary model	Hybrid capability
Amazon SNS + SQS	Pub/sub + queue	SNS fans out to SQS queues — textbook pub/sub built from queue primitives
Google Pub/Sub	Pub/sub	Pull subscriptions are queues; seek/replay adds log behavior; exactly-once delivery is per-subscription
NATS	Pub/sub (core)	JetStream adds log (streams) with retention, replay, and queue groups (competing consumers)
Redis Streams	Log	Consumer groups add queue semantics; Pub/Sub channels are ephemeral pub/sub with no persistence

The pattern: **queues are easy to build on top of logs** (add consumer groups + offset commits), and **pub/sub is easy to build on top of queues** (one queue per subscription). The reverse is harder — you cannot add replay to a destructive queue without an external store.

This is why many teams converge on a log (Kafka, Redpanda, Kinesis) as the durable center and layer queue and pub/sub semantics on the consumer side. It is also why understanding the *log* deeply — partitions, offsets, retention, ISR — pays for itself even if you primarily think in queue terms. Chapter 3 turns to Kafka as the canonical log implementation.

Choosing among the models

Use this decision tree. Start from the consumer's need, not the producer's.

```
Is there exactly one logical consumer per message?
  YES → Do you need replay / reprocessing?
          YES → Partitioned log (Kafka, Kinesis, RabbitMQ Streams)
          NO  → Queue (SQS, RabbitMQ classic/quorum)
  NO — multiple independent consumers per message
      → Do consumers need independent replay and retention?
          YES → Log with multiple consumer groups (Kafka multi-group)
          NO  → Pub/Sub (SNS+SQS, Google Pub/Sub, NATS)
              → If filtering at the broker matters, prefer topic with f
ilter policy
```

Additional forcing functions:

- **Ordering required?** If total order per entity, you need keyed partitioning (log) or FIFO queue — not a standard queue or unordered pub/sub.
- **Backpressure?** If consumers are bursty or slow, a log's retention and offset-based flow control is more robust than a queue's bounded buffer + DLQ.
- **Exactly-once aspiration?** No broker gives it alone. But logs with transactional producers/consumers (Kafka transactions, see Ch 2–3) get closest, because offsets and writes can be committed atomically. Queues require consumer-side dedup tables.
- **Operations budget?** A managed queue/pub-sub (SQS, Pub/Sub) is nearly zero-ops; a self-managed log (Kafka) is a distributed system you now operate.

Anti-patterns

- **Treating a queue as a log.** Polling a queue for replay, or expecting to re-read after ack. If you need replay, use a log.
- **Assuming global order from a partitioned log.** Order is per-partition. A consumer reading multiple partitions sees interleaving that is not meaningful. If you need cross-partition order, you need a single partition (and you have a bottleneck) or application-level sequencing.
- **One topic/queue per tenant without bounds.** Topic count, partition count, and queue count are broker metadata that must be replicated and watched. Thousands are fine; hundreds of thousands require careful broker sizing and often a control-plane service.

- **Synchronous RPC disguised as messaging.** Producing a message and then polling for a reply message with a correlation ID is a distributed RPC with extra steps and worse failure modes. If you need a response, use RPC; use messaging for fire-and-forget.
- **Ignoring the visibility/ack deadline.** In any at-least-once queue, a handler that exceeds the visibility timeout causes duplicate delivery while the first attempt is still running. The result is two concurrent executions of a non-idempotent handler. Size your timeout above p99 handler latency or use heartbeat extensions (SQS `ChangeMessageVisibility`, RabbitMQ consumer ack timeout).

Key takeaways

- Messaging exists to decouple services in time, space, and failure mode. It trades latency and ordering simplicity for durability and independence.
- Queues give point-to-point, destructive, competing consumption — one message to one consumer. They are the right primitive for tasks.
- Logs give append-only, retained, replayable, per-partition-ordered storage. They are the right primitive for events and system-of-record streams, and they scale by partitioning.
- Pub/sub gives fan-out — one message to every subscription, each with independent backlog and ack state. It is the right primitive for notifications and broadcast.
- Most production brokers are hybrids. Logs can emulate queues (consumer groups) and pub/sub (multiple groups); queues can be composed into pub/sub (one queue per subscription). The log is the most general primitive.
- Ordering, replay, and scaling guarantees are not interchangeable. Choosing the wrong model produces subtle correctness bugs that only appear under failure or load.
- Prefetch, visibility timeouts, and per-subscription backlogs are not tuning knobs — they are the backpressure mechanism that keeps the decoupling honest.

Further reading

- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11 — Stream Processing and Ch. 12 — The Future of Data Systems. O'Reilly, 2017. The definitive treatment of logs vs. queues and the log as system of record.
- Kreps, J. "The Log: What every software engineer should know about real-time data's unifying abstraction." LinkedIn Engineering Blog, 2013. https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying
- Apache Kafka documentation 3.7 — Introduction and design. https://kafka.apache.org/37/documentation.html
- RabbitMQ documentation — Queues, Exchanges, Streams. https://www.rabbitmq.com/docs/queues https://www.rabbitmq.com/docs/streams
- Google Cloud Pub/Sub documentation — Publisher/subscriber model, push vs. pull, filtering, exactly-once delivery. https://cloud.google.com/pubsub/docs
- Amazon SQS and SNS documentation — Queue types, visibility timeout, fan-out pattern. https://docs.aws.amazon.com/sqs/ https://docs.aws.amazon.com/sns/
- NATS JetStream documentation — Streams, consumers, queue groups. https://docs.nats.io/nats-concepts/jetstream
- Redis Streams documentation — `XADD`, `XREADGROUP`, `XPENDING`. https://redis.io/docs/latest/commands/xadd/

Chapter 2 — Delivery Semantics: At-Most, At-Least, Exactly-Once

---

What this chapter covers. "Exactly once" is the most expensive phrase in messaging. Every broker advertises it, almost none delivers what the phrase seems to promise, and the gap between marketing and mechanism causes real outages. This chapter makes delivery semantics precise: what at-most-once, at-least-once, and effectively-once (often mislabeled exactly-once) actually guarantee, where the guarantee lives (broker, consumer, or end-to-end), and what you must build to get the

behavior your domain requires. We start from the impossibility that forces the taxonomy — the sender's ambiguity when an acknowledgment is lost — then define each semantic with its cost and failure mode, show how brokers implement redelivery and deduplication (Kafka 3.7 idempotent producer and transactions, SQS FIFO dedup, RabbitMQ confirms), and build the consumer-side patterns that determine the real guarantee: ack ordering, deduplication tables, the transactional outbox, and the transactional consume-transform-produce loop. The chapter closes with a decision framework and the operational cost of each choice.

Learning goals — after this chapter you should be able to:

- State why exactly-once *delivery* is impossible over a lossy network and why the achievable property is exactly-once *effect*.
- Define at-most-once, at-least-once, and effectively-once precisely, including what is lost or duplicated under each and what each costs in latency, throughput, and complexity.
- Explain the broker mechanisms that implement each semantic: fire-and-forget, ack + redelivery, and deduplication windows.
- Describe Kafka 3.7's idempotent producer (PID + sequence) and transactions (transactional ID, transaction coordinator, LSO) and what they do and do not guarantee.
- Implement the consumer-side dedup table and the offset-commit ordering discipline that actually determines delivery semantics.
- Choose correctly among the three semantics for a given workload and explain the choice in terms of domain tolerance for loss vs. duplication.

> **Boundary note.** The *theory* of idempotency — its formal definition `f(f(x)) = f(x)`, the conversion of non-idempotent operations into idempotent ones by naming the effect, the atomic check-and-record protocol, storage choices for idempotency keys, and Stripe-style key discipline — is developed in Vol 6, Chapter 9 — Idempotency, Deduplication, and Exactly-Once. This chapter treats *broker mechanics*: what Kafka, SQS, RabbitMQ, and Google Pub/Sub actually promise on the wire, how their deduplication windows work, and what consumer-side code must add to reach end-to-end guarantees. Read Vol 6 Ch 9 for the why and the application protocol; read this chapter for the broker contract and the wiring.

The forcing function: silence is ambiguous

Delivery semantics exist because networks lose messages and processes crash. The minimal case from Vol 6 Ch 9 is worth restating here in broker terms:

A producer sends a message and waits for the broker's ack. Three indistinguishable cases produce the same observation — silence:

1. The message was lost on the way to the broker. Nothing was written.
2. The message was written and the ack was lost on the way back. The message *is* in the log/queue.
3. Everything is slow. The message may be written at any moment.

The producer cannot distinguish (1) from (2) without additional information, and no amount of waiting resolves (3) in an asynchronous network. This is the Two Generals problem in work clothes. The consequence is a forced choice:

- If the producer **does not retry**, a message lost in case (1) is gone forever — **at-most-once**.
- If the producer **does retry**, a message already written in case (2) will be written again — **at-least-once**, with duplicates.

There is no third option at the transport layer. Exactly-once delivery — "every message is delivered once and only once, regardless of failures" — would require the producer to solve the indistinguishability. It cannot. What systems label "exactly once" is therefore always **effectively once**: at-least-once delivery plus deduplication and idempotent handling that make duplicates invisible to the application.

```
sequenceDiagram
    participant Prod as Producer
    participant Net as Network
    participant Broker as Broker
    Prod->>Net: send(msg, seq=7)
    Net->>Broker: delivered — written to log
    Broker->>Net: ack(seq=7)
    Note over Net: ack LOST
    Prod->>Prod: timeout — did seq=7 commit?
    Note over Prod: Cannot tell (1) from (2).
Retry → duplicate.
Don't retry → possible loss.
    Prod->>Net: retry send(msg, seq=7)
    Net->>Broker: delivered again
    Broker->>Broker: must deduplicate
or consumer sees duplicate
```

Figure 2-1: The producer's ambiguity. After a timeout, retrying risks duplication and not retrying risks loss. The broker's deduplication window and the consumer's dedup table exist to contain this.

```
stateDiagram-v2
    [*] --> Sent: producer sends
    Sent --> Acked: ack received — done
    Sent --> Uncertain: timeout — no ack
    Uncertain --> Retried: retry (at-least-once path)
    Uncertain --> Dropped: give up (at-most-once path)
    Retried --> Acked: ack on retry
    Retried --> Uncertain: timeout again — retry loop
    Retried --> Duplicated: broker already had it
now has two copies
unless deduped
    Dropped --> Lost: message never delivered
    Acked --> [*]
    Lost --> [*]
    Duplicated --> [*]
```

Figure 2-2: Delivery state machine. The uncertain state is unavoidable; the outgoing edge chosen there defines the semantic.

The three semantics, precisely

At-most-once: fire and forget

Guarantee: each message is delivered **zero or one** times. No retries on the send path; no redelivery on the consume path.

Mechanism: producer sends with `acks=0` (Kafka) or `autoAck=true` / `noAck` (RabbitMQ, SQS without visibility retry), and moves on. On the consume side, the message is deleted or its offset committed *before* processing, or processing failures are ignored.

What is lost: any message that encounters a transient failure — network blip, broker crash before fsync, consumer crash before processing completes.

When it is correct: when loss is acceptable and duplication or latency from retries is not. Telemetry, metrics, best-effort presence, high-volume sampling where a few dropped points do not change the signal. Never for money, inventory, or state transitions that must not be lost.

```
Producer                          Broker
Consumer
  |--- send (acks=0) ------------->|                                 |
  |   (no ack expected)             |--- deliver (auto-ack) --------->|
  |                                 |   (deleted on send)             |-
-- process
  |                                 |
|    (if crash, message is gone)
```

Cost: minimal latency and maximal throughput — no ack round-trip, no retry state, no dedup table.

At-least-once: retry and redeliver

Guarantee: each message is delivered **one or more** times. No message is lost due to transient failures, but duplicates are possible and expected.

Mechanism: producer retries on timeout/error until acked; broker retains until consumer acks; on consumer failure or visibility timeout, the broker redelivers.

What duplicates: every retry that races with a successful but unacked write, and every consumer crash or timeout between delivery and ack.

Duplicates are not rare edge cases — they are the normal consequence of the guarantee.

When it is correct: when loss is unacceptable and the consumer can tolerate or deduplicate duplicates. This is the **default correct choice** for most backend work: order processing, notifications, ETL, event-driven state machines.

The consumer discipline that makes at-least-once safe:

1. **Process, then ack** — commit the offset / ack the message *after* the side effect is durable. Acking before processing turns at-least-once into at-most-once on crash.
2. **Make the handler idempotent or deduplicated** — so redelivery is harmless. See Vol 6 Ch 9 for the idempotency-key protocol; this chapter shows the broker side.
3. **Bound redelivery** — DLQ after N attempts (see Ch 8) so poison messages do not loop forever.

In practice this discipline interacts with batching and concurrency. A consumer that processes messages in parallel (thread pool, async handlers) must not ack a batch until *all* messages in the batch have been processed — otherwise a crash after acking the batch but before the slowest handler finishes loses that message. Either ack per-message after each handler completes, or track per-message completion within the batch and commit the high watermark only after the last one succeeds. The same applies to Kafka's `max.poll.records`: a batch of 500 that acks after the first 10 is at-most-once for the remaining 490.

Effectively-once (what brokers call "exactly once"): at-least-once plus dedup

Guarantee: each message *affects the system* once, even though it may be delivered more than once on the wire. The application cannot observe duplication.

Mechanism: at-least-once delivery plus **two** deduplication layers:

- **Broker-side dedup window** — the broker suppresses duplicate writes within a bounded window (Kafka PID + sequence per partition, SQS FIFO 5-minute dedup, Pub/Sub exactly-once ack with retry window).

- **Consumer-side dedup table** — the consumer (or sink) records processed message IDs and skips duplicates durably, atomically with the side effect.

Neither layer alone suffices. The broker window is time- or size-bounded and partition-scoped; the consumer table is durable and cross-partition but must be maintained. True end-to-end "exactly once" is the composition of both, plus transactional commit of offsets with side effects where the broker supports it.

> **Why vendors say "exactly once" and mean effectively once.** Kafka's "exactly-once semantics" (EOS) and Google Pub/Sub's "exactly-once delivery" both mean: within their stated conditions (Kafka: idempotent producer + transactions + `isolation.level=read_committed` + correct consumer commit; Pub/Sub: ack deadline + dedup window + exactly-once ack semantics enabled), a consumer that follows the prescribed protocol will not *observe* duplicates. The wire still carries at-least-once; the dedup makes it look like once.

Semantic	Producer behavior	Broker behavior	Consumer behavior	Duplicate?	Loss?	Cost
At-most-once	No retry	Delete on send	Ack before or without processing	No	Yes — on any failure	Lowes
At-least-once	Retry until ack	Retain until ack; redeliver on timeout	Process then ack; handler must be idempotent/ deduped	Yes — on retry or crash	No (modulo retention)	Moder retries DLQ
Effectively-once	Retry with dedup identity	Dedup window + transactions	Dedup table + atomic offset commit	Wire yes, effect no	No	Highe transa dedup storag LSO la

Broker mechanisms in detail

Kafka 3.7: idempotent producer and transactions

Kafka's path from at-least-once to effectively-once is the most instructive because it exposes the mechanism.

Idempotent producer (per-partition, intra-session dedup). When `enable.idempotence=true` (Kafka 3.7, default with `acks=all`), the broker assigns each producer a **Producer ID (PID)** and the producer tags every batch with a **monotonic sequence number** per partition. The broker's log keeps `lastSequence[PID][partition]` and rejects any batch whose sequence is not `last + 1` — a duplicate retry is detected and acked without appending.

```
# Kafka 3.7 — idempotent producer (librdkafka / confluent-kafka 2.4,
Python 3.11)
# pip install confluent-kafka==2.4.0
from confluent_kafka import Producer

conf = {
    "bootstrap.servers": "kafka-1.internal:9092,kafka-2.internal:9092",
    "client.id": "payments-producer",
    "acks": "all",                      # required for idempotence
    "enable.idempotence": True,         # enables PID + sequence dedup
per partition
    "retries": 5,
    "max.in.flight.requests.per.connection": 5,  # safe with
idempotence (was 1 pre-2.0)
    "compression.type": "lz4",
    "transactional.id": "payments-tx-1",  # enables transactions (EOS)
— see below
}
p = Producer(conf)
p.init_transactions(timeout=10)          # registers transactional.id
with coordinator

# Transactional produce — atomically commit messages + offsets
p.begin_transaction()
p.produce("payments", key="order-123", value=b'{"amount": 5000}')
# ... produce to other topics/partitions ...
p.send_offsets_to_transaction(
    [{"topic": "payments", "partition": 0, "offset": 42}],  # consumer
offsets, if consuming
    group_metadata=None,
)
p.commit_transaction(timeout=10)        # or abort_transaction() on
error
# Consumers with isolation.level=read_committed skip aborted messages
(LSO)
```

Limits that matter:

- **Scope:** per `(PID, partition)` — not cross-partition. Producing the same logical message to two partitions creates two independent sequences.
- **Window:** bounded by broker retention of PID state (`transactional.id.expiration.ms` 7 days by default; PID sequence state lives in memory + log). A producer that restarts with a new PID gets a new sequence space — duplicates across restarts require the transactional ID fence (see below).

- **Throughput:** `acks=all` + ISR replication adds latency; batching (`linger.ms`, `batch.size`) amortizes it.

Transactions (cross-partition, consume-transform-produce). Transactions extend the guarantee across partitions and across the consume-produce loop. The producer registers a `transactional.id`; the **transaction coordinator** (a broker, elected via KRaft metadata quorum) fences old producers with the same ID (epoch bump) and manages two-phase commit. On `commit_transaction`, the coordinator writes transaction markers; consumers with `isolation.level=read_committed` advance their **Last Stable Offset (LSO)** past aborted transactions, hiding them.

What transactions give you: atomic multi-partition produce + atomic offset commit — the consume-transform-produce loop commits "I consumed offset N and produced result R" atomically. A crash between consume and produce either fully commits or fully aborts; no partial effect, no duplicate downstream write.

What they do not give you: exactly-once interaction with an external system (database, HTTP service). If the transaction produces to Kafka and also writes to Postgres, the two commits are not atomic without an outbox (see Ch 6). And LSO lag means `read_committed` consumers see higher end-to-end latency when transactions are open — monitor `kafka.server:type=BrokerTopicMetrics` and transaction coordinator metrics.

SQS, RabbitMQ, and Google Pub/Sub

Broker	At-most-once	At-least-once	Effectively-once mec
Amazon SQS Standard	Not offered — always at-least-once	Default. Visibility timeout + redelivery, DLQ after `maxReceiveCount`	Client-side dedup table broker dedup
Amazon SQS FIFO	Not offered	Default within message group ordering	5-minute dedup windov `MessageDeduplicatio` `ContentBasedDedupli` SHA-256 of body); 20k dedup IDs per queue

Broker	At-most-once	At-least-once	Effectively-once mec
RabbitMQ 3.13	`autoAck=true` or `basic.get` without ack	`manualAck`, publisher confirms (`confirm.select`), redelivery with `redelivered` flag	No broker dedup — co table or quorum-queue plugin; publisher confi nacks but not dedup
Google Pub/Sub	Not offered for pull/push subscriptions	Default. Ack deadline (10s–600s) + redelivery, DLQ topic	Exactly-once delivery (subscription): server-si + retry window; still re idempotent handler for window edge
Kafka 3.7	`acks=0` produce, `enable.auto.commit=true` before processing	`acks=all` + manual offset commit after processing	Idempotent producer (transactions + `isolation.level=rea` + consumer dedup tabl side

```
# SQS FIFO — broker dedup window (boto3 1.35, Python 3.11)
# pip install boto3==1.35.0
import boto3, hashlib, json
sqs = boto3.client("sqs", region_name="us-east-1")
payload = json.dumps({"order_id": "ord-123", "amount": 5000})
sqs.send_message(
    QueueUrl="https://sqs.us-east-1.amazonaws.com/123/orders.fifo",
    MessageBody=payload,
    MessageGroupId="user-42",
# ordering scope — one in-flight per group
    MessageDeduplicationId=hashlib.sha256(payload.encode()).hexdigest()
,  # 5-min window
)
# Duplicate send within 5 minutes with same dedup ID → broker returns
same MessageId, no enqueue
```

```
# RabbitMQ 3.13 — publisher confirms + manual ack (pika 1.3.2, Python
3.11)
# pip install pika==1.3.2
import pika
conn = pika.BlockingConnection(pika.ConnectionParameters("rabbitmq.inte
rnal"))
ch = conn.channel()
ch.confirm_delivery()  # enable publisher confirms — broker acks each
publish
ch.queue_declare(queue="orders.process", durable=True)
# publish — wait for confirm (at-least-once)
if not ch.basic_publish(exchange="", routing_key="orders.process",
body=b'{"id":123}',

properties=pika.BasicProperties(delivery_mode=2)):  # persistent
    raise RuntimeError("nack — broker did not accept, retry")
# consume — manual ack after processing
for method, props, body in ch.consume("orders.process", inactivity_time
out=1):
    if body is None:
        continue
    try:
        handle(body)
        ch.basic_ack(method.delivery_tag)
    except TransientError:
        ch.basic_nack(method.delivery_tag, requeue=True)
    except PermanentError:
        ch.basic_nack(method.delivery_tag, requeue=False)  # → DLX /
DLQ
```

The consumer is the guarantee

Whatever the broker promises, the consumer determines the effective semantic. Two disciplines matter above all others.

Ack ordering

```
WRONG — at-most-once on crash:          RIGHT — at-least-once, safe on
crash:

on message(msg):                         on message(msg):
    ack(msg)        // deleted               result = handle(msg)  // ma
y crash here
    handle(msg)     // if crash, lost      ack(msg)              // onl
y after success
```

For log offsets, the same rule: **commit offset after processing**, and commit the *next* offset to read. Auto-commit before processing (`enable.auto.commit=true` with Kafka's default 5s interval) is at-most-once on crash — a rebalance or crash after commit but before processing loses the message.

```
# Kafka 3.7 — correct offset discipline (confluent-kafka 2.4, Python
3.11)
from confluent_kafka import Consumer

conf = {
    "bootstrap.servers": "kafka-1.internal:9092",
    "group.id": "orders-processor",
    "enable.auto.commit": False,
# manual commit — we control the guarantee
    "isolation.level": "read_committed",   # hide aborted transactions
    "auto.offset.reset": "earliest",
}
c = Consumer(conf)
c.subscribe(["orders"])

while True:
    msg = c.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        continue  # handle
    try:
        handle(msg.value(), msg.key())     # side effect — must be
idempotent/deduped
        c.commit(msg)                      # commit offset only after
success
    except TransientError:
        # don't commit — will be redelivered after poll timeout /
rebalance
        continue
    except PermanentError:
        publish_to_dlq(msg)                # see Ch08
        c.commit(msg)                      # skip poison message —
commit to advance
```

The deduplication table: the reliable backstop

When the sink is an external system (database, downstream HTTP), broker transactions do not help — the consumer must deduplicate. The canonical pattern, from Vol 6 Ch 9, applied at the consumer:

```
-- Postgres 16 — consumer dedup table, committed atomically with the
effect
-- The dedup key is the producer's idempotency key or message ID, not
the offset

CREATE TABLE consumer_dedup (
    message_id  TEXT PRIMARY KEY,          -- idempotency key from
producer
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Optional TTL for bounded storage: DELETE WHERE processed_at < now()
- interval '7 days'
-- Size the TTL beyond max redelivery window + DLQ max retention

-- Atomic insert-and-process: the INSERT is the dedup gate
BEGIN;
INSERT INTO consumer_dedup(message_id) VALUES ('msg-abc-123')
ON CONFLICT DO NOTHING;
-- Check row_count: 0 means duplicate — skip processing, still commit
offset
-- 1 means first time — process and commit

-- ... perform side effect (INSERT INTO orders ...) in same
transaction ...

COMMIT;
-- Only after COMMIT succeeds, commit the Kafka offset / ack the SQS
message
```

The transactional variant folds the offset commit into the same database transaction when the broker supports it (Kafka `send_offsets_to_transaction`), or commits the offset to the *same database* as the effect (storing `__consumer_offsets` in Postgres rather than in Kafka) — the outbox pattern of Chapter 6 is this idea generalized.

```
flowchart LR
    Msg[Message\nid=msg-abc] --> Check{INSERT dedup\nON CONFLICT?}
    Check -- "inserted = 1\n(first time)" --> Effect[Apply side
effect\nINSERT orders ...]
    Effect --> CommitOffset[Commit offset / ack]
    Check -- "inserted = 0\n(duplicate)" --> Skip[Skip side effect]
    Skip --> CommitOffset
    CommitOffset --> Done[Done — effect applied once]
    style Check fill:#fff3e0
    style Effect fill:#e8f5e9
    style Skip fill:#e3f2fd
```

Figure 2-3: Consumer-side dedup — the INSERT is the atomic gate. Duplicate deliveries take the skip branch; the effect is applied exactly once.

End-to-end effectively-once requires both sides

```
Producer                          Broker
Consumer + Sink
  |--- send (idempotent, retry) -->|                                  |
  |<-- ack (PID+seq dedup) --------|                                  |
  |                                 |--- deliver (at-least-once) ---->|
  |                                 |                                  |-
-- INSERT dedup (atomic gate)
  |                                 |                                  |-
-- side effect (if first time)
  |                                 |<-- ack / commit offset ---------|
```

The producer dedup prevents duplicate *writes* from retries; the consumer dedup prevents duplicate *effects* from redelivery. Remove either and a failure window reopens. This is why "the broker gives exactly once" is never the whole story — the end-to-end property is a *protocol* between producer discipline, broker window, and consumer discipline.

Choosing a semantic

Workload	Tolerates loss?	Tolerates duplication?	Right semantic	Handler requirement
Metrics / telemetry sampling	Yes	Yes (counter increment is idempotent)	At-most-once or at-least-once	None or idempotent increment
Email / push notification	No	No (user sees two emails)	Effectively-once	Dedup table on (user, campaign, idempotency key)
Payment / charge	No	No	Effectively-once	Idempotency key on payment intent (Vol 6 Ch 9)

Workload	Tolerates loss?	Tolerates duplication?	Right semantic	Handler requirement
Order state machine	No	No (double decrement)	Effectively-once	Dedup + conditional state transition
Search index / cache refresh	No	Yes (last write wins)	At-least-once	Idempotent upsert (natural)
Log ingestion / ETL	No	Yes if sink is append + dedup	At-least-once + sink dedup	Dedup on event ID at sink

Rule of thumb: **default to at-least-once with an idempotent handler**. It is the only semantic that survives transient failures without data loss, and idempotence is cheaper than reasoning about which messages can be lost. Reserve at-most-once for sampled signals and effectively-once transactions for money-adjacent paths where the dedup and LSO cost is justified.

Failure scenarios and what each semantic does

Two concrete failures make the taxonomy visceral:

Scenario A — producer retry after ack loss. The producer sends `msg-42` to a Kafka partition with `acks=all`. The broker appends at offset 100, replicates to ISR, and sends the ack — which is lost. The producer times out and retries. With `enable.idempotence=false`, offset 101 is a duplicate `msg-42`; the consumer will see both unless it dedups. With `enable.idempotence=true`, the broker sees `PID=7, seq=4` already at offset 100 and returns success for offset 100 without appending — a wire duplicate suppressed at the log.

Scenario B — consumer crash between processing and commit. A consumer polls `msg-42` at offset 100, inserts the order into Postgres, and crashes before `commit(offset 101)`. On restart (or rebalance), the group re-delivers offset 100. Without a dedup table, the order is inserted twice. With the `INSERT ... ON CONFLICT DO NOTHING` gate in the same Postgres transaction as the order insert, the second delivery hits the dedup row

and skips the effect — the offset is then committed and the group advances.

In both scenarios, at-most-once would have lost `msg-42` (no retry in A, committed before processing in B), at-least-once would have duplicated the effect, and effectively-once — broker dedup in A, consumer dedup in B — makes the duplication invisible.

Retry budgets and backoff: the distributed-systems cost of at-least-once

At-least-once is not free. Every retry is load, and naive retries amplify failures.

- **Exponential backoff with jitter** — `retry.backoff.ms` (default 100ms in `librdkafka`) with full jitter prevents thundering herds when a broker recovers and every producer retries at once. Without jitter, synchronized retries create a second outage.
- **Delivery timeout as a circuit breaker** — `delivery.timeout.ms` (120s default) bounds total retry time. A producer that retries forever holds memory and can OOM; one that times out too quickly surfaces spurious failures. Size it from p99 produce latency plus replication lag.
- **Consumer-side retry budgets** — a consumer that nacks and redelivers on every transient error can loop faster than it makes progress. Cap retries per message (SQS `maxReceiveCount`, RabbitMQ `x-death` count, Kafka manual DLQ after N attempts — see Ch 8) and add backoff between deliveries (SQS delay queues, RabbitMQ delayed exchange, Kafka retry topics with timestamp-based re-enqueue).
- **Idempotence makes retries safe, but not cheap** — the dedup table and transaction coordinator add latency (LSO lag) and storage. Measure duplicate rate under failure injection (kill a broker, pause a consumer) to verify the budget holds.

Operational signals to watch:

- **Duplicate rate** — `redelivered` flag (RabbitMQ), `duplicate` ack (Pub/Sub), consumer dedup hit rate. A rising duplicate rate usually means visibility/ack deadlines are too short or consumers are too slow.

- **LSO lag** (Kafka transactions) — `kafka.server:type=BrokerTopicMetrics,name=...` and `LogEndOffset - LastStableOffset`. Open transactions hold back `read_committed` consumers.
- **DLQ depth** — the poison-message rate. See Chapter 8.

Key takeaways

- Exactly-once delivery is impossible over a lossy network; the achievable property is exactly-once *effect* via at-least-once delivery plus deduplication and idempotent handling.
- At-most-once loses messages on any failure; at-least-once duplicates on retry or crash; effectively-once composes at-least-once with dedup to make duplicates invisible, at higher cost.
- Producer dedup (Kafka PID+sequence, SQS FIFO dedup ID) suppresses duplicate *writes* within a bounded window; consumer dedup (INSERT-gated table, transactional offset commit) suppresses duplicate *effects* durably.
- Ack/commit ordering determines the real guarantee: ack before processing is at-most-once on crash; process then ack is at-least-once.
- Kafka 3.7 transactions give atomic consume-transform-produce across partitions via the transaction coordinator and LSO, but add latency and do not extend to external sinks — there the outbox (Ch 6) and consumer dedup table do.
- Default to at-least-once with idempotent handlers; pay for effectively-once transactions only where duplication is unacceptable and the cost is measured.

Further reading

- Vol 6, Chapter 9 — Idempotency, Deduplication, and Exactly-Once. Formal idempotency theory, the idempotency-key protocol, and the atomic check-and-record pattern.
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11. The end-to-end argument for exactly-once and why it is an application property.

- Apache Kafka documentation 3.7 — Producer idempotence and transactions, `isolation.level`. https://kafka.apache.org/37/documentation.html#semantics
- Amazon SQS documentation — FIFO queues, deduplication interval, `MessageDeduplicationId`. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html
- RabbitMQ documentation 3.13 — Publisher confirms, consumer acknowledgements, quorum queues. https://www.rabbitmq.com/docs/confirms https://www.rabbitmq.com/docs/quorum-queues
- Google Cloud Pub/Sub documentation — Exactly-once delivery. https://cloud.google.com/pubsub/docs/exactly-once-delivery
- Kreps, J. "Exactly-once Support in Apache Kafka." Confluent Blog, 2017 (updated for transactions and KRaft). https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/

Chapter 3 — Apache Kafka Architecture

What this chapter covers. Kafka is the most widely deployed log in backend systems and the most commonly misunderstood. Teams adopt it as "a queue," discover partitions, lose ordering, misconfigure `acks`, and learn about ISR the day a broker dies. This chapter builds Kafka from the metal up as a **partitioned, replicated, append-only log** — what it stores, how it replicates, how it serves reads and writes, and how the control plane has evolved from ZooKeeper to KRaft. We cover the durable core (topics, partitions, segments, indexes, zero-copy reads), the replication protocol (leader, followers, ISR, high watermark, `acks` and `min.insync.replicas`), the producer and consumer paths (batching, compression, fetch, consumer groups and cooperative rebalancing), and the operational surface that determines whether the cluster survives a failure (KRaft metadata quorum, rack awareness, unclean leader election, tiered storage). Every configuration is pinned to **Kafka 3.7** (KRaft stable, Scala 2.13, Java 17) and uses `confluent-kafka` / `librdkafka` and `kafka` CLI as the concrete client.

Learning goals — after this chapter you should be able to:

- Describe the storage layout: topic → partitions → segments → batch/index/timeindex, and how sequential I/O and page cache make the log fast.
- Explain the replication protocol: leader, followers, ISR, high watermark (HWM), leader epoch, and how `acks` and `min.insync.replicas` trade durability for availability.
- Configure a producer and consumer correctly for each delivery semantic (Ch 2) in Kafka 3.7, including idempotence and transactions.
- Explain consumer groups, partition assignment, and the cooperative rebalancing protocol, including why `max.poll.interval.ms` and `session.timeout.ms` exist.
- Describe the KRaft metadata quorum (Raft over the `__cluster_metadata` log) and how it replaces ZooKeeper.
- Reason about failure: broker loss, ISR shrinkage, unclean leader election, and the durability/availability trade-off at each `acks` setting.
- Operate the cluster: topic sizing, partition count, replication factor, rack awareness, monitoring (JMX/Metrics), and tiered storage.

> **Boundary note.** Vol 6, Chapter 9 — Idempotency, Deduplication, and Exactly-Once develops the *theory* of idempotency (formal definition, naming the effect, the atomic check-and-record protocol, and Stripe-style key discipline). Chapter 2 in this volume maps that theory onto *broker delivery semantics* generally. This chapter is narrower still: the *Kafka broker mechanism* — PID + sequence, transaction coordinator, LSO, and ISR — that implements those semantics on a partitioned log. For system-level idempotency design, read Vol 6 Ch 9; for the broker contract, read Ch 2; for the Kafka wire, read here.

The log, physically

Topics, partitions, segments

A **topic** is a named log. It is split into **partitions** — ordered, append-only sub-logs numbered `0 … N-1`. Each partition is stored as a sequence of **segment files** on the leader broker's disk:

```
/var/lib/kafka/data/orders-0/
  00000000000000000000.log        # segment — batches of records
  00000000000000000000.index      # offset → file position
  00000000000000000000.timeindex  # timestamp → offset
  00000000000000000123.log        # next segment after roll
  00000000000000000123.index
  leader-epoch-checkpoint
```

The base filename is the **base offset** of the first batch in the segment. Segments roll on size (`segment.bytes`, default 1 GiB) or time (`segment.ms`, default 7 days). The **active segment** is the one currently appended to; closed segments are immutable and eligible for retention or tiered-storage offload.

A **record** (Kafka 3.7, magic 2) carries `timestamp`, `key`, `value`, `headers`, `offset` (assigned by the leader, monotonic per partition), and `sequence` (for idempotent producers). Records are written in **batches** (a `RecordBatch` with CRC, compression, and producer/sequence metadata) — batching is the throughput lever.

Why sequential I/O is fast

The log does almost no random I/O. Writes are sequential appends to the active segment; reads are sequential scans from an offset, often served from the OS **page cache** without hitting disk. Consumers use `sendfile(2)` (zero-copy) to transfer bytes from page cache to socket without copying through userspace. The result: a single partition can sustain hundreds of MB/s on commodity SSDs, and throughput scales linearly with partitions (until network or broker CPU saturates).

Three indexes make random access cheap when needed:

- **Offset index** — sparse map `offset → file position`, one entry per `index.interval.bytes` (default 4 KiB of batch data) so it stays small.
- **Time index** — `timestamp → offset` for `offsetsForTimes` and retention by timestamp.

- **Leader epoch checkpoint** — `leaderEpoch → startOffset` for truncation after failover (see ISR section).

```
flowchart TB
    Topic[Topic: orders
partitions=6, RF=3]
    Topic --> P0[Partition 0
leader: broker-1]
    Topic --> P1[Partition 1
leader: broker-2]
    Topic --> PN[Partition 5
leader: broker-3]

    P0 --> Seg0A[Segment baseOffset 0
00000.log + .index + .timeindex
closed, 1 GiB]
    P0 --> Seg0B[Segment baseOffset 12345
active — appends here]
    P0 --> F0A[Follower: broker-2
replica]
    P0 --> F0B[Follower: broker-3
replica]

    P0 -.-> ISR0["ISR: {1,2,3}
HWM: 12340
LEO: 12348"]

    style P0 fill:#e8f5e9
    style Seg0B fill:#fff3e0
    style ISR0 fill:#e3f2fd
```

Figure 3-1: Topic → partitions → segments. Each partition has one leader that serves all reads and writes; followers replicate the log; the ISR tracks in-sync replicas and the HWM marks committed data.

Replication: ISR, high watermark, and the durability contract

Leader, followers, ISR

For each partition, one replica is the **leader** and the rest are **followers**. All produces and reads go through the leader. Followers fetch from the leader (same fetch path as consumers, but with `replica.fetch.*` tuning) and append to their local log. A follower that has caught up to the leader's **Log End Offset (LEO)** — the offset of the next write — is **in-**

sync and belongs to the **In-Sync Replica set (ISR)**. Liveness is tracked by `replica.lag.time.max.ms` (default 30s in Kafka 3.7): a follower that has not fetched within that window is ejected from the ISR.

```
Partition 0, RF=3, ISR={1,2,3}, LEO=100 on all replicas:

broker-1 (leader)  log: [0 ... 99][100 LEO]   HWM=100
broker-2 (follower) log: [0 ... 99][100 LEO]  in ISR — caught up
broker-3 (follower) log: [0 ... 99][100 LEO]  in ISR — caught up

After broker-3 stalls (GC pause):

broker-3 log: [0 ... 90]                      lag 10, still within repl
ica.lag.time.max.ms → ISR
... 30s later without fetch → ejected → ISR={1,2}, HWM still 100
(needs min ISR acks)
```

High watermark and committed data

The **high watermark (HWM)** is the offset of the last *committed* record — the highest offset such that every replica in the ISR has that record. Consumers only see data up to the HWM (or LSO when transactions are active — see below). The HWM is the durability line: data above the HWM is not yet replicated to the ISR and can be lost on leader failure.

```
Leader log:  offset 0 1 2 3 4 5 6 7 8 9  LEO=10
Follower A:  offset 0 1 2 3 4 5 6 7        LEO=8
Follower B:  offset 0 1 2 3 4 5            LEO=6
ISR={leader, A, B}  (all within lag window)
HWM = 6  (min LEO across ISR = 6) — offsets 6..9 are uncommitted
Consumer fetch returns up to 6; offsets 7..9 invisible until B catches
up
```

With `acks=all`, a produce is acked only after the HWM advances past the new records — i.e., after every ISR replica has them. This is the durability guarantee; the producer blocks (or times out at `delivery.timeout.ms`) until it holds.

`acks` and `min.insync.replicas`: the durability/availability dial

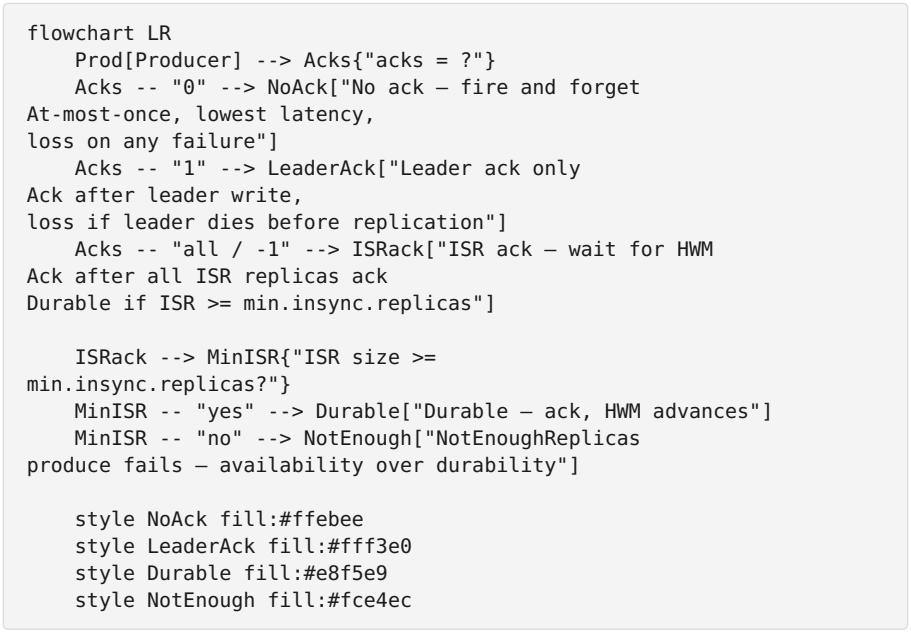


Figure 3-2: The acks/min.insync.replicas dial. acks=all with min.insync.replicas=2 on RF=3 survives one replica loss durably; a second loss makes the partition unavailable for writes rather than risking data loss.

acks	min.insync.replicas (RF=3)	Durability	Availability on one replica down	Availability on two replicas down
`0`	—	None — loss on leader crash	Available (no ISR check)	Available
`1`	—	Leader durable only	Available	Available
`all`	`1`	At least one replica durable	Available	Available

`acks`	`min.insync.replicas` (RF=3)	Durability	Availability on one replica down	Availability on two replicas down
`all`	`2`	At least two replicas durable	Available (ISR=2 meets min)	**Unavailable** — `NotEnoughReplicas`
`all`	`3`	All three durable	**Unavailable** — ISR=2 < 3	Unavailable

The production default for data you cannot afford to lose: `acks=all`, `min.insync.replicas=2`, `replication.factor=3`. This survives one replica failure with no data loss and remains writable; a second failure makes the partition unwritable (fail-closed) rather than risking loss. Setting `min.insync.replicas=1` keeps the partition writable through two failures but admits loss if the sole ISR replica dies before a new follower catches up.

Leader election, leader epoch, and unclean election

When a leader dies, the controller (KRaft quorum, see below) elects a new leader from the ISR. **Leader epoch** (a monotonic integer per partition, stored in `leader-epoch-checkpoint`) fences stale leaders: a new leader increments the epoch, and followers truncate any divergent tail written by a zombie old leader that was partitioned but still serving writes.

Unclean leader election — promoting a replica outside the ISR — is disabled by default (`unclean.leader.election.enable=false`) because it can lose committed data (the out-of-sync replica may be missing records up to the HWM). Enable it only for availability-at-all-costs topics where loss is preferable to unavailability, and monitor `UnderReplicatedPartitions`.

The write path: producer

```
sequenceDiagram
    participant App as Application
    participant Prod as Producer
(librdkafka)
    participant Leader as Partition Leader
(broker-1)
    participant Follower as Follower
(broker-2)
    participant ISR as ISR / HWM

    App->>Prod: produce(topic, key, value)
    Prod->>Prod: partition = hash(key) % N
batch by partition + linger.ms
    Prod->>Prod: compress (lz4/zstd), assign sequence
PID + seq per partition
    Prod->>Leader: ProduceRequest (batch, acks=all)
    Leader->>Leader: append to active segment
assign offsets, CRC
    Leader->>Follower: Fetch (replication)
    Follower->>Leader: FetchResponse (ack LEO)
    Leader->>ISR: advance HWM when ISR acks
    Leader->>Prod: ProduceResponse (offsets, error=None)
    Prod->>App: delivery callback (acked)
```

Figure 3-3: The produce path. Batching and compression happen client-side; durability is the HWM advance after ISR replication.

```
# Kafka 3.7 — producer for each semantic (confluent-kafka 2.4, Python
3.11)
# pip install confluent-kafka==2.4.0
from confluent_kafka import Producer

# At-least-once with durability (default for important data)
durable = Producer({
    "bootstrap.servers": "kafka-1.internal:9092,kafka-2.internal:9092,k
afka-3.internal:9092",
    "client.id": "orders-producer",
    "acks": "all",
    "enable.idempotence": True,            # PID+seq dedup — no extra
cost, leave on
    "retries": 5,
    "delivery.timeout.ms": 120000,
    "compression.type": "lz4",             # or zstd for better ratio
at higher CPU
    "linger.ms": 10,                       # batch up to 10ms
    "batch.size": 65536,
})

# At-most-once / lowest latency (telemetry) — not recommended for
important data
fire_and_forget = Producer({
    "bootstrap.servers": "kafka-1.internal:9092",
    "acks": "0",
    "enable.idempotence": False,           # must be false with acks=0
})

# Exactly-once (effectively-once) — transactional (see Ch02 for full
discussion)
tx = Producer({
    "bootstrap.servers": "kafka-1.internal:9092,kafka-2.internal:9092,k
afka-3.internal:9092",
    "client.id": "payments-producer",
    "acks": "all",
    "enable.idempotence": True,
    "transactional.id": "payments-tx-01",  # stable ID — fencing on
restart
})
tx.init_transactions(timeout=10)
tx.begin_transaction()
tx.produce("payments", key="order-123", value=b'{"amount": 5000}')
# ... produce to multiple partitions/topics ...
tx.commit_transaction(timeout=10)          # coordinator writes commit
marker, LSO advances
```

Key producer tunables for the distributed-systems lens:

- `delivery.timeout.ms` (default 120s) caps total retry time — after this, the delivery callback gets an error even if `retries` remain. Size it above `retries * retry.backoff.ms` + replication lag.
- `max.in.flight.requests.per.connection` — safe at 5 with idempotence (ordering preserved by sequence), was 1 without.
- `compression.type` — `lz4` is the balanced default; `zstd` gives better ratio for JSON-heavy payloads at higher CPU. Compression is per batch, so larger batches compress better.
- `partitioner` — default is murmur2 hash of key; `sticky` partitioner batches without a key to fewer partitions for better compression when order is irrelevant.

The read path: consumer, groups, and rebalancing

Fetch

Consumers **pull** (long-poll `FetchRequest`). The leader serves from page cache / `sendfile`; `fetch.min.bytes` and `fetch.max.wait.ms` let the broker wait briefly to accumulate a batch rather than returning a tiny response. `fetch.max.bytes` and `max.partition.fetch.bytes` bound the response size so a single large partition does not starve others.

Two watermarks bound what a consumer sees:

- **HWM** — committed data (ISR-acked) — visible to all consumers.
- **LSO (Last Stable Offset)** — HWM minus open transactions' first offsets — visible to `isolation.level=read_committed` consumers. Aborted transactions are skipped (the broker sends the abort markers and the client filters).

Consumer groups and partition assignment

A **consumer group** (`group.id`) is a set of consumers cooperating to consume a topic. Each partition is assigned to **exactly one** consumer in the group; a consumer may own multiple partitions. Two consumers in the same group never share a partition — that is how ordering per partition is preserved (one reader, one order). Different groups read independently at their own offsets.

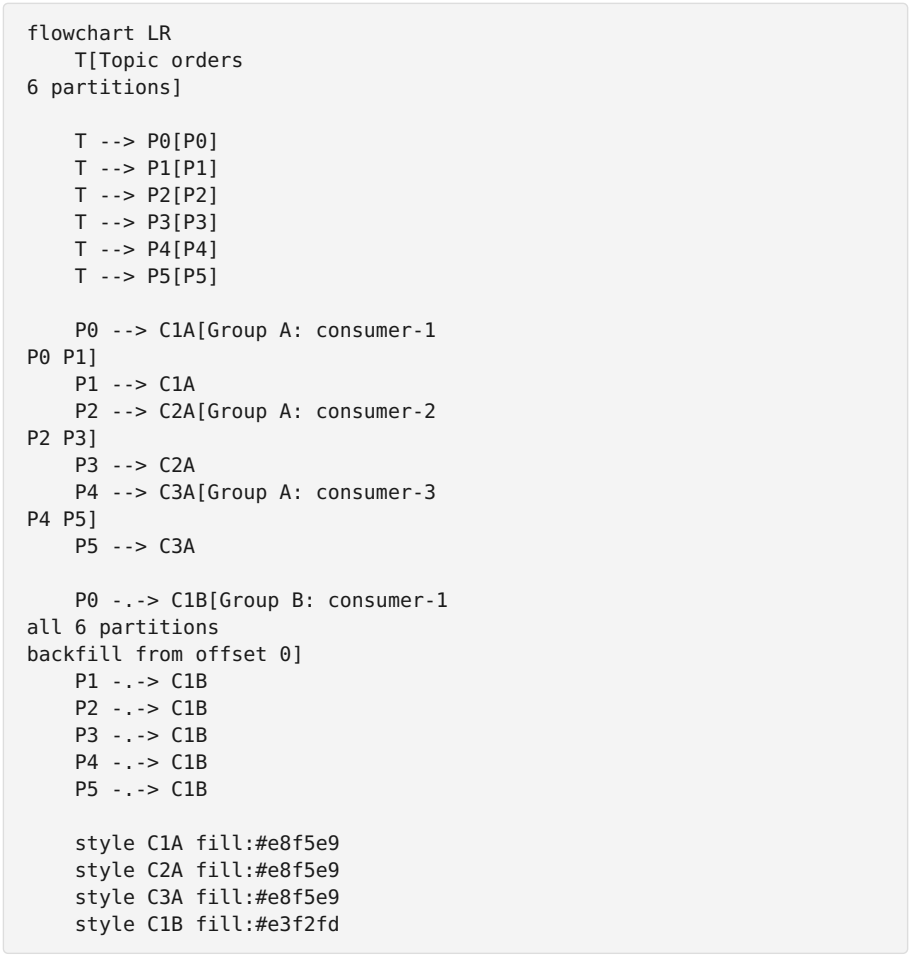


Figure 3-4: Consumer groups. Group A scales by adding consumers up to partition count; Group B reads the same partitions independently at a different offset.

Assignment strategies (Kafka 3.7, `partition.assignment.strategy`):

Strategy	Behavior	When to use
`range`	Contiguous partitions per topic per consumer	Default, but can be uneven with many topics

Strategy	Behavior	When to use
`roundrobin`	Round-robin across all topic-partitions	Even, simple
`cooperative-sticky`	Sticky + cooperative rebalance — minimal partition movement, no stop-the-world	**Preferred in 3.7** — incremental rebalance without global pause
`cooperative-sticky` + `group.instance.id` (static membership)	Partitions stay pinned across restarts — no rebalance on rolling deploy	Rolling restarts without churn

Rebalancing: why it exists and why it hurts

When a consumer joins or leaves (or crashes past `session.timeout.ms`), the group must reassign partitions. In the **eager** protocol (legacy), all consumers revoke all partitions, the leader recomputes assignment, and everyone re-fetches — a stop-the-world pause proportional to `max.poll.interval.ms` and downstream commit latency. In the **cooperative** protocol (Kafka 2.4+, default in 3.7 with `cooperative-sticky`), only the partitions that must move are revoked, and the rebalance is incremental — no global pause.

Two timeouts govern liveness:

- `session.timeout.ms` (default 45s in 3.7 with `group.consumer` session) — heartbeat timeout. If the broker misses heartbeats for this long, the consumer is considered dead and a rebalance is triggered.
- `max.poll.interval.ms` (default 5 minutes) — maximum time between `poll()` calls. If the consumer's handler blocks longer than this without polling, the consumer is considered failed even though heartbeats may still flow (in the old protocol, heartbeats were tied to poll; in 3.7 with KIP-848 the next-gen consumer separates them, but the timeout still applies).

The production pitfall: a handler that occasionally takes 6 minutes (large batch, slow downstream) exceeds `max.poll.interval.ms`, triggers a rebalance, and causes duplicate processing as partitions move. Either raise the interval, reduce `max.poll.records`, or move heavy work off the poll thread.

```
# Kafka 3.7 — consumer with correct commit discipline (confluent-kafka
2.4, Python 3.11)
from confluent_kafka import Consumer

conf = {
    "bootstrap.servers": "kafka-1.internal:9092,kafka-2.internal:9092,k
afka-3.internal:9092",
    "group.id": "orders-processor",
    "group.instance.id": "orders-proc-1",          # static membership
— no rebalance on restart
    "partition.assignment.strategy": "cooperative-sticky",
    "enable.auto.commit": False,
# manual commit — we control the guarantee
    "isolation.level": "read_committed",           # hide aborted
transactions
    "auto.offset.reset": "earliest",
    "fetch.min.bytes": 1024,
    "fetch.max.wait.ms": 500,
    "max.poll.interval.ms": 300000,                # 5 min — must
exceed worst handle() time
    "session.timeout.ms": 45000,
    "max.poll.records": 500,
# bound handler work per poll
}
c = Consumer(conf)
c.subscribe(["orders"])

try:
    while True:
        msg = c.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            # handle INVALID_OFFSET, etc.
            continue
        try:
            handle(msg.value(), msg.key())         # must be
idempotent/deduped for at-least-once
            c.commit(msg)                          # commit offset
after success
        except TransientError:
            # don't commit — redelivery after poll timeout / rebalance
            continue
        except PermanentError:
            publish_to_dlq(msg)                    # see Ch08
            c.commit(msg)                          # advance past
poison message
finally:
    c.close()  # leaves group cleanly — no rebalance timeout
```

```
sequenceDiagram
    participant C1 as Consumer 1
    participant Coord as Group Coordinator
(broker)
    participant C2 as Consumer 2 (new)

    C1->>Coord: JoinGroup (cooperative-sticky)
    Coord->>C1: Assignment: P0 P1 P2
    Note over C1: steady state — owns P0-2

    C2->>Coord: JoinGroup
    Coord->>C1: Revoke: P2 (only P2 moves)
    Coord->>C2: Assign: P2
    Note over C1: continues fetching P0 P1
no pause on unmoved partitions
    Note over C2: fetches P2 from committed offset
```

Figure 3-5: Cooperative rebalance. Only the migrating partition pauses; the rest of the group continues fetching — no stop-the-world.

The control plane: KRaft

From ZooKeeper to KRaft

Kafka 3.7 runs KRaft (Kafka Raft) as the stable metadata quorum; ZooKeeper is deprecated and removed in 4.0. The controller quorum is a Raft group (typically 3 or 5 controller nodes) that replicates a single **metadata log** (`__cluster_metadata` topic, `TopicId AAAAAAAAAAAAAAAAAAAAAA`). All metadata — topic configs, partition assignments, ISR, configs, ACLs — is a record in this log. Brokers act as Raft followers for metadata and apply records to their in-memory image.

Why this matters operationally:

- No external ZooKeeper ensemble to operate, secure, and version-skew.
- Metadata replication is Raft-linearizable — no ZK session expiry or ephemeral-node subtleties.
- Controller failover is Raft leader election, not ZK leader election + broker controller election — one fewer moving part.
- `process.roles=broker,controller` allows colocated nodes for small clusters; production clusters separate them.

```
# Kafka 3.7 KRaft — controller quorum (server.properties / kraft/
server.properties)
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@kafka-1.internal:9093,2@kafka-2.internal:909
3,3@kafka-3.internal:9093
listeners=PLAINTEXT://:9092,CONTROLLER://:9093
inter.broker.listener.name=PLAINTEXT
controller.listener.names=CONTROLLER
log.dirs=/var/lib/kafka/data
num.network.threads=8
num.io.threads=8
# Topic defaults
num.partitions=6
default.replication.factor=3
min.insync.replicas=2
offsets.topic.replication.factor=3
transaction.state.log.replication.factor=3
transaction.state.log.min.isr=2
# KRaft requires explicit cluster ID on format
# kafka-storage.sh format -t <uuid> -c server.properties  (once, before
first start)
```

What the quorum does

The active controller (Raft leader) is the sole writer of metadata. On broker failure, it:

1. Detects liveness via broker heartbeat to the quorum.
2. Removes the failed broker from ISRs it belonged to (ISR shrinkage).
3. Elects new partition leaders from the remaining ISR (or triggers unclean election if enabled and ISR is empty).
4. Writes the new leader/ISR to the metadata log; all brokers apply it.

The quorum size determines availability: 3 controllers survive 1 failure, 5 survive 2. Never run 2 or 4 — even numbers do not improve fault tolerance over the next odd number.

Failure, durability, and the operator's dials

Broker failure walkthrough (RF=3, min ISR=2)

```
t0: P0 ISR={1,2,3} leader=1 HWM=100 LEO=100 on all
t1: broker-3 dies (disk failure)
    → ISR={1,2} (3 ejected after replica.lag.time.max.ms or heartbeat l
oss)
    → HWM still 100 (2 replicas remain, meets min ISR=2) — writes conti
nue
t2: producer with acks=all writes offset 100..109
    → leader 1 appends, follower 2 replicates, HWM advances to 110 — ac
ked
t3: broker-2 dies as well
    → ISR={1} — now below min ISR=2
    → partition becomes unwritable: NotEnoughReplicasException
    → consumers can still read up to HWM=110
t4: broker-2 returns, catches up, ISR={1,2} — writes resume
    (if unclean election were enabled, broker-3 could have been
elected at t3 with data loss)
```

The lesson: `min.insync.replicas` is the **durability floor that trades availability**. With `RF=3, minISR=2` you survive one failure with no loss and remain writable; the second failure makes the partition read-only until a replica returns. That is the correct default for important topics. For availability-at-all-costs topics (metrics, traces), use `minISR=1`.

Rack awareness and tiered storage

- **Rack awareness** (`broker.rack=us-east-1a`) — the controller places replicas across racks/AZs so a rack loss does not take all replicas. The replica selector also prefers fetching from the closest replica (KIP-392, `client.rack`).
- **Tiered storage** (KIP-405, early access in 3.7, `remote.log.storage`) — closed segments are offloaded to S3/GCS; the broker retains only the active working set. Consumers reading far behind fetch from remote storage via the broker. This decouples retention (now bounded by object-store cost, not broker disk) from broker sizing.

Monitoring that matters

Signal	JMX / metric	What it tells you
Under-replicated partitions	`UnderReplicatedPartitions`	ISR < RF — replication lagging or broker down
Offline partitions	`OfflinePartitionsCount`	No ISR replica available — partition unavailable
ISR shrink/expand rate	`IsrShrinksPerSec` / `IsrExpandsPerSec`	Flapping followers — GC, network, or disk issue
Request latency (produce/fetch)	`RequestMetrics` per `request`	Broker overload or disk stall
Consumer lag	`consumer lag` (per group/partition) or Burrow / `kafka-consumer-groups.sh`	Consumer falling behind — scale consumers or partitions
LSO lag	`LogEndOffset - LastStableOffset`	Open transactions holding back `read_committed` readers
Controller queue	`ControllerEventManager` queue size	Metadata overload — topic creation storm or broker flapping

```
# Kafka 3.7 — essential CLI checks (KRaft, no ZooKeeper flag)
kafka-topics.sh --bootstrap-server kafka-1.internal:9092 --describe --
topic orders
# Topic: orders  PartitionCount: 6  ReplicationFactor: 3  Configs:
min.insync.replicas=2,...
#   Partition: 0  Leader: 1  Replicas: 1,2,3  Isr: 1,2  (replica 3
lagging)

kafka-consumer-groups.sh --bootstrap-server kafka-1.internal:9092 --
describe --group orders-processor
# GROUP              TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET
LAG
# orders-processor   orders  0          12340           12348
8

# JMX via jmxterm or Prometheus jmx_exporter — scrape
UnderReplicatedPartitions
```

Sizing guidance

- **Partition count** — partitions are the parallelism unit. More partitions = more throughput and more consumers, but also more file handles, more ISR replication, and higher controller metadata. Start with `partitions ≈ max expected consumer parallelism × 1.5`; grow by splitting (KIP-848 topic auto-split or manual `kafka-topics --alter --partitions`). Never use hundreds of thousands of partitions without testing controller and file-handle limits.
- **Replication factor** — 3 for important data, 2 for ephemeral or cost-sensitive data, 1 only for development.
- **Retention** — `retention.ms` / `retention.bytes` or tiered storage. Size for the longest consumer's catch-up window (backfill, reprocessing) plus compliance needs, not just "a few days."
- **Batch and linger** — `linger.ms=5–10ms` and `batch.size=32–64 KiB` are sensible defaults; increase for throughput, decrease for latency.
- **Compression** — `lz4` or `zstd` for JSON/text payloads; measure — binary payloads (Protobuf, Avro) compress less.

Key takeaways

- Kafka is a partitioned, replicated log — sequential segment files, offset/index/timeindex, page cache + `sendfile` for speed. Partitions scale throughput; replication scales durability.
- ISR + HWM + `acks` / `min.insync.replicas` is the durability contract. `acks=all` with `RF=3, minISR=2` survives one failure with no loss and fails closed on two — the right default for important topics.
- The producer batches, compresses, and sequences per partition (PID+seq for idempotence); the broker appends, replicates to ISR, and advances the HWM before acking.
- Consumers pull via fetch, see only HWM (or LSO with transactions), and coordinate through consumer groups with cooperative-sticky assignment — no stop-the-world on rebalance.
- KRaft (Raft over `__cluster_metadata`) replaces ZooKeeper; the controller quorum manages ISR, leader election, and leader epoch fencing. Run 3 or 5 controllers, separate from brokers in production.
- Rack awareness, tiered storage, and the two timeouts (`session.timeout.ms`, `max.poll.interval.ms`) are the operational levers that determine whether the cluster survives a failure and whether consumers make progress.
- Monitor under-replicated partitions, offline partitions, consumer lag, and LSO lag — they are the early warning for replication, availability, and transaction health.

Further reading

- Apache Kafka documentation 3.7 — Design, replication, KRaft, producer/consumer configs. https://kafka.apache.org/37/documentation.html
- Kafka Improvement Proposals: KIP-500 (KRaft), KIP-848 (next-gen consumer group protocol), KIP-405 (tiered storage), KIP-392 (rack-aware fetch).
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11 — The log abstraction and Kafka's place in it.
- Kreps, J., Narkhede, N., Rao, J. "Kafka: a Distributed Messaging System for Log Processing." NetDB 2011.

- Confluent blog — "Exactly-once Semantics are Possible" (transactions), "Introducing KRaft". https://www.confluent.io/blog/
- `librdkafka` / `confluent-kafka` configuration reference 2.4. https://github.com/confluentinc/librdkafka/blob/master/CONFIGURATION.md
- Narkhede, N. et al. *Kafka: The Definitive Guide*, 2nd ed. O'Reilly, 2021 (supplement with 3.7 KRaft changes).

Chapter 4 — Stream Processing

What this chapter covers. Batch says "process what arrived yesterday." Stream processing says "process what is arriving right now, continuously, and never stop." The difference is not just latency — it is a fundamentally different execution model: unbounded data, event-time semantics, incremental state, and failure recovery that must preserve correctness without reprocessing the world. This chapter builds stream processing from its primitives — streams vs. tables, event time vs. processing time, windows, watermarks, and keyed state — then makes it concrete in the two dominant engines you will operate in production: **Apache Flink 1.19/1.20** and **Kafka Streams 3.7**. We cover Flink's checkpointing barrier protocol and exactly-once state, Kafka Streams' state stores and transactional topology, windowing and watermarks in both, and the operational reality of scaling, rebalancing, late data, and backpressure. Every config is pinned to real versions and uses the Kafka 3.7 and Flink 1.19 APIs you will actually deploy.

Learning goals — after this chapter you should be able to:

- Distinguish stream vs. batch vs. micro-batch and explain why the unbounded-data model changes correctness, latency, and failure semantics.
- Define event time, processing time, and ingestion time, explain why event time is necessary, and describe how watermarks bound lateness.
- Describe window types (tumbling, sliding, session, global) and triggers, and choose correctly for counting, sessionization, and session-timeout workloads.

- Explain keyed state, operator state, checkpointing, and the Chandy-Lambert barrier protocol that gives Flink exactly-once state without stopping the world.
- Build a stateful Flink DataStream job (1.19): Kafka source with watermarks, keyed windowed aggregation, checkpointed state, and a Kafka sink with exactly-once commit.
- Build a Kafka Streams 3.7 topology: KStream/KTable, state stores, windowed aggregation, and exactly-once via `processing.guarantee=exactly_once_v2`.
- Reason about ordering (per-key total order, cross-key partial order), exactly-once end-to-end (source → state → sink), and the scaling/recovery path when a task manager or stream thread dies.
- Decide between Flink and Kafka Streams for a given workload and operational context.

> **Boundary note.** Vol 6 — Distributed Systems — develops the *theory* that constrains stream processing: time and clocks (Ch 2), consistency models (Ch 3), consensus that underpins checkpoint coordination, and the impossibility arguments behind exactly-once delivery (Ch 9 — the producer ambiguity after a lost ack). This chapter is *mechanics*: how streaming engines implement event time, watermarks, windowed state, and fault tolerance on top of a partitioned log. Vol 7, Chapter 7 — Event-Driven Architecture — treats streams at the *system-design* level — when to choose events over RPC, choreography vs. orchestration — and names Kafka as the log without opening the processing engine. For why exactly-once is impossible at the transport layer and what "effectively once" costs, read Vol 6 Ch 9; for when to stream at all, read Vol 7 Ch 7; for how the engine actually keeps state correct under failure, read here.

Streams, tables, and why batch is not enough

A **batch** computation has a start and an end: read a finite dataset, compute, write a result, terminate. A **stream** computation never terminates: it consumes an unbounded sequence of events and continuously maintains results that evolve as new data arrives. The canonical duality (Kreps, "The Log") is:

- **Stream as log** — an append-only sequence of events keyed by time: `... e1, e2, e3, ...` with no end.

- **Table as state** — the materialized view of a stream at a point in time: `SELECT COUNT(*) FROM events` is a table whose rows update as the stream advances.

Every stream processor is a bridge between the two: it reads a stream, holds **state** (a table), and emits a new stream (changelog) or updates a sink table. The "stream-table duality" is literal in Kafka Streams (`KStream` vs. `KTable`) and implicit in Flink (a `DataStream` keyed and aggregated becomes queryable state).

Property	Batch (e.g., Spark batch, MapReduce)	Stream (Flink, Kafka Streams)
Input	Bounded — "all orders from yesterday"	Unbounded — "every order as it is placed"
Latency	Minutes to hours (job launch + full scan)	Milliseconds to seconds (per-event or per-window)
State	Ephemeral per job	Durable, keyed, checkpointed across failures
Correctness on failure	Re-run the job	Restore state from checkpoint + replay from last offset
Output	Single result	Continuous updates, retractions, or windowed aggregates

Micro-batch (Spark Structured Streaming in micro-batch mode) splits the difference: it runs tiny batch jobs on short intervals (100ms–1s). It simplifies reasoning for teams that think in batch, but it trades latency floor and per-event state semantics for scheduler simplicity. True streaming (Flink event-at-a-time, Kafka Streams per-record) processes each event through the operator graph as it arrives.

```
flowchart LR
    subgraph Batch[Batch — finite, terminate]
        B1[(Input dataset
bounded)]
        B2[Job
map + reduce]
        B3[(Output
single result)]
        B1 --> B2 --> B3
    end
    subgraph Stream[Stream — infinite, continuous]
        S1[(Event log
unbounded)]
        S2["Processor
stateful operators"]
        S3[(State
checkpointed)]
        S4[(Sink / changelog
continuous updates)]
        S1 --> S2 <--> S3
        S2 --> S4
        S4 -.->|replay on failure| S2
    end
    style S3 fill:#e8f5e9
    style S2 fill:#e3f2fd
```

Figure 4-1: Batch terminates; streaming holds state and emits continuously. Failure recovery for streaming is state restore plus log replay, not full recomputation.

Time: the hard part

Stream correctness lives or dies on how you handle time. There are three clocks, and conflating them is the most common source of silent data errors.

Clock	Meaning	Who assigns it	Affected by
Event time	When the thing *happened* at the source	Producer — `event.occurred_at` in the payload	Nothing — it is a fact, even if delayed

Clock	Meaning	Who assigns it	Affected by
Ingestion time	When the broker *received* the record	Broker — Kafka's `LogAppendTime`	Broker clock skew, produce latency
Processing time	When the processor *sees* the record	Processor — `System.currentTimeMillis()` on the task manager	Scheduling, GC, backpressure, lag

Processing time is trivial to use and almost always wrong for business logic. If a mobile device buffers events offline for 6 hours and delivers them late, a processing-time window of "orders in the last 5 minutes" will count the event 6 hours late — or miss the window entirely. **Event time** is the only clock that preserves correctness when events are delayed, reordered, or backfilled.

But event time introduces a hard question: *when has all data for a given window arrived?* In an unbounded stream you can never be sure. The answer is **watermarks**.

Watermarks

A **watermark** is a monotone assertion from the source: "I have observed all events with event time ≤ T (up to some bounded lateness)." It flows through the operator graph and tells window operators when to close and fire.

Formally, a watermark `W(T)` means no future event with `event_time < T` is expected (or will be considered on-time). Events that arrive after their window's watermark has passed are **late**.

```
event_time:   0s   2s   5s   4s   7s   11s   8s  ...
watermark:    ---- W(3s) ---- W(6s) ---- W(10s) ---
                         ↑ late: 4s < W(6s)
                                    ↑ late: 8s < W(10s)
```

In Flink, watermarks are generated at the source and propagated operator-to-operator. In Kafka Streams, the equivalent is `grace period` on windowed aggregations (how long after window end to accept late events).

```
sequenceDiagram
    participant P as Producer
    participant K as Kafka (event log)
    participant F as Flink Source
    participant W as Window Operator
    P->>K: order {event_time=12:00:03}
    P->>K: order {event_time=12:00:07}
    P->>K: order {event_time=12:00:02}
    Note over P,K: out-of-order — mobile delay
    K->>F: consume in log order
    F->>F: watermarkStrategy
maxOutOfOrderness=5s
    F->>W: watermark W(12:00:04)
    Note over W: closes window [12:00:00, 12:00:05)
late event 12:00:02 → side output / dropped
    F->>W: watermark W(12:00:08)
    Note over W: closes [12:00:05, 12:00:10)
```

Figure 4-2: Event-time processing with watermarks. The watermark advances based on observed event times minus bounded out-of-orderness; late events are handled explicitly.

Windows

A **window** slices an infinite stream into finite buckets that can be aggregated. The type determines the slicing rule.

Window	Definition	Trigger	Example — "count orders"
Tumbling	Fixed-size, non-overlapping	Every N seconds	Orders per 1-minute bucket — no overlap
Sliding	Fixed-size, overlapping (period < size)	Every P seconds	Orders in last 5 min, updated every 30s
Session	Activity-based, gap-defined	Gap of inactivity closes session	User session = events within 30 min of each other
Global	One window per key, never closes by time	Custom trigger (count, punctuation)	"First 100 events per user, then fire"

```
gantt
    title Windows over event time — key = user_42
    dateFormat X
    axisFormat %S s
    section Tumbling 10s
    window [0,10) :0, 10
    window [10,20) :10, 20
    window [20,30) :20, 30
    section Sliding 10s / every 5s
    w1 [0,10) :0, 10
    w2 [5,15) :5, 15
    w3 [10,20) :10, 20
    section Session gap=8s
    session 1 (events at 1,4,9) :0, 12
    gap :12, 20
    session 2 (events at 22,24) :20, 30
```

Figure 4-3: Tumbling windows partition time cleanly; sliding windows overlap for rolling aggregates; session windows merge events separated by less than the gap timeout.

Late-data handling is explicit: allow lateness with a side output (Flink `allowedLateness` + `sideOutputLateData`), or drop after grace (Kafka Streams `grace(Duration)`). There is no implicit "correct" choice — dropping loses data, allowing lateness emits retractions/updates. Your sink must handle both.

Stateful processing and the exactly-once state problem

A stateless `map` or `filter` needs no memory across events. Anything interesting — counting, joining, deduplication, sessionization — needs **state**: per-key counters, window buffers, join tables, dedup sets. State is partitioned ("keyed") and colocated with the key's processor.

Failure makes state correctness hard. Consider a counter `count[user_42] = 137` on task manager TM-1. TM-1 crashes. The replacement task restores from ... what? If it replays the log from the last committed offset but re-applies events already counted, the counter double-counts. If it restores stale state and skips replay, it under-counts.

The correct answer is **checkpointed state + replay from checkpoint offset atomically**. Both Flink and Kafka Streams implement this, differently.

Flink: barriers, checkpoints, savepoints

Flink's fault tolerance is the **Chandy-Lamport distributed snapshot** via **barrier injection**.

1. The `CheckpointCoordinator` (JobManager) periodically injects a **barrier** (a control event carrying `checkpointId`) into every source.
2. Barriers flow with data through the operator graph, aligned per operator (an operator waits until it has received the barrier on *all* inputs before snapshotting — "barrier alignment").
3. Each operator snapshots its state to durable storage (RocksDB incremental checkpoint to S3/GCS/HDFS) and records its input offsets.
4. When all operators have acked, the checkpoint is **completed**. Offsets and state are now durably paired.
5. On failure, the job restores every operator to the last completed checkpoint and rewinds sources to the checkpointed offsets.

Because barriers flow with data, the snapshot is *consistent* — it reflects a cut across the graph where no in-flight event is counted twice or lost — without stopping processing (aside from brief alignment stalls).

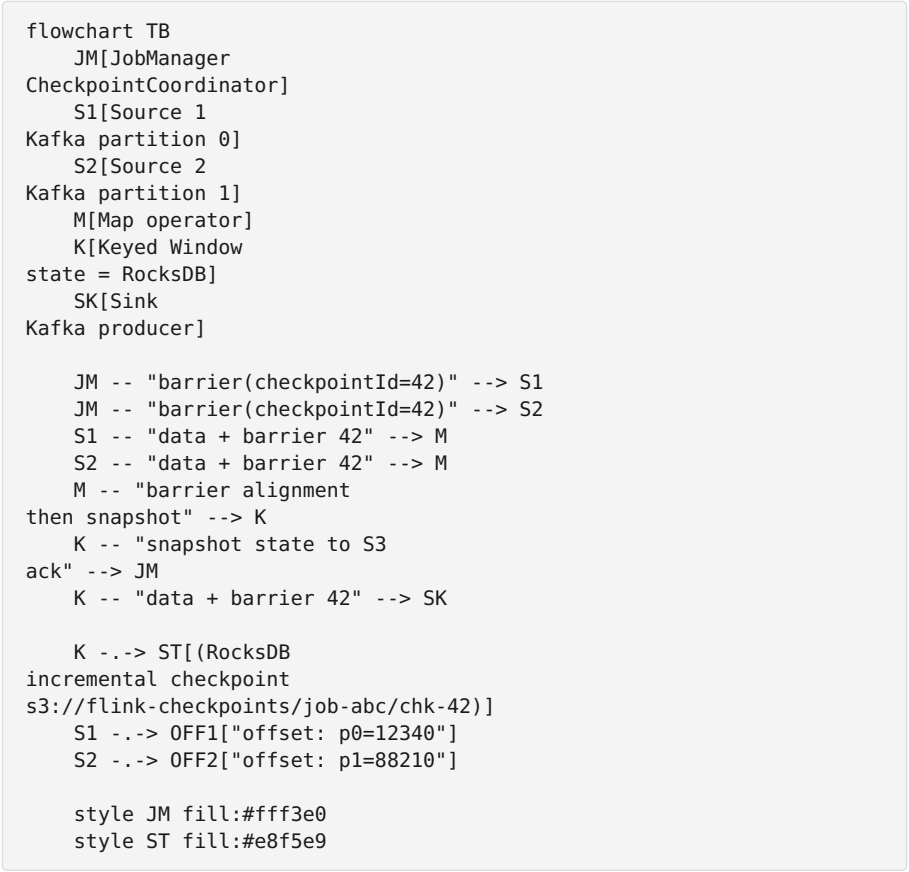


Figure 4-4: Flink barrier protocol. Barriers flow with data; each operator snapshots state and offsets. The completed checkpoint is the consistent restore point.

An **exactly-once sink** (Kafka) participates in the checkpoint via Flink's **two-phase commit sink** (`KafkaSink` with `DeliveryGuarantee.EXACTLY_ONCE`): the sink pre-commits the Kafka transaction on checkpoint, and the JobManager commits it only after the checkpoint completes. If the checkpoint fails, the transaction aborts. This extends exactly-once from Flink state to the external sink.

A **savepoint** is a manually triggered checkpoint that includes additional metadata for job upgrades (topology changes, rescaling). Use savepoints for deploys; checkpoints for automatic recovery.

Kafka Streams: state stores, changelog, and transactions

Kafka Streams embeds the processor inside your application process — there is no separate cluster. Each `KafkaStreams` instance runs **stream threads**; each thread owns a set of **tasks**, each task owns a set of **partitions**, and each task has a **state store** (RocksDB by default, or in-memory) backed by a **changelog topic** (an internal compacted Kafka topic that replicates state).

Fault tolerance: on failure, the task migrates to another instance, restores its state store by replaying the changelog topic from the last checkpoint (offset commit), then resumes processing. There is no barrier protocol — ordering is per-partition and recovery is per-task replay.

Exactly-once in Kafka Streams is `processing.guarantee=exactly_once_v2` (EOS v2, Kafka 3.7): the consumer, state update, and producer commit are wrapped in a single Kafka transaction per task, coordinated by the transaction coordinator. The guarantee is **read-process-write exactly once within the topology** — end-to-end exactly once requires a transactional sink (Kafka topic) and a read-committed downstream consumer.

Dimension	Flink	Kafka Streams
Deployment	Separate cluster (JobManager + TaskManagers) or Application mode on K8s	Library embedded in your service (no cluster)
State	RocksDB / heap, checkpointed to object store via barriers	RocksDB / in-memory, changelog topic on Kafka
Scaling	Rescale via savepoint, reactive mode	Partition count = max parallelism; add instances, tasks rebalance
Exactly-once	Checkpoint + 2PC sink (Kafka), `EXACTLY_ONCE`	`exactly_once_v2` (transactions per task)
Latency	Milliseconds (event-at-a-time, pipelined)	Milliseconds (per-record)
Operations	Checkpoint tuning, TM memory, RocksDB tuning	Consumer group rebalances, state-store sizing, changelog retention

Dimension	Flink	Kafka Streams
When to choose	Complex topologies, large state, multi-source/sink, needs SQL (Flink SQL)	Kafka-in/Kafka-out, team wants no extra cluster, moderate state

---

Flink in practice — a stateful windowed job

A realistic job: from an `orders` Kafka topic (Avro, event time in `occurred_at`), compute **per-merchant revenue per 1-minute tumbling window**, with 10s of allowed lateness for late mobile events, and write results to a `merchant_revenue_1m` Kafka topic exactly once.

```
// Flink 1.19 — DataStream API, Java 17
// Dependencies: flink-streaming-java, flink-connector-kafka 3.1,
flink-avro, flink-statebackend-rocksdb
import org.apache.flink.api.common.eventtime.*;
import org.apache.flink.api.common.functions.AggregateFunction;
import org.apache.flink.connector.kafka.sink.KafkaSink;
import org.apache.flink.connector.kafka.sink.KafkaRecordSerializationSc
hema;
import org.apache.flink.connector.kafka.source.KafkaSource;
import org.apache.flink.connector.kafka.source.enumerator.initializer.O
ffsetsInitializer;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnviro
nment;
import org.apache.flink.streaming.api.windowing.assigners.TumblingEvent
TimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import org.apache.flink.streaming.api.windowing.triggers.EventTimeTrigg
er;
import org.apache.flink.util.OutputTag;

import java.time.Duration;

public class MerchantRevenueJob {

    // Late events that arrive after allowed lateness go here for
reprocessing / DLQ
    private static final OutputTag<Order> LATE_TAG = new OutputTag<Orde
r>("late-orders") {};

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env = StreamExecutionEnvironment.get
ExecutionEnvironment();

        // --- Checkpointing: exactly-once state ---
        env.enableCheckpointing(30_000); // every 30s
        env.getCheckpointConfig().setMinPauseBetweenCheckpoints(10_000)
;
        env.getCheckpointConfig().setCheckpointTimeout(5 * 60_000);
        env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);

env.getCheckpointConfig().setTolerableCheckpointFailureNumber(3);
        // Retain on cancellation so you can resume from savepoint
after deploy
        env.getCheckpointConfig().setExternalizedCheckpointCleanup(
            org.apache.flink.runtime.state.ExternalizedCheckpointCleanu
p.RETAIN_ON_CANCELLATION);

        // --- Kafka source with event-time watermarks ---
        KafkaSource<Order> source = KafkaSource.<Order>builder()
```

```
            .setBootstrapServers("kafka-1.internal:9092,kafka-2.interna
l:9092,kafka-3.internal:9092")
            .setTopics("orders")
            .setGroupId("flink-merchant-revenue")
            .setStartingOffsets(OffsetsInitializer.committedOffsets(
                org.apache.flink.connector.kafka.source.enumerator.init
ializer.OffsetsInitializer.OffsetResetStrategy.EARLIEST))
            .setValueOnlyDeserializer(new AvroOrderDeserializationSchem
a())
            .setProperty("isolation.level", "read_committed") // only
committed transactional writes
            .build();

        WatermarkStrategy<Order> wmStrategy = WatermarkStrategy
            .<Order>forBoundedOutOfOrderness(Duration.ofSeconds(5)) //
max expected reordering
            .withTimestampAssigner((order, ts) -> order.getOccurredAt()
.toEpochMilli())
            .withIdleness(Duration.ofSeconds(30)); // mark idle
partitions so watermarks advance

        DataStream<Order> orders = env.fromSource(source, wmStrategy, "
kafka-orders");

        // --- Keyed tumbling window: 1 minute, event time, 10s
lateness ---
        DataStream<MerchantRevenue> revenue = orders
            .keyBy(Order::getMerchantId)
            .window(TumblingEventTimeWindows.of(Time.minutes(1)))
            .trigger(EventTimeTrigger.create())
            .allowedLateness(Time.seconds(10))
            .sideOutputLateData(LATE_TAG)
            .aggregate(new RevenueAggregate(), new RevenueWindowFunctio
n());

        // Late events — send to DLQ / reprocessing topic
        revenue.getSideOutput(LATE_TAG)
            .map(o -> new LateOrder(o, "late beyond 10s grace"))
            .sinkTo(buildLateSink());

        // --- Kafka sink with exactly-once (2PC) ---
        KafkaSink<MerchantRevenue> sink = KafkaSink.<MerchantRevenue>bu
ilder()
            .setBootstrapServers("kafka-1.internal:9092,kafka-2.interna
l:9092,kafka-3.internal:9092")
            .setRecordSerializer(KafkaRecordSerializationSchema.builder(
)
                .setTopic("merchant_revenue_1m")
                .setKeySerializationSchema(o -> o.getMerchantId().getBy
tes())
                .setValueSerializationSchema(new AvroRevenueSerializati
```

```
onSchema())
                .build())
            .setDeliveryGuarantee(
                org.apache.flink.connector.kafka.sink.KafkaSink.Deliver
yGuarantee.EXACTLY_ONCE)
            .setTransactionalIdPrefix("flink-merchant-revenue-")
            // Transaction timeout must exceed checkpoint interval +
max checkpoint duration
            .setProperty("transaction.timeout.ms", "600000")
            .build();

        revenue.sinkTo(sink);
        env.execute("merchant-revenue-1m");
    }

    // Incremental aggregate — keeps running sum in RocksDB, not full
window buffer
    static class RevenueAggregate implements AggregateFunction<Order, R
evenueAcc, RevenueAcc> {
        @Override public RevenueAcc createAccumulator() { return new Re
venueAcc(0L, 0L); }
        @Override public RevenueAcc add(Order o, RevenueAcc acc) {
            return new RevenueAcc(acc.count + 1, acc.totalCents + o.get
TotalCents());
        }
        @Override public RevenueAcc getResult(RevenueAcc acc) { return
acc; }
        @Override public RevenueAcc merge(RevenueAcc a, RevenueAcc b) {
            return new RevenueAcc(a.count + b.count, a.totalCents + b.t
otalCents);
        }
    }
}
```

```
# flink-conf.yaml — production-relevant settings (Flink 1.19 on
Kubernetes Application mode)
jobmanager.memory.process.size: 2048m
taskmanager.memory.process.size: 4096m
taskmanager.numberOfTaskSlots: 4
parallelism.default: 12
# should divide evenly by Kafka partitions (e.g., 24 partitions → 12
parallelism OK)

state.backend: rocksdb
state.backend.incremental: true              # incremental checkpoints
— only SST diff, not full state
state.backend.rocksdb.memory.managed: true
state.checkpoints.dir: s3://my-flink-checkpoints/merchant-revenue/
state.savepoints.dir: s3://my-flink-savepoints/merchant-revenue/
state.checkpoints.num-retained: 3

execution.checkpointing.interval: 30s
execution.checkpointing.mode: EXACTLY_ONCE
execution.checkpointing.timeout: 5min
execution.checkpointing.tolerable-failed-checkpoints: 3

# RocksDB tuning for large keyed state (per-merchant windows)
state.backend.rocksdb.block.cache-size: 256m
state.backend.rocksdb.writebuffer.size: 64m
```

Key choices and why:

- `forBoundedOutOfOrderness(5s)` — watermark lags the max observed event time by 5s, so an event up to 5s out of order is still on-time. Larger bound = more latency before windows fire, but fewer late events.
- `withIdleness(30s)` — if a Kafka partition goes idle (no events for a merchant), the watermark would stall forever without this. Idleness marks the source idle so downstream watermarks advance on other partitions.
- `allowedLateness(10s)` + `sideOutputLateData` — windows fire on watermark, then accept updates for 10s more (emitting retractions/updates to the sink). Beyond that, events go to the side output for offline reprocessing.
- `EXACTLY_ONCE` sink — the `KafkaSink` participates in the checkpoint as a two-phase commit. On checkpoint, it flushes and pre-commits the Kafka transaction; the JobManager commits on checkpoint

completion. A failed checkpoint aborts the transaction — no partial writes visible to `read_committed` consumers.

- RocksDB incremental checkpoints — for large state (millions of merchant windows), full checkpoints would be too large and slow. Incremental checkpoints upload only new SST files since the last checkpoint.

Kafka Streams in practice — topology, state, and EOS

Same business logic in Kafka Streams: per-merchant 1-minute revenue from `orders` → `merchant_revenue_1m`.

```
// Kafka Streams 3.7, Java 17
// Dependencies: kafka-streams 3.7.0, kafka-clients 3.7.0
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.*;
import org.apache.kafka.streams.kstream.*;
import org.apache.kafka.streams.state.Stores;

import java.time.Duration;
import java.util.Properties;

public class MerchantRevenueStreams {

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "merchant-
revenue-streams");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG,
            "kafka-1.internal:9092,kafka-2.internal:9092,kafka-3.intern
al:9092");
        props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
Serdes.String().getClass());
        props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, AvroS
erde.class);
        // Exactly-once within the topology (EOS v2 — requires Kafka
3.7 brokers, transactional.id fencing)
        props.put(StreamsConfig.PROCESSING_GUARANTEE_CONFIG, StreamsCon
fig.EXACTLY_ONCE_V2);
        props.put(StreamsConfig.REPLICATION_FACTOR_CONFIG,
3);           // for internal topics (changelog, repartition)
        props.put(StreamsConfig.NUM_STANDBY_REPLICAS_CONFIG,
1);         // warm standby for fast failover
        props.put(StreamsConfig.STATE_DIR_CONFIG, "/var/lib/kafka-
streams");
        props.put(StreamsConfig.COMMIT_INTERVAL_MS_CONFIG,
5_000);       // commit/produce interval
        props.put(StreamsConfig.CACHE_MAX_BYTES_BUFFERING_CONFIG, 10 *
1024 * 1024);
        // Consumer isolation — only read committed transactional
writes from upstream
        props.put("isolation.level", "read_committed");

        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, Order> orders = builder.stream("orders",
            Consumed.with(Serdes.String(), new AvroSerde<>())
                .withTimestampExtractor(new OrderEventTimeExtractor()))
; // event time from payload, not LogAppendTime

        // 1-minute tumbling window, 10s grace for late events (Kafka
Streams 3.7 window API)
```

```
        TimeWindows oneMinute =
TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1))
            .grace(Duration.ofSeconds(10));

        KTable<Windowed<String>, RevenueAcc> revenue = orders
            .groupBy((key, order) -> order.getMerchantId(),
                Grouped.with(Serdes.String(), new AvroSerde<>()))
            .windowedBy(oneMinute)
            .aggregate(
                () -> new RevenueAcc(0L, 0L),
                (merchantId, order, acc) -> new RevenueAcc(acc.count +
1, acc.totalCents + order.getTotalCents()),
                Materialized.<String, RevenueAcc>as(
                    Stores.persistentWindowStore("merchant-revenue-
store",
                        Duration.ofMinutes(5),          // retention —
must exceed window + grace
                        Duration.ofMinutes(1),          // window size
                        false))
                    .withKeySerde(Serdes.String())
                    .withValueSerde(new JsonSerde<>())
            );

        // Emit to output topic — EOS ensures this produce is part of
the same transaction as the consume + state update
        revenue.toStream()
            .map((windowedKey, acc) -> {
                String merchantId = windowedKey.key();
                long windowStart = windowedKey.window().start();
                MerchantRevenue out = new MerchantRevenue(merchantId, w
indowStart, acc.count, acc.totalCents);
                return KeyValue.pair(merchantId, out);
            })
            .to("merchant_revenue_1m", Produced.with(Serdes.String(), n
ew AvroSerde<>()));

        KafkaStreams streams = new KafkaStreams(builder.build(), props)
;
        // Uncaught exception handler — close and let K8s restart;
state restores from changelog
        streams.setUncaughtExceptionHandler(ex -> {
            // Log, emit metric, then shutdown — do not silently
swallow
            org.slf4j.LoggerFactory.getLogger(MerchantRevenueStreams.cl
ass)
                .error("Streams uncaught exception — shutting down for
restart", ex);
            return StreamsUncaughtExceptionHandler.StreamThreadExceptio
nResponse.SHUT_DOWN_CLIENT;
        });
        streams.start();
```

```
        Runtime.getRuntime().addShutdownHook(new Thread(streams::close)
);
    }

    // Event-time extractor — business time, not wall clock
    static class OrderEventTimeExtractor implements org.apache.kafka.cl
ients.consumer.ConsumerRecordTimestampExtractor {
        @Override
        public long extract(org.apache.kafka.clients.consumer.ConsumerR
ecord<Object, Object> record) {
            Order order = (Order) record.value();
            return order.getOccurredAt().toEpochMilli();
        }
    }
}
```

Operational notes:

- `EXACTLY_ONCE_V2` wraps each task's `poll → process → state update → produce → commit` in one Kafka transaction. If the instance crashes mid-transaction, the transaction aborts and the next owner replays from the last committed offset — no duplicates in the output topic when consumed with `isolation.level=read_committed`.
- **Changelog topics** (`merchant-revenue-streams-merchant-revenue-store-changelog`) are internal, compacted, and replicated per `REPLICATION_FACTOR_CONFIG`. Monitor their lag — a large state restore on rebalance replays this topic.
- **Standby replicas** (`NUM_STANDBY_REPLICAS=1`) keep a warm copy of each task's state on another instance. On failure, promotion is near-instant instead of replaying minutes of changelog.
- **Grace period** (`grace(10s)`) is the Kafka Streams equivalent of Flink's `allowedLateness`. After `window_end + grace`, the window is closed and purged. Late events beyond grace are dropped (or routed via a branch + side topic if you explicitly handle them).
- **Partition count is the parallelism ceiling.** A topic with 6 partitions can run at most 6 stream threads doing useful work (one task per partition). Over-provision partitions up front or plan a partition-split migration.

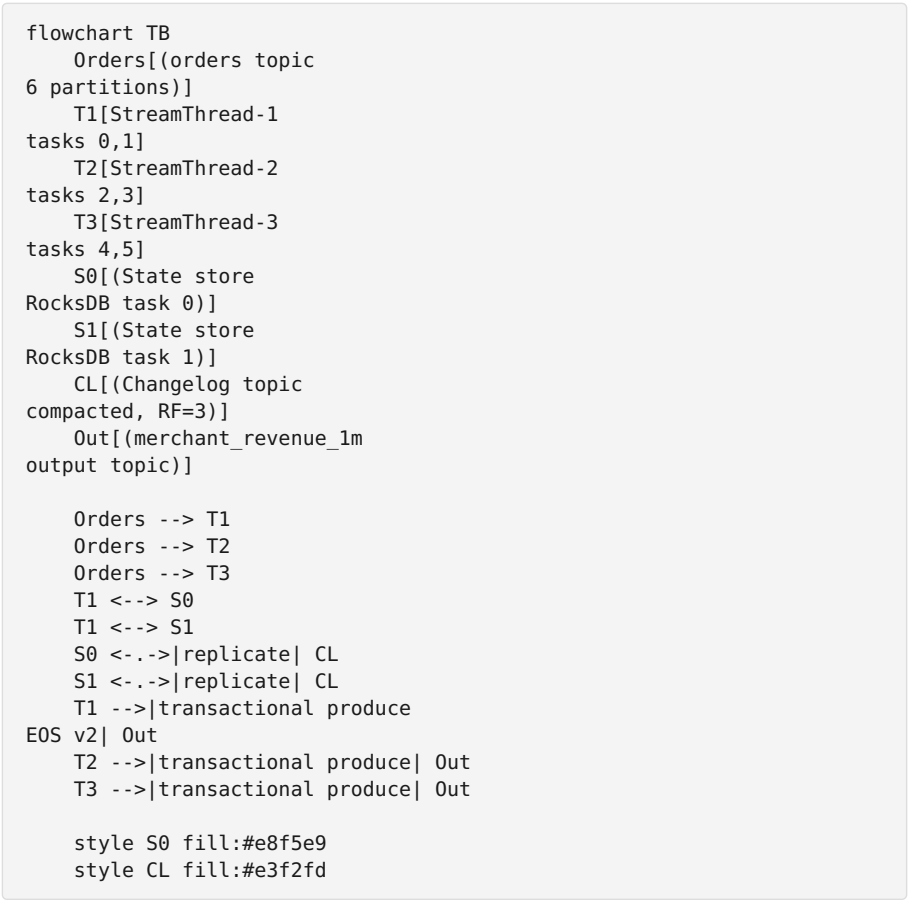


Figure 4-5: Kafka Streams deployment. Each task owns partitions and a RocksDB state store backed by a changelog topic. EOS v2 makes consume + state update + produce atomic per task.

Distributed-systems lens: ordering, exactly-once, and failure

Stream processing inherits every distributed-systems constraint from Vol 6 and adds one more: it must stay correct while never stopping.

Ordering

- **Within a key, total order.** All events for `merchant_42` go to one Kafka partition and one Flink subtask / one Kafka Streams task. Within that partition the log offset is the total order. Use a deterministic partition key (`merchant_id`) at produce time — without it, there is no ordering guarantee at all.
- **Across keys, partial order.** `merchant_42` and `merchant_99` may be processed on different tasks in any interleaving. If your logic needs cross-key ordering (e.g., global sequence), you must either repartition through a single-partition topic (a correctness bottleneck) or redesign to avoid the requirement.
- **Across windows, watermark order.** Windows fire in watermark order, not arrival order. An event with `event_time=12:00:02` that arrives after the watermark has passed `12:00:05` is late by definition — the engine did exactly what you told it to.

Exactly-once — what is actually guaranteed

"Exactly once" in stream processing means **each event's effect on state and output is applied once**, even under failure. It does not mean each event is delivered to the processor once — the log will redeliver after a failure. The guarantee is that redelivery's effect is deduplicated by restoring state to the checkpoint and replaying from the checkpointed offset, and by committing output transactionally so partial writes are invisible.

End-to-end exactly-once requires **all three** to be transactional: source offsets, operator state, and sink writes. In Flink that is barrier checkpoints + RocksDB snapshot + 2PC KafkaSink. In Kafka Streams that is `exactly_once_v2` + changelog + transactional produce. If your sink is not transactional (e.g., HTTP POST, non-transactional Postgres without idempotency keys), you have at-least-once output regardless of engine — see Vol 6 Ch 9 for the consumer-side dedup table that makes at-least-once effectively once.

Failure and scaling

Failure	Flink behavior	Kafka Streams behavior
TaskManager / stream thread crash	JobManager restores from last checkpoint, rewinds sources, replays	Task migrates to another instance, restores RocksDB from changelog, replays
Kafka broker loss	Source stalls on affected partitions until ISR recovers; checkpoint may timeout and retry	Consumer stalls / rebalance; tasks reassigned
Checkpoint / commit timeout	Checkpoint marked failed, next one retries; job continues	Transaction timeout → abort, retry on next commit interval
Backpressure (sink slow)	Network buffers fill, upstream operators throttle, checkpoint barriers delayed	`poll` slows, consumer lag grows, `max.poll.interval.ms` risk

Scaling out means increasing parallelism and repartitioning state. In Flink, rescale from a savepoint (state is redistributed by key group). In Kafka Streams, add instances — the group rebalance assigns unowned partitions/tasks automatically, with standby replicas warming the move.

> **Anti-patterns that cause incidents.** - Using **processing time** for business windows and calling it "real-time" — then discovering the 3% of late mobile events silently corrupt aggregates. - **No idleness handling** — one idle Kafka partition stalls the global watermark forever and windows never fire. - **Sink without exactly-once** (plain `KafkaProducer` or HTTP) behind an exactly-once engine — the engine guarantees state, not output. - **Partition count = 1** for "simplicity" — then throughput hits a single-task ceiling and scaling requires a disruptive repartition.

Choosing between Flink and Kafka Streams

Use **Flink** when: topology is complex (multi-source joins, async I/O, Flink SQL / Table API), state is large (GBs per key, incremental checkpoints to

S3), you need exactly-once to non-Kafka sinks (JDBC 2PC, Pulsar), or the team can operate a Flink cluster (or use a managed Flink / Ververica).

Use **Kafka Streams** when: data is already in Kafka and stays in Kafka (Kafka-in/Kafka-out), state is moderate, the team prefers a library over a cluster, and operational simplicity outweighs Flink's richer operator set. The ceiling is partition count and RocksDB-on-local-disk.

Many organizations run both: Kafka Streams for lightweight per-service transformations colocated with the service, Flink for the central streaming platform that powers large windowed analytics, joins, and ML feature pipelines.

Key takeaways

- Streaming processes unbounded data continuously, holding keyed state and emitting incremental results — failure recovery is state restore plus log replay, not batch recomputation.
- Correctness depends on **event time** and **watermarks**; processing time is fast but wrong for delayed or reordered events. Watermarks bound lateness and drive window firing.
- **Windows** (tumbling, sliding, session, global) slice the infinite stream into finite aggregates; choose by whether you need non-overlapping buckets, rolling views, or activity-defined sessions. Late data is explicit — allowed lateness / grace or side output.
- **Flink** achieves exactly-once state via Chandy-Lamport barriers and incremental RocksDB checkpoints; an exactly-once KafkaSink participates as a two-phase commit so output is also exactly once.
- **Kafka Streams** achieves exactly-once via `exactly_once_v2` transactions per task, with RocksDB state backed by a compacted changelog topic and warm standbys for fast failover.
- Ordering is per-key total order (one partition → one task); cross-key order is partial. End-to-end exactly-once requires transactional source offsets, state, and sink — without a transactional sink, make the sink idempotent (Vol 6 Ch 9).
- Operate with idle-source handling, partition-key discipline, checkpoint/commit monitoring, and savepoints for deploys — and never assume a sink is exactly once just because the engine is.

Further reading

- Flink documentation 1.19/1.20 — Streaming Concepts, Event Time & Watermarks, Checkpointing, Kafka Connector (exactly-once). https://nightlies.apache.org/flink/flink-docs-release-1.19/ and https://flink.apache.org/
- Kafka Streams documentation 3.7 — Concepts, windowing, exactly-once (EOS v2), state stores. https://kafka.apache.org/37/documentation/streams/
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11 — Stream processing, watermarks, and the log.
- Akidau, T. et al. *Streaming Systems* (O'Reilly, 2018) — The definitive treatment of watermarks, windows, and triggers (the Dataflow/Beam model that Flink implements).
- Carbone, P. et al. "Apache Flink: Stream and Batch Processing in a Single Engine." IEEE Data Engineering Bulletin, 2015. And Flink Forward talks on checkpointing barriers and RocksDB state.
- Kreps, J. "The Log: What every software engineer should know about real-time data's unifying abstraction." LinkedIn Engineering, 2013.
- Confluent / Kafka Improvement Proposals: KIP-129 (EOS), KIP-447 (EOS v2 / transactional fencing), KIP-441 (grace period semantics).

Chapter 5 — Event Sourcing and CQRS

What this chapter covers. Most services store *current state* — `UPDATE orders SET status='shipped' WHERE id=...` — and discard how they got there. When you need an audit trail, temporal queries ("what did this order look like last Tuesday?"), or the ability to rebuild a read model after a bug, that choice is irreversible: the history is gone. **Event sourcing** inverts the model: the append-only sequence of domain events *is* the source of truth, and current state is a left-fold over that sequence. **CQRS** (Command Query Responsibility Segregation) is its common companion: separate the write model that enforces invariants from the read models optimized for queries, with projections bridging the two. Together they power the most auditable, replayable architectures — and the most

commonly over-applied. This chapter builds both patterns from mechanics: the event store as a per-aggregate ordered log with optimistic concurrency, aggregates as consistency boundaries, projections as derived read models, snapshots as an optimization, and the distributed-systems reality of eventual consistency between writes and reads. We implement a concrete event store on **Postgres 16** and on **EventStoreDB 23/24**, show a CQRS command handler with idempotent projection, cover schema evolution without breaking replays, and name the operational costs that determine whether these patterns earn their keep.

Learning goals — after this chapter you should be able to:

- Define event sourcing precisely — stream per aggregate, append-only events, current state as a fold — and contrast it with state-oriented persistence on auditability, temporal query, and replay.
- Describe the event store contract: per-stream total order, global ordering options, expected-version optimistic concurrency, idempotent append, and why the stream *is* the lock.
- Model an aggregate as a consistency boundary: commands vs. events, invariant enforcement, and why an aggregate must be small and transactionally loaded whole.
- Distinguish CQRS from event sourcing and explain when each earns its complexity: CQRS without event sourcing, and event sourcing without CQRS.
- Implement the pattern on Postgres (DDL, append with `FOR UPDATE` / advisory lock, projection poller) and on EventStoreDB (streams, expected revision, subscriptions).
- Build a projection that is idempotent, ordered per aggregate, and resumable from a checkpoint — with exactly-once effect via a dedup/position table.
- Handle schema evolution (upcasting, weak schema, versioned events) and snapshots without breaking replay or time travel.
- Reason about the distributed lens: eventual consistency between write and read models, ordering guarantees for cross-aggregate projections, and failure/recovery when a projector falls behind.

> **Boundary note.** Vol 6 develops the *theory* behind event sourcing: consistency models and linearizability that constrain what a store can promise (Ch 3), optimistic concurrency and compare-and-swap as a coordination primitive (Ch 7–8), and idempotency/deduplication that

> makes projected-at-least-once safe (Ch 9). Vol 7, Chapter 7 — Event-Driven Architecture — treats events at the *system-design* level — when to use events at all, choreography vs. orchestration, and service-boundary scoping — and introduces the term "domain event" without opening the store. This chapter is *mechanics*: the event store's per-stream ordering and concurrency contract, aggregate design, projection correctness, and the outbox/CDC wiring that publishes events durably. For the dual-write problem that arises when publishing events from an event-sourced aggregate, see Chapter 6 — The Outbox Pattern; the two chapters share the same transactional invariant and are best read together.

From state to events: what changes

In a state-oriented service, persistence answers "what is true now?"

```
-- State-oriented: current row, history lost
UPDATE orders SET status = 'shipped', shipped_at = now() WHERE id = 'or
d_9f3a';
-- What was the status before? Who changed it? In what order did
transitions happen? — not in this row.
```

In an event-sourced service, persistence answers "what has happened?" — and current state is derived:

```
stream: orders-ord_9f3a  (one stream per aggregate instance)
  event 0: OrderPlaced      {order_id, user_id, items[], total_cents, p
laced_at}
  event 1: PaymentAuthorized {payment_id, amount_cents}
  event 2: InventoryReserved {items[], warehouse_id}
  event 3: OrderShipped      {shipment_id, carrier, shipped_at}
  ──────────────────────────────────────────────────────
  current state = fold(apply, empty, [0..3]) → {status: shipped, ...}
```

The consequences are not subtle:

Property	State-oriented	Event-sourced
Source of truth	Current row / document	Append-only event stream
History	Lost unless separately audited	Complete, ordered, by construction

Property	State-oriented	Event-sourced
Temporal query	Requires extra tables / CDC	Replay stream to any point in time
Invariant enforcement	Constraints / transactions on current state	Aggregate logic on the fold of past events
Bug recovery	Restore from backup, lose intervening writes	Replay events into a fixed projector; re-derive the read model
Complexity	Lower for CRUD	Higher — modeling, versioning, projection ops

Event sourcing is not "store JSON blobs instead of rows." It is a persistence *discipline* with a precise store contract and modeling constraints that must be understood before adopting it.

```
flowchart TB
    subgraph StateOriented["State-oriented"]
        Cmd1[Command] --> Svc1[Service]
        Svc1 --> DB1[(Row: orders
UPDATE in place)]
        DB1 --> Read1[Read current row]
    end
    subgraph EventSourced["Event-sourced"]
        Cmd2[Command] --> Agg[Aggregate
load events → decide]
        Agg --> Store[(Event store
append-only stream)]
        Store --> Fold["Fold: apply 0..N
→ current state"]
        Store --> Proj[Projector
→ read models]
        Proj --> Read2[(Read DB
query-optimized)]
        Fold -.->|same store| Agg
    end
    style Store fill:#e8f5e9
    style Proj fill:#e3f2fd
```

Figure 5-1: State-oriented persistence overwrites; event sourcing appends and derives state. The read model in CQRS is a separate projection, not the aggregate's own state.

The event store contract

An event store is not a generic message broker. It is a **per-aggregate ordered log with optimistic concurrency**. The minimal contract every implementation must provide — whether Postgres, EventStoreDB, DynamoDB conditional writes, or Kafka with per-aggregate partitioning — is:

Streams, order, and append

- **Stream identity** — a stream is named for one aggregate instance: `orders-ord_9f3a`, `inventory-sku_123`, `account-usr_42`. Many designs also expose **category streams** (`$ce-orders` = all order streams) and a **global log** (`$all`) for cross-aggregate subscriptions.
- **Per-stream total order** — events in a single stream have a strict, gap-free sequence (`streamRevision 0,1,2,...`). This is the *only* hard ordering guarantee. Global order is best-effort or logical if it exists at all.
- **Append-only** — streams are never updated or deleted in place (tombstoning/soft-deletion aside). Compaction, if any, is a read-time projection concern, not a store mutation.
- **Expected version (optimistic concurrency)** — each append carries the revision it *expects* to follow. If the store's current revision does not match, the append is rejected with a concurrency conflict. There is no pessimistic lock — the stream *is* the lock.

Why optimistic concurrency matters: two concurrent commands loading the same aggregate at revision 5 and both trying to append revision 6 — only one can win. The loser reloads, re-evaluates invariants against the new history, and retries or rejects. This is the distributed-systems primitive that prevents lost updates without distributed locks (cf. Vol 6, Ch 7–8).

```
client A: load 0..5, decide → append event 6 with expectedVersion=5  →
✓  stream now at 6
client B: load 0..5, decide → append event 6 with expectedVersion=5  →
✗  WrongExpectedVersion (now at 6)
client B: reload 0..6, re-decide → append event 7 with
expectedVersion=6 → ✓
```

- **Idempotent append** — an idempotency key (typically `eventId`) makes retries safe: re-appending the same `eventId` returns success without duplicating. Without this, the producer ambiguity from Ch 2 / Vol 6 Ch 9 reappears inside the store.

```
sequenceDiagram
    participant C as Command handler
    participant A as Aggregate
    participant S as Event store
(Postgres / EventStoreDB)
    C->>S: load stream orders-ord_9f3a
events 0..N
    S-->>C: [OrderPlaced, PaymentAuthorized]
    C->>A: handle(ShipOrder) with state=fold(0..N)
    Note over A: enforce invariants:
must be authorized
not already shipped
    A-->>C: event OrderShipped {shipment_id}
    C->>S: append orders-ord_9f3a
expectedVersion=N
eventId=evt_abc, type=OrderShipped
    alt expectedVersion matches
        S-->>C: ack revision N+1
        C->>C: publish to projections
    else WrongExpectedVersion
        S-->>C: 409 Conflict
        C->>C: reload + retry or reject
    end
```

Figure 5-2: The event-store append path. The aggregate is pure decision logic; the store's expected-version check is the concurrency gate.

Aggregates: the consistency boundary

An **aggregate** (DDD) is a cluster of domain objects treated as a unit for invariants and persistence. In event sourcing the aggregate is the

consistency boundary: all invariants that must hold atomically live inside one aggregate instance, and one command touches one aggregate.

- A **command** is an imperative request: `PlaceOrder {items, user_id}`. It may be rejected.
- An **event** is a fact that has happened: `OrderPlaced {order_id, ...}`. It is immutable and past-tense.
- The aggregate **handles** a command by loading its event history, folding to current state, checking invariants, and **emitting** zero or more events (or rejecting the command).

```
aggregate Order (stream: orders-{orderId}):
  state: {status, items[], totalCents, paymentId?, shipmentId?}

  on PlaceOrder(cmd):
    require cmd.items non-empty
    require cmd.totalCents > 0
    emit OrderPlaced {order_id, user_id, items, total_cents, placed_at}

  on AuthorizePayment(cmd):
    require state.status == placed
    require cmd.amount == state.totalCents
    emit PaymentAuthorized {payment_id, amount}

  on ShipOrder(cmd):
    require state.status == authorized
    require not already shipped
    emit OrderShipped {shipment_id, carrier, shipped_at}
```

Design rules that prevent incidents:

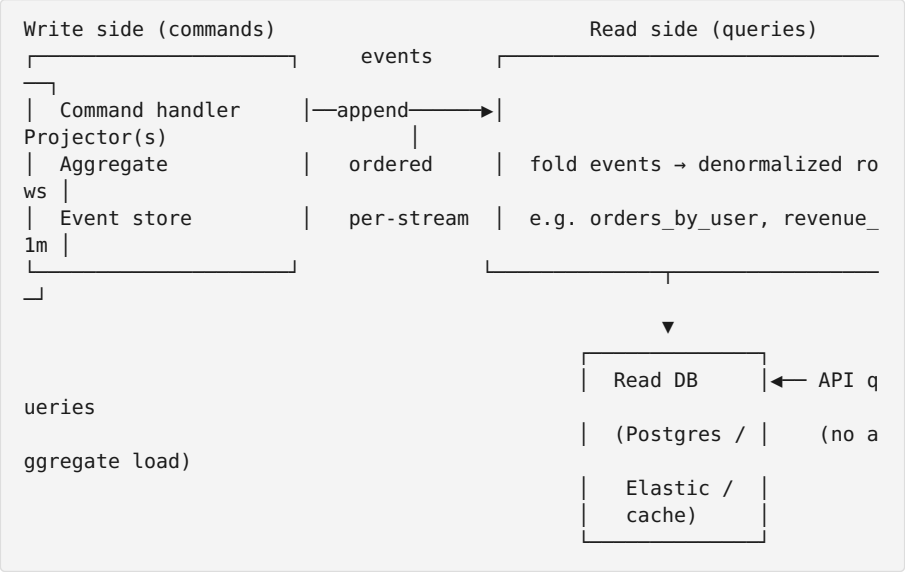
- **Small aggregates.** An aggregate is loaded whole on every command (`load 0..N, fold`). A stream with tens of thousands of events is a latency and concurrency hazard — split the boundary or snapshot (below). Canonical aggregates are one order, one account, one cart — not "all orders for a merchant."
- **One aggregate per transaction.** Cross-aggregate invariants ("total inventory across SKUs must not exceed…") cannot be enforced atomically by the store's per-stream OCC. They require a process manager / saga at a higher level — eventual, compensating, not atomic (see Vol 7 Ch 7 for choreography vs. orchestration).
- **No queries inside aggregates.** An aggregate decides from *its own* history, not by querying other aggregates or read models. If you need

data from elsewhere, pass it in the command or model the workflow as a saga.

- **Events are the API.** Renaming `OrderPlaced` to `OrderCreated` is a breaking change for every projector. Treat event names and shapes as versioned contracts (schema section below).

CQRS: separating writes from reads

CQRS says: use different models for **commands** (writes, invariant enforcement) and **queries** (reads, view optimization). The command side is the aggregate + event store; the query side is one or more **read models** (projections) tailored to specific access patterns.



Why separate? Write and read needs conflict:

Concern	Write model	Read model(s)
Shape	Normalized, invariant-rich aggregate	Denormalized, query-shaped (join-free)
Consistency	Strong per-aggregate (OCC)	Eventual — lags writes by projector latency

Concern	Write model	Read model(s)
Scale	Write throughput = aggregate concurrency	Read throughput = independently scalable view
Evolution	Events are append-only, versioned	Views are disposable and rebuildable by replay

Crucially, **CQRS and event sourcing are independent**. You can do CQRS with a CRUD write model (write to one table, project to another) and you can do event sourcing without CQRS (fold the stream on every read). They pair well because a durable event log makes projections reliable and replayable, but neither implies the other.

The distributed-systems cost: the read model is **eventually consistent** with the write model. After `OrderPlaced` is appended, a query to `GET /orders?user=usr_42` may not see it for milliseconds to seconds, depending on projector lag. Clients that need read-after-write (e.g., "I just placed an order, show it") must either read from the aggregate stream, wait for the projection checkpoint, or accept the window — there is no way to make a derived view strongly consistent without reintroducing coupling (Vol 6, Ch 3).

Implementation I — Event store on Postgres 16

Postgres is the most common event store in teams that already operate it. The schema is small; correctness is in the append protocol.

```
-- Postgres 16 — event store DDL
-- One table for the ordered log, one for snapshots (optional), one for
projector checkpoints.

CREATE TABLE event_store (
    stream_id       TEXT        NOT NULL,          -- e.g., 'orders-
ord_9f3a'
    stream_revision BIGINT      NOT NULL,          -- 0,1,2,... per
stream — gap-free, unique
    event_id        UUID        NOT NULL UNIQUE,   -- globally unique,
idempotency key
    event_type      TEXT        NOT NULL,          -- e.g.,
'OrderPlaced'
    event_version   INT         NOT NULL DEFAULT
1,-- schema version of this event type
    payload         JSONB       NOT NULL,          -- event body —
validated by app, not DB
    metadata        JSONB       NOT NULL DEFAULT '{}'::jsonb, --
causationId, correlationId, userId, traceparent
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (stream_id, stream_revision)
);
CREATE INDEX event_store_global_order ON event_store (created_at, strea
m_id, stream_revision);
CREATE INDEX event_store_type ON event_store (event_type);

-- Checkpoints for each projector — exactly-once effect depends on this
CREATE TABLE projection_checkpoint (
    projector_name TEXT PRIMARY KEY,
    last_position  BIGINT NOT NULL DEFAULT 0,       -- last global
position processed (or per-stream map as JSONB)
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Snapshots — optional optimization, not a separate source of truth
CREATE TABLE aggregate_snapshot (
    stream_id       TEXT        PRIMARY KEY,
    stream_revision BIGINT      NOT NULL,
    state           JSONB       NOT NULL,           -- folded state at
that revision
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Append with optimistic concurrency and idempotency

```
// Java 17, Spring + jOOQ/JDBC — append with OCC + idempotent retry
// Postgres 16, SERIALIZABLE or READ COMMITTED + explicit locking both
work; this uses advisory/row-level discipline.

public record AppendResult(long newRevision, boolean wasDuplicate) {}

public class PostgresEventStore {
    private final DataSource ds;

    /**
     * Append a single event to a stream.
     * @param streamId        e.g., "orders-ord_9f3a"
     * @param expectedRevision the revision the caller believes is
current (-1 for new stream, 0..N otherwise)
     * @param eventId         globally unique — idempotency key;
retrying the same eventId is a no-op
     * @param eventType       e.g., "OrderPlaced"
     * @param payload         JSON payload
     * @return new revision on success
     * @throws WrongExpectedVersion if OCC check fails
     */
    public AppendResult append(String streamId, long expectedRevision,
UUID eventId,
                               String eventType, String payloadJson) th
rows SQLException {
        // Idempotency check first — if this eventId already exists,
return its revision without duplicating
        try (Connection c = ds.getConnection()) {
            try (PreparedStatement ps = c.prepareStatement(

"SELECT stream_id, stream_revision FROM event_store WHERE event_id
= ?")) {
                ps.setObject(1, eventId);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        return new AppendResult(rs.getLong(2), true);
                    }
                }
            }

            long nextRevision = expectedRevision + 1;

            // For a new stream, expectedRevision == -1 and
nextRevision == 0
            // Acquire a per-stream lock to serialize concurrent
appends to the same stream.
            // pg_advisory_xact_lock is transaction-scoped and avoids
table-level contention.
            // Key: hashtext(stream_id) — collisions are vanishingly
rare and only cause extra serialization, not correctness loss.
```

```
            try (PreparedStatement lock = c.prepareStatement("SELECT
pg_advisory_xact_lock(hashtext(?))")) {
                lock.setString(1, streamId);
                lock.execute();
            }

            // Verify expected revision still holds after lock
            long currentMax = -1;
            try (PreparedStatement ps = c.prepareStatement(
                    "SELECT COALESCE(MAX(stream_revision), -1) FROM
event_store WHERE stream_id = ?")) {
                ps.setString(1, streamId);
                try (ResultSet rs = ps.executeQuery()) { rs.next(); cur
rentMax = rs.getLong(1); }
            }
            if (currentMax != expectedRevision) {
                throw new WrongExpectedVersion(
                    "stream " + streamId + " expected " + expectedRevis
ion + " but is at " + currentMax);
            }

            try (PreparedStatement ps = c.prepareStatement(
                    "INSERT INTO event_store(stream_id,
stream_revision, event_id, event_type, payload, metadata) "
                  + "VALUES (?, ?, ?, ?, ?::jsonb, ?::jsonb)")) {
                ps.setString(1, streamId);
                ps.setLong(2, nextRevision);
                ps.setObject(3, eventId);
                ps.setString(4, eventType);
                ps.setString(5, payloadJson);
                ps.setString(6, "{\"correlation_id\":\"" + eventId + "\
"}");
                ps.executeUpdate();
            }
            c.commit();
            return new AppendResult(nextRevision, false);
        }
    }

    /** Load a stream's events in order — for aggregate rehydration. */
    public List<StoredEvent> load(String streamId, long fromRevisionInc
lusive) throws SQLException {
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(
                 "SELECT stream_revision, event_id, event_type,
payload, created_at "
               + "FROM event_store WHERE stream_id = ? AND
stream_revision >= ? ORDER BY stream_revision")) {
            ps.setString(1, streamId);
            ps.setLong(2, fromRevisionInclusive);
            try (ResultSet rs = ps.executeQuery()) {
```

```
                List<StoredEvent> out = new ArrayList<>();
                while (rs.next()) out.add(new StoredEvent(rs.getLong(1)
, (UUID) rs.getObject(2),
                    rs.getString(3), rs.getString(4),
rs.getTimestamp(5).toInstant()));
                return out;
            }
        }
    }
}
```

Notes:

- `pg_advisory_xact_lock(hashtext(stream_id))` serializes appends per stream without blocking appends to unrelated streams. For very high throughput per stream (rare — aggregates are small), consider an explicit `SELECT ... FOR UPDATE` on a `stream_head` table instead.
- The primary key `(stream_id, stream_revision)` enforces gap-free ordering structurally; a duplicate revision is a constraint violation, which surfaces as OCC failure.
- For global subscriptions (projectors that consume across all streams), order by `created_at` + `(stream_id, stream_revision)` or introduce a `global_position BIGSERIAL` column if strict global total order is needed. Postgres sequences give per-insert order but not transactional commit order — document the guarantee.

Projector — poll, checkpoint, idempotent apply

```
// Polling projector: reads new events in global order, applies to a
read model, checkpoints atomically.
// The transaction is: (update read model) + (update checkpoint)
atomically → effectively-once.

public class OrderSummaryProjector {
    private final DataSource ds;
    private static final String PROJECTOR = "order_summary";

    // Poll loop — run every 200ms or via LISTEN/NOTIFY
    public void tick() throws SQLException {
        long lastPos;
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(

"SELECT last_position FROM projection_checkpoint WHERE projector_name
= ?")) {
            ps.setString(1, PROJECTOR);
            try (ResultSet rs = ps.executeQuery()) {
                lastPos = rs.next() ? rs.getLong(1) : 0L;
            }
        }

        // Read next batch in global order — adjust batch size for lag/
throughput
        List<StoredEvent> batch = readGlobalBatch(lastPos, 500);
        if (batch.isEmpty()) return;

        for (StoredEvent e : batch) {
            // Idempotent apply: INSERT ... ON CONFLICT DO NOTHING /
UPDATE guarded by event_id
            // Example: maintain orders_summary read model
            try (Connection c = ds.getConnection()) {
                c.setAutoCommit(false);
                applyEvent(c, e); // writes to read-model tables,
deduped by event_id

                // Advance checkpoint transactionally with the side
effect — this is the exactly-once-effect primitive.
                // If this transaction commits, the event will not be
reprocessed; if it rolls back, it will.
                try (PreparedStatement ps = c.prepareStatement(
                        "INSERT INTO
projection_checkpoint(projector_name, last_position) VALUES (?, ?) "
                      + "ON CONFLICT (projector_name) DO UPDATE SET
last_position = EXCLUDED.last_position")) {
                    ps.setString(1, PROJECTOR);
                    ps.setLong(2, e.globalPosition());
                    ps.executeUpdate();
                }
```

```
                c.commit();
            }
        }
    }

    private void applyEvent(Connection c, StoredEvent e) throws SQLExce
ption {
        // Each event type maps to a deterministic, idempotent mutation
of the read model.
        // Example for OrderPlaced — insert if not already present
(dedup by event_id in a side table if needed).
        if ("OrderPlaced".equals(e.eventType())) {
            try (PreparedStatement ps = c.prepareStatement(
                    "INSERT INTO orders_summary(order_id, user_id,
status, total_cents, placed_at) "
                  + "VALUES ((?::jsonb)->>'order_id', (?::jsonb)-
>>'user_id', 'placed', ((?::jsonb)->>'total_cents')::bigint, now()) "
                  + "ON CONFLICT (order_id) DO NOTHING")) {
                ps.setString(1, e.payload()); ps.setString(2,
e.payload()); ps.setString(3, e.payload());
                ps.executeUpdate();
            }
        } else if ("OrderShipped".equals(e.eventType())) {
            try (PreparedStatement ps = c.prepareStatement(
                    "UPDATE orders_summary SET status='shipped',
shipped_at=now() WHERE order_id=(?::jsonb)->>'order_id'")) {
                ps.setString(1, e.payload());
                ps.executeUpdate();
            }
        }
        // Record event_id in a dedup table if the mutation itself is
not naturally idempotent
        try (PreparedStatement ps = c.prepareStatement(

"INSERT INTO projection_dedup(projector_name, event_id) VALUES (?, ?)
ON CONFLICT DO NOTHING")) {
            ps.setString(1, PROJECTOR); ps.setObject(2, e.eventId()); p
s.executeUpdate();
        }
    }
}
```

For lower latency than polling, use `LISTEN` / `NOTIFY` or Debezium CDC on `event_store` (see Ch 6 for CDC wiring), but keep the checkpoint discipline identical — the channel is an optimization, not the correctness mechanism.

Implementation II — EventStoreDB 23/24

EventStoreDB is a purpose-built event store with first-class streams, expected-revision concurrency, and persistent subscriptions.

```
// EventStoreDB Java client 5.x — append with expected revision,
subscribe to $all
import com.eventstore.dbclient.*;

EventStoreDBClient client = EventStoreDBClient.create(
    EventStoreDBConnectionString.parse("esdb://
esdb-1.internal:2113,esdb-2.internal:2113,esdb-3.internal:2113?
tls=true"));

// Append to a single aggregate stream — OCC via expectedRevision
EventData orderPlaced = EventData.builderAsJson("OrderPlaced", new Orde
rPlaced("ord_9f3a", "usr_42", 4999))
    .eventId(UUID.randomUUID()) // idempotency key — server dedups on
eventId
    .build();

// expectedRevision: NO_STREAM (-1) for new aggregate, or exact
revision, or ANY for no OCC (avoid)
AppendToStreamOptions opts = AppendToStreamOptions.get()
    .expectedRevision(ExpectedRevision.expectedRevision(5L)); // we
loaded 0..5, so next is 6 with expected 5
    // For a new stream: ExpectedRevision.NO_STREAM
    // For blind append (no concurrency check): ExpectedRevision.ANY —
loses OCC, do not use for aggregates

try {
    WriteResult result = client.appendToStream("orders-ord_9f3a",
opts, orderPlaced).get();
    long nextRevision = result.getNextExpectedRevision(); // 6
} catch (WrongExpectedVersionException ex) {
    // Another writer won — reload 0..current, re-evaluate, retry
}

// Subscribe a projector to the global log — ordered, resumable from
checkpoint
// Persistent subscription or catch-up subscription; catch-up is
simpler to reason about for exactly-once-effect

CheckpointStore checkpointStore = new PostgresCheckpointStore(dataSourc
e); // same table as above

SubscriptionListener listener = new SubscriptionListener() {
    @Override public void onEvent(Subscription sub, ResolvedEvent re) {
        // Idempotent apply — same pattern as Postgres projector:
(apply + checkpoint) atomically
        // EventStoreDB delivers at-least-once; dedup by eventId in the
read model transaction
        applyToReadModel(re.getEvent()); // must be idempotent
        checkpointStore.save("order_summary",
re.getEvent().getPosition()); // commit position atomically with apply
```

```
    }
};

// Catch-up: read from last checkpoint, then stay subscribed
Position lastPos = checkpointStore.load("order_summary"); //
Position.START for first run
client.subscribeToAll(listener,

SubscribeToAllOptions.get().fromPosition(lastPos)); // resumes from
checkpoint, no missed or duplicated effect
```

Operational notes for EventStoreDB:

- **Streams are cheap** — one per aggregate instance is the intended design. Do not multiplex aggregates into a single stream; you lose per-aggregate OCC.
- **Projections** — EventStoreDB has server-side projections (JavaScript) that can emit linked streams. Use them for simple category/global transforms, but keep business projections in your own service for testability and exactly-once checkpoint control.
- **Scavenging** — deleted/tombstoned streams still occupy index space until scavenged. Schedule scavenges in low-traffic windows.
- **Clustering** — 3-node cluster with Raft-like leader election; writes go to leader, reads can go to followers with `NodePreference`. Monitor `esdb WorthExpectedVersion` conflicts and subscription lag.

```
flowchart TB
    Cmd[Command] --> Handler[Command handler]
    Handler --> Load["Load stream
orders-ord_9f3a 0..N"]
    Load --> Agg[Aggregate
fold + decide]
    Agg -- "emit event" --> Append["Append
expectedRevision=N"]
    Append --> ES[(Event store
Postgres / EventStoreDB)]
    ES --> Sub[Subscription
global order]
    Sub --> Proj[Projector
idempotent apply]
    Proj --> RM[(Read model
orders_summary)]
    Proj --> CP[(Checkpoint
last_global_position)]
    RM --> Query[Query API
GET /orders?user=...]

    ES -.->|replay 0..N| Replay[Rebuild /
new projector]
    Replay -.-> RM2[(New read model)]

    style ES fill:#e8f5e9
    style Proj fill:#e3f2fd
    style RM fill:#fff3e0
```

Figure 5-3: CQRS + event sourcing topology. The write path is per-aggregate OCC; the read path is a lagging, replayable projection with a checkpoint. New read models are built by replaying, not by migrating the write model.

Schema evolution: events live forever

Events are persisted forever, so their schemas must evolve without breaking replay. The rule: **never mutate a persisted event in place** — version and upcast.

Strategies

Strategy	How	When
Versioned event types	`OrderPlaced_v1`, `OrderPlaced_v2` as distinct types	Explicit, simple for large breaking changes
Weak schema (JSON + optional fields)	Add optional fields, never remove required ones	Works for additive changes (new optional `coupon_code`)
Upcaster chain	`v1 JSON → v2 JSON` transformer applied on read	Keeps storage on v1 while code reads v2; chain `v1→v2→v3`
Copy-on-write migration	Rewrite old events into new streams offline, switch readers	For deep restructures; requires careful cutover

```
// Upcaster — applied on load before folding
public interface Upcaster { JsonNode upcast(JsonNode payload, int fromV
ersion, int toVersion); }

public class OrderPlacedUpcaster implements Upcaster {
    @Override public JsonNode upcast(JsonNode payload, int
fromVersion, int toVersion) {
        // v1 had `total` as string "49.99"; v2 is `total_cents` int
        if (fromVersion == 1 && toVersion >= 2) {
            ObjectNode n = (ObjectNode) payload;
            String totalStr = n.get("total").asText(); // "49.99"
            long cents = Math.round(Double.parseDouble(totalStr) * 100)
;
            n.remove("total");
            n.put("total_cents", cents);
            n.put("event_version", 2);
        }
        // chain further: v2 → v3 if needed
        return payload;
    }
}

// On load: read event_version from row, upcast stepwise to current
domain version before apply
StoredEvent raw = loadOne(...);
JsonNode current = upcasterChain.upcast(raw.payload(),
raw.eventVersion(), OrderPlaced.CURRENT_VERSION);
OrderPlaced domain = objectMapper.treeToValue(current, OrderPlaced.clas
s);
```

Rules:

- **Additive only on the wire.** New consumers must handle old events (missing optional fields). Old events are never rewritten to add new fields retroactively.
- **Keep the event name stable.** `OrderPlaced` stays `OrderPlaced`; bump `event_version` inside. Renaming creates a new stream of type that replays diverge on.
- **Test replay.** Your CI should load a fixture of real historical events (v1..vN) and assert that `fold(apply, empty, history) == expectedState`. Without this, a schema change silently breaks time travel.

Snapshots: optimization, not truth

Replaying thousands of events to load one aggregate is slow. A **snapshot** caches the folded state at a revision so loading becomes `load snapshot at N + replay N+1..current`.

```
-- Save snapshot every 100 events or when stream exceeds a threshold
INSERT INTO aggregate_snapshot(stream_id, stream_revision, state)
VALUES ('orders-ord_9f3a', 100, '{"status":"authorized","total_cents":4
999}'::jsonb)
ON CONFLICT (stream_id) DO UPDATE SET stream_revision=EXCLUDED.stream_r
evision, state=EXCLUDED.state;
```

```
AggregateState loadAggregate(String streamId) {
    Snapshot snap = snapshotStore.load(streamId); // may be null
    long from = (snap == null) ? 0 : snap.revision() + 1;
    List<StoredEvent> tail = eventStore.load(streamId, from);
    AggregateState state = (snap == null) ? AggregateState.EMPTY :
snap.state();
    for (StoredEvent e : tail) state = apply(state, upcast(e));
    return state;
}
```

Constraints:

- Snapshots are **derived** — they can be deleted and rebuilt from events without loss. Never treat a snapshot as authoritative over the event stream.
- Snapshot frequency is a throughput knob: every 50–200 events for hot aggregates, less for cold ones. Measure load latency vs. write amplification.
- EventStoreDB has built-in `$maxCount` / explicit snapshot streams — same principle, different API.

Distributed-systems lens: consistency, ordering, and failure

Eventual consistency between write and read

After a successful `append`, the aggregate's state is immediately visible to the next command on that aggregate (it reloads the new event), but read models lag by projector latency (milliseconds to seconds). This is inherent

to CQRS — there is no cross-model transaction. Clients must be designed for it:

- **Read-after-write for the writer** — redirect or poll the aggregate stream, not the read model, or return the new state in the command response.
- **Other readers** — accept staleness, or expose a `projection_lag` metric and let callers wait if they need freshness.
- **Cross-aggregate queries** — a read model that joins `orders` and `payments` sees each stream at a different watermark; there is no global snapshot isolation across streams without additional coordination. State the staleness bound in the API contract.

Ordering for projections

- **Per-aggregate order** — guaranteed by per-stream revisions; a projector that processes one aggregate's events sequentially is correct by construction.
- **Cross-aggregate order** — only eventual in the global log. If two aggregates emit causally related events (order placed → payment authorized in a different aggregate), the projector may see them in either global order unless you enforce causation metadata (`causationId`, `correlationId`) and make projection logic commutative or explicitly ordered.
- **Partition key for scale** — when fanning projections to multiple consumers, partition by `stream_id` (aggregate id) so all events for one aggregate go to one projector instance — otherwise you lose per-aggregate ordering.

Failure and recovery

Failure	Effect	Recovery
Projector crashes mid-batch	Some events applied, checkpoint not advanced	On restart, replay from last checkpoint — apply is idempotent, so reprocessing is safe
Event store unavailable	Commands fail (cannot load or append)	No partial writes — OCC ensures all-or-nothing per append; retry with backoff

Failure	Effect	Recovery
Read model DB unavailable	Projections stall, commands still succeed	Projector retries; read model catches up when DB returns — no data loss
Schema bug in projector	Read model diverges	Fix projector, **replay** the event log into a fresh read model, cut over — no migration of the write side

The essential property: the event log is durable and replayable, so any derived state can be rebuilt. This is why the log — not any read model — is the system of record.

```
stateDiagram-v2
    [*] --> Empty: stream does not exist
    Empty --> Placed: PlaceOrder → OrderPlaced
    Placed --> Authorized: AuthorizePayment → PaymentAuthorized
    Authorized --> Reserved: ReserveInventory → InventoryReserved
    Reserved --> Shipped: ShipOrder → OrderShipped
    Placed --> Cancelled: CancelOrder → OrderCancelled
    Authorized --> Cancelled: CancelOrder → OrderCancelled
    Shipped --> [*]
    Cancelled --> [*]
    Note right of Authorized: Aggregate enforces:
placed before authorized
authorized before shipped
no ship after cancel
```

Figure 5-4: Aggregate state machine — the set of valid event sequences. The aggregate rejects commands that would emit an invalid transition; the projector never needs to enforce these invariants again.

When not to use event sourcing and CQRS

These patterns have real costs — model them before adopting:

- **Modeling overhead** — every state change becomes an explicit event type with versioning, upcasting, and projection logic. For CRUD-heavy domains with little audit or replay value (user preferences, CMS content), a current-state table with an audit log side table is simpler and sufficient.

- **Operational overhead** — you now operate an event store, projector(s) with checkpoint monitoring, snapshot strategy, replay tooling, and schema evolution discipline. Teams that adopt event sourcing for one aggregate and then avoid operating its projector create the worst outcome: history without the ability to use it.
- **Query complexity** — ad-hoc queries against an event log are painful. Without a well-maintained read model, even "count orders by status" requires a full fold.
- **Small aggregates only** — if your invariant spans many entities ("no more than 100 active orders per merchant"), an aggregate-per-order cannot enforce it atomically. You need a different boundary or a saga — not a larger aggregate.

Heuristics for when the trade is worthwhile:

- The domain has **audit, compliance, or temporal query** requirements that are load-bearing, not decorative.
- You need **replay** to fix bugs or build new read models without backfilling from backups.
- The write and read access patterns are **sharply different** (high-write, fan-out reads, or per-query denormalization that would burden the write model).
- The team will **operate the projector** with the same seriousness as the write path — lag alerting, checkpoint durability, replay runbooks.

Otherwise, keep a normalized write table, add an `audit_log` table or CDC (Ch 6), and use CQRS only where a specific query just says it needs it.

Key takeaways

- Event sourcing stores the ordered sequence of domain events as the source of truth; current state is a pure left-fold over that sequence — giving complete audit history, temporal queries, and replay for free.
- The event store contract is **per-stream total order + expected-version optimistic concurrency + idempotent append by eventId** — the stream is the lock, and two concurrent writers to the same aggregate are serialized or rejected.

- Aggregates are **consistency boundaries**: one command, one aggregate, invariant enforcement on the fold — small, transactionally loaded whole, with no cross-aggregate queries inside.
- **CQRS** separates a normalized write model from denormalized read models; the two are bridged by **projectors** that consume the global log in order, apply idempotently, and checkpoint atomically with their side effects for effectively-once projection.
- **Schema evolution** is append-only: version events, upcast on read, test replay against historical fixtures — never mutate persisted events.
- **Snapshots** are a load-time optimization (state at revision N plus tail replay), not a source of truth — they can be rebuilt or discarded without loss.
- The read model is **eventually consistent** with the write model; design clients for the window, partition projectors by aggregate id to preserve per-aggregate order, and remember that any derived state can be rebuilt by replaying the log.
- Do not adopt these patterns for CRUD without audit/replay needs — the modeling and operational costs are substantial and must be justified per aggregate, not per system.

Further reading

- Fowler, M. "Event Sourcing." martinfowler.com, 2005. https://martinfowler.com/eaaDev/EventSourcing.html — The canonical definition.
- Young, G. "CQRS Documents." cqrs.wordpress.com — The original CQRS formulation and its separation from event sourcing.
- Vernon, V. *Implementing Domain-Driven Design* (Addison-Wesley, 2013) — Ch. on aggregates and bounded contexts as consistency boundaries.
- EventStoreDB documentation 23/24 — Streams, expected revision, subscriptions, projections. https://developers.eventstore.com/
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11 — The log, event sourcing, and stream-table duality.
- Postgres documentation 16 — Transaction isolation, advisory locks, `LISTEN` / `NOTIFY`. https://www.postgresql.org/docs/16/

- Rinat Abdullin / Event Sourcing articles — Upcasting, snapshotting, and versioning patterns. https://abdullin.com/
- Brandolini, A. *EventStorming* — Collaborative modeling technique for discovering events, commands, and aggregates before cutting code.

Chapter 6 — The Outbox Pattern and the Dual-Write Problem

What this chapter covers. Every service that writes to a database and publishes an event faces the same trap: `UPDATE db` then `publish(event)` is two writes with no atomicity. If the process crashes between them, the database and the event log diverge — silently, permanently, without an error to alert on. This is the **dual-write problem**, and it is the most common source of ghost orders, missing payments, and phantom inventory in event-driven systems. This chapter names the failure precisely, shows why naive fixes (publish-then-commit, two-phase commit, last-resource gambit) do not solve it, and builds the two production patterns that do: the **transactional outbox** (polling relay and CDC/transaction-log tailing with Debezium) and, where needed, the **inbox** on the consumer. We implement both with real **Postgres 16**, **Debezium 2.5**, and **Kafka 3.7** configuration, cover the distributed-systems guarantees (exactly-once effect, ordering, and the idempotent-consumer contract that makes at-least-once delivery safe), and close with operational guidance: monitoring the relay, handling poison outbox rows, and when to reach for Kafka transactions instead.

Learning goals — after this chapter you should be able to:

- Define the dual-write problem and enumerate the four failure cases of `commit-then-publish` and `publish-then-commit` with their observable consequences.
- Explain why distributed transactions (XA/2PC) are the wrong fix for database-plus-broker atomicity in modern backends.
- Implement the transactional outbox on Postgres 16: DDL, atomic business write + outbox insert, and the relay that publishes to Kafka exactly once (effectively).

- Configure Debezium 2.5 CDC as the relay: `pgoutput` logical replication, connector config, outbox event router (SMT), and offset durability.
- Contrast polling relay vs. CDC on latency, load, ordering, failure modes, and operational cost, and choose correctly.
- Describe the end-to-end exactly-once-effect pipeline: transactional outbox → at-least-once publish → idempotent consumer with inbox/dedup table, and what each stage guarantees.
- Reason about ordering (per-aggregate outbox order → per-partition Kafka order via deterministic key), and what breaks ordering and how to detect it.
- Operate the pattern: relay lag monitoring, poison-row handling, exactly-once sink options (Kafka transactions vs. idempotent consumers), and schema evolution of outbox payloads.

> **Boundary note.** Vol 6, Chapter 9 — Idempotency, Deduplication, and Exactly-Once — develops the *theory* that makes the outbox correct end-to-end: why exactly-once delivery is impossible over a lossy network, the atomic check-and-record protocol, and the idempotency-key discipline that makes at-least-once safe. Chapter 2 in this volume maps that theory onto *broker delivery semantics* (at-most/at-least/effectively-once, Kafka PID+sequence and transactions). Chapter 5 — Event Sourcing and CQRS — uses the same event-store append invariant (expected-version OCC) and publishes derived events from the log. This chapter is narrower and more operational: the *dual-write between a business table and a broker* and the *transactional outbox mechanics* (polling vs. CDC) that eliminate it. For the formal reasoning about idempotency and the event-store's own concurrency contract, read Vol 6 Ch 9 and Ch 5 respectively; for the relay wire and its failure modes, read here.

The dual-write problem: two writes, no atomicity

Consider the canonical order service that must both update its relational database and publish an event so downstream services (inventory, payments, search) can react:

```
// The naive sequence — looks correct, is not
public void placeOrder(PlaceOrder cmd) {
    db.execute("INSERT INTO orders(id, status, total_cents) VALUES (?,
'placed', ?)",
               cmd.orderId(), cmd.totalCents());          // write 1:
database
    kafkaProducer.send(new ProducerRecord<>("orders",     // write 2:
broker
        cmd.orderId(), new OrderPlaced(cmd.orderId(), cmd.totalCents())
));
    // What if the process dies here, between the two writes?
}
```

There are two orderings, and both fail:

```
sequenceDiagram
    participant S as Service
    participant DB as Postgres
    participant K as Kafka
    Note over S: Case A: commit then publish
    S->>DB: BEGIN; INSERT orders ...; COMMIT
    DB-->>S: ack — row durable
    Note over S: CRASH — publish never happens
    Note over K: Event never published.
Downstream never learns
order exists. Silent divergence.
    S->>K: send(OrderPlaced) — never reached
```

```
sequenceDiagram
    participant S as Service
    participant DB as Postgres
    participant K as Kafka
    Note over S: Case B: publish then commit
    S->>K: send(OrderPlaced)
    K-->>S: ack — event durable
    S->>DB: BEGIN; INSERT orders ...; COMMIT
    Note over S: CRASH — commit never happens
or COMMIT fails (constraint, deadlock)
    Note over K: Event published for an order
that does not exist.
Downstream acts on a ghost.
```

More precisely, there are four failure modes and each is observable as a different incident:

Sequence	Failure point	DB state	Broker state	Observable bug
Commit → publish	Crash after commit, before publish	Row exists	No event	**Ghost write** — order in DB, inventory never reserved, search never indexed
Commit → publish	Publish fails (broker down, timeout, serialization error)	Row exists	No event or ambiguous (was it written?)	Same as above, plus retry ambiguity
Publish → commit	Crash after publish, before commit	No row	Event exists	**Ghost event** — inventory reserved for non-existent order, payment attempted
Publish → commit	Commit fails (constraint violation, deadlock)	No row	Event exists	Same as above

A retry wrapper does not help — it can only turn ghost writes into ghost events or vice versa. The root cause is that two independent systems (database and broker) have no shared transaction. There is no `BEGIN` that spans Postgres and Kafka.

> **Why not XA / two-phase commit?** XA (distributed transactions across DB and broker) provides atomicity via a coordinator, prepare, and commit phase. In theory it solves dual writes. In practice: (1) most managed Postgres and virtually no cloud Kafka expose XA to application code; (2) the coordinator is a single point of failure and a latency tax on every write; (3) a failed coordinator leaves participants holding locks with heuristic outcomes; (4) Kafka's transaction model (Ch 3) is broker-internal, not XA. The industry consensus for business-transaction + event publishing is the transactional outbox, not XA. See Vol 6, Ch 7 for why coordinator-based atomic commit scales poorly.

The invariant: one local transaction

The outbox pattern replaces two distributed writes with **one local transaction plus an asynchronous relay**:

- **Inside one database transaction**, atomically write the business row(s) *and* an outbox row that describes the event to be published.
- **After commit**, a relay process reads outbox rows and publishes them to the broker. Publishing may be at-least-once, but the outbox row's lifecycle makes the effect exactly once when paired with an idempotent consumer.

```
BEFORE (dual write — no atomicity):
  tx { INSERT orders }  +  send(Kafka)    — two systems, no shared comm
it

AFTER (outbox — one atomic write):
  tx { INSERT orders + INSERT outbox }    — one system, one commit
       ──────────────────┬──────────────────
                         └── relay ──► Kafka (at-least-once, deduped down
stream)
```

The database is the single source of truth for "what must be published." The broker is a derived, eventually consistent copy. If the relay is down, events accumulate in the outbox — no divergence, just lag.

Figure 6-1: The transactional outbox invariant. Business state and the intent to publish are committed atomically; the relay makes that intent durable in the broker. The only observable delay is relay lag, not divergence.

Implementing the outbox on Postgres 16

DDL

```
-- Postgres 16 — transactional outbox
CREATE TABLE orders (
    id          TEXT PRIMARY KEY,
    user_id     TEXT        NOT NULL,
    status      TEXT        NOT NULL DEFAULT 'placed',
    total_cents BIGINT      NOT NULL CHECK (total_cents > 0),
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- The outbox: one row per event to be published. Rows are deleted (or
marked) after successful publish.
CREATE TABLE outbox (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    aggregate_type TEXT       NOT NULL,              -- e.g., 'Order'
    aggregate_id   TEXT       NOT NULL,              -- e.g.,
'ord_9f3a' — used as Kafka partition key
    event_type     TEXT       NOT NULL,              -- e.g.,
'OrderPlaced'
    event_version  INT        NOT NULL DEFAULT 1,
    payload        JSONB      NOT NULL,              -- event body —
validated by app
    headers        JSONB      NOT NULL DEFAULT '{}'::jsonb, --
traceparent, correlation_id, causation_id
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    -- publish tracking — used by polling relay; CDC relay can omit
these and delete on publish
    published_at   TIMESTAMPTZ,
    -- idempotency: event_id == id; retries use the same UUID
    UNIQUE (id)
);
CREATE INDEX outbox_unpublished ON outbox (created_at, id) WHERE publis
hed_at IS NULL;
CREATE INDEX outbox_aggregate ON outbox (aggregate_type, aggregate_id,
created_at);

-- Consumer-side inbox/dedup (see "Closing the loop" below) — often
colocated with the consumer's DB
CREATE TABLE inbox (
    event_id   UUID        PRIMARY KEY,              -- outbox.id —
dedup key
    event_type TEXT        NOT NULL,
    processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Design choices:

- **`aggregate_id` as partition key.** All events for one aggregate must land on one Kafka partition to preserve per-aggregate order. The outbox stores `aggregate_id` so the relay can set the Kafka key deterministically.
- **`payload` as JSONB.** Keeps the outbox decoupled from a specific serialization (Avro/Protobuf bytes can be stored as `BYTEA` if you prefer wire fidelity; JSONB is simpler for polling).
- **`published_at` vs. delete.** Polling relays usually `UPDATE ... SET published_at=now()` or `DELETE` after publish. CDC relays delete or tombstone via the connector — no `published_at` column needed. Choose one discipline and be consistent.
- **Retention.** Published rows should be deleted or archived after a retention window (e.g., 7 days) — the Kafka topic is the durable copy after publish. Keep them long enough to debug "was this event published?" without growing unbounded.

Atomic business write + outbox insert

```
// Java 17, plain JDBC — one transaction, two inserts, one commit
public class OrderService {
    private final DataSource ds;

    public void placeOrder(PlaceOrder cmd) throws SQLException {
        UUID eventId = UUID.randomUUID(); // stable for this attempt;
on retry from caller, reuse the same eventId
        String payload = """
            {"order_id":"%s","user_id":"%s","total_cents":
%d,"placed_at":"%s"}
            """.formatted(cmd.orderId(), cmd.userId(), cmd.totalCents()
, Instant.now());

        try (Connection c = ds.getConnection()) {
            c.setAutoCommit(false);
            try {
                // 1. Business write
                try (PreparedStatement ps = c.prepareStatement(
                        "INSERT INTO orders(id, user_id, status,
total_cents) VALUES (?, ?, 'placed', ?)")) {
                    ps.setString(1, cmd.orderId());
                    ps.setString(2, cmd.userId());
                    ps.setLong(3, cmd.totalCents());
                    ps.executeUpdate();
                }

                // 2. Outbox write — same transaction, same commit
                try (PreparedStatement ps = c.prepareStatement(
                        "INSERT INTO outbox(id, aggregate_type,
aggregate_id, event_type, payload, headers) "
                      + "VALUES (?, 'Order', ?,
'OrderPlaced', ?::jsonb, ?::jsonb)")) {
                    ps.setObject(1, eventId);
                    ps.setString(2, cmd.orderId());
                    ps.setString(3, payload);
                    ps.setString(4, """
                        {"traceparent":"%s","correlation_id":"%s"}
                        """.formatted(currentTraceparent(), cmd.correla
tionId()));
                    ps.executeUpdate();
                }

                c.commit(); // ← atomic boundary: both rows or neither
            } catch (SQLException ex) {
                c.rollback();
                // Constraint violation, deadlock, etc. — no event will
be published, which is correct
                throw ex;
            }
        }
```

```
        // After commit, the relay will publish. The service does not
publish directly.
    }
}
```

What this guarantees:

- If `commit` succeeds, both the order row and the outbox row are durable. The relay *will* publish — eventually, at-least-once — even if the service crashes immediately after commit.
- If `commit` fails, neither row is durable. No event will be published. No ghost event.
- The `eventId` (`outbox.id`) is the end-to-end idempotency key. If the HTTP caller retries `placeOrder` with the same `orderId`, the `INSERT orders` will conflict (primary key) and the second `INSERT outbox` will use a *new* `eventId` only if a new row is actually created — never publish a duplicate for a failed command.

> **Idempotent command handling.** If the API layer retries `placeOrder` with a client-supplied idempotency key (Vol 6 Ch 9), check that key *inside the same transaction* before inserting. Otherwise two concurrent requests with the same business key can both commit and produce two outbox rows for one logical command.

Relay I — Polling publisher

The simplest relay: a background process polls `outbox WHERE published_at IS NULL` in `created_at` order, publishes each row to Kafka, and marks it published.

```
// Polling relay — Java 17, kafka-clients 3.7, single-threaded for
ordering
public class OutboxPollingRelay implements Runnable {
    private final DataSource ds;
    private final KafkaProducer<String, String> producer;
    private static final String TOPIC = "orders";
    private static final int BATCH = 200;
    private static final Duration POLL_INTERVAL =
Duration.ofMillis(500);

    @Override public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            try {
                List<OutboxRow> batch = fetchUnpublished(BATCH);
                for (OutboxRow row : batch) {
                    // Deterministic key → per-aggregate order on one
Kafka partition
                    ProducerRecord<String, String> rec = new ProducerRe
cord<>(
                        TOPIC, row.aggregateId(), row.payload());
                    rec.headers().add("event_type", row.eventType().get
Bytes());
                    rec.headers().add("event_id", row.id().toString().g
etBytes());
                    rec.headers().add("traceparent",
row.headers().get("traceparent").getBytes());

                    // Synchronous send — do not mark published until
Kafka acks
                    // acks=all, enable.idempotence=true on the
producer (see Ch 3) prevents broker-side duplicates on retry
                    RecordMetadata meta = producer.send(rec).get(10, Ti
meUnit.SECONDS);

                    // Mark published only after ack — at-least-once
publish, at-most-once mark
                    markPublished(row.id());
                    // Alternatively DELETE FROM outbox WHERE id=? —
leaner table, same guarantee
                }
                if (batch.size() < BATCH) Thread.sleep(POLL_INTERVAL.to
Millis());
            } catch (Exception ex) {
                // Publish failed or DB error — do not mark published;
next tick will retry (at-least-once)
                log.warn("Outbox relay tick failed — will retry", ex);
                sleepQuietly(POLL_INTERVAL);
            }
        }
    }
```

```
    private List<OutboxRow> fetchUnpublished(int limit) throws SQLExcep
tion {
        // FOR UPDATE SKIP LOCKED — multiple relay instances can run
concurrently without double-publishing
        // Each row is locked by the relay that fetched it; others skip
it.
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(
                 "SELECT id, aggregate_type, aggregate_id, event_type,
payload, headers "
               + "FROM outbox WHERE published_at IS NULL "
               + "ORDER BY created_at, id LIMIT ? FOR UPDATE SKIP
LOCKED")) {
            ps.setInt(1, limit);
            try (ResultSet rs = ps.executeQuery()) {
                List<OutboxRow> out = new ArrayList<>();
                while (rs.next()) out.add(mapRow(rs));
                return out;
            }
        }
    }

    private void markPublished(UUID id) throws SQLException {
        try (Connection c = ds.getConnection();
             PreparedStatement ps = c.prepareStatement(
                 "UPDATE outbox SET published_at = now() WHERE id = ?")
) {
            ps.setObject(1, id);
            ps.executeUpdate();
            c.commit();
        }
    }
}
```

```
# Kafka producer for the relay — durability first (Ch 3)
acks=all
enable.idempotence=true
retries=5
delivery.timeout.ms=120000
max.in.flight.requests.per.connection=5
compression.type=lz4
```

Behavior under failure:

- **Relay crashes after publish but before `markPublished`.** The row stays unpublished; the next relay tick republishes — **at-least-once** to

Kafka. Two records with the same `event_id` may exist in the topic. The consumer's dedup table (inbox) makes the effect exactly once.

- **Relay crashes before publish.** Row stays unpublished; next tick publishes — no duplicate.
- **Multiple relay instances.** `FOR UPDATE SKIP LOCKED` shards work naturally. Two instances never publish the same row concurrently; on failover, an unmarked row is picked up by the survivor.
- **Poison row** (payload that always fails to serialize/publish). Without handling, the relay retries the same row forever and stalls the batch behind it. Mitigate with a `publish_attempts` counter and a dead-letter outbox table (see Ch 8) — after N failures, move the row aside and alert.

Pros and cons:

Aspect	Polling relay
Latency	Poll interval (500ms–2s typical) + DB round-trip; no sub-millisecond streaming
DB load	Periodic `SELECT` with index scan; fine for thousands/sec, visible at tens of thousands/sec
Ordering	`ORDER BY created_at, id` gives global publish order; Kafka partition key restores per-aggregate order
Ops	No extra infrastructure; just the service + DB + Kafka
Failure mode	At-least-once publish; relies on consumer dedup for correctness
Schema coupling	Reads JSONB; must handle payload evolution

Relay II — CDC with Debezium 2.5

Change Data Capture tails the database's own transaction log (Postgres WAL) and streams committed changes to Kafka with no polling and no `published_at` column. **Debezium** is the standard CDC engine.

Postgres setup

```
-- postgresql.conf (or ALTER SYSTEM)
wal_level = logical                 -- must be logical for decoding
max_replication_slots = 10
max_wal_senders = 10
max_replication_slots and max_wal_senders must exceed the number of Deb
ezium connectors + reserves
```

```
-- One publication for Debezium — limit to the outbox table so only
outbox changes are streamed
CREATE PUBLICATION debezium_outbox_pub FOR TABLE outbox;
-- Alternative: publish the business tables too if you want CDC for
other projections — but scope tightly

-- Role for Debezium — REPLICATION privilege, plus SELECT on the
published tables
CREATE ROLE debezium_user WITH LOGIN REPLICATION PASSWORD '...';
GRANT SELECT ON outbox TO debezium_user;
GRANT SELECT ON orders TO debezium_user; -- if also published
```

Debezium connector configuration

```
// Debezium 2.5 — Postgres connector with outbox event router (SMT)
// POST to /connectors  {"name":"pg-outbox-connector","config":{ ... }}
{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnecto
r",
  "database.hostname": "pg-primary.internal",
  "database.port": "5432",
  "database.user": "debezium_user",
  "database.password": "${file:/secrets/pg-
password.properties:password}",
  "database.dbname": "orders_db",
  "topic.prefix": "pg",
  "publication.name": "debezium_outbox_pub",
  "slot.name": "debezium_outbox_slot",
  "plugin.name": "pgoutput",
  "publication.autocreate.mode": "filtered",

  // Snapshot — take consistent snapshot on first start, then stream
WAL
  "snapshot.mode": "initial",

  // Offsets and schema history — both must be durable (Kafka topics,
RF=3)
  "offset.storage.topic": "debezium-offsets",
  "offset.storage.replication.factor": 3,
  "schema.history.internal.kafka.topic": "debezium-schema-history",
  "schema.history.internal.kafka.bootstrap.servers": "kafka-1.internal:
9092,kafka-2.internal:9092",
  "schema.history.internal.kafka.recovery.poll.interval.ms": 5000,

  // Heartbeats — keep WAL slot advancing even when outbox is idle, so
WAL does not grow unbounded
  "heartbeat.interval.ms": "10000",
  "heartbeat.topic.prefix": "__debezium-heartbeat",

  // Outbox Event Router SMT — turns each outbox row into one Kafka
record on the right topic/partition
  "transforms": "outbox",
  "transforms.outbox.type":
"io.debezium.transforms.outbox.EventRouter",
  "transforms.outbox.table.field.event.id": "id",
  "transforms.outbox.table.field.event.key": "aggregate_id",
  "transforms.outbox.table.field.event.payload": "payload",
  "transforms.outbox.table.field.event.payload.id": "aggregate_id",
  "transforms.outbox.table.expand.json.payload": "true",
  "transforms.outbox.route.by.field": "aggregate_type",
  "transforms.outbox.route.topic.replacement": "${routedByValue}",

  // Tombstone handling — delete outbox row after routing to avoid re-
emission on snapshot
```

```
  "tombstones.on.delete": "false",
  "delete.handling.mode": "rewrite",
  "transforms.outbox.table.fields.additional.placement": "headers:heade
rs"
}
```

How the outbox event router works:

1. Debezium captures the committed `INSERT INTO outbox` from WAL (not by polling, but by decoding `pgoutput`).
2. The **EventRouter SMT** reads `aggregate_type` to choose the destination topic (e.g., `Order` → `orders`), `aggregate_id` to set the Kafka key (per-aggregate ordering), and `payload` as the Kafka value. Headers flow through as Kafka headers.
3. The resulting Kafka record is produced transactionally by the connector's internal producer. The connector commits its WAL offset only after the Kafka produce succeeds — so **WAL position and Kafka publish are atomically paired** at the connector's offset granularity.
4. After routing, the connector can be configured to emit a tombstone or the application can `DELETE FROM outbox` — either way, the row does not remain to be re-polled.

```
sequenceDiagram
    participant App as Service
    participant PG as Postgres WAL
    participant Deb as Debezium
connector
    participant K as Kafka
    App->>PG: COMMIT { INSERT orders + INSERT outbox }
    Note over PG: WAL entry durable,
publication decodes it
    PG->>Deb: pgoutput: outbox INSERT
id=evt_abc, aggregate_id=ord_9f3a
payload={...}
    Deb->>Deb: EventRouter SMT:
topic=orders, key=ord_9f3a
headers=traceparent
    Deb->>K: produce orders {key=ord_9f3a, value=payload}
    K-->>Deb: ack
    Deb->>Deb: commit offset
(slot LSN advances)
    Note over PG: WAL segment for that LSN
now eligible for recycling
```

Figure 6-2: CDC relay. The WAL is the source of truth; Debezium decodes committed outbox inserts and routes them to Kafka. No publish-before-commit is possible because only committed WAL entries are decoded.

Pros and cons:

Aspect	CDC / Debezium
Latency	Near-real-time — WAL decode + SMT + Kafka produce, typically 10–100ms
DB load	Negligible on hot path — reads WAL, not the table; no polling queries
Ordering	Per-aggregate order preserved via key; global order is WAL commit order
Ops	Extra infrastructure — Kafka Connect cluster (3 workers, RF=3), replication slot, WAL retention monitoring, SMT versioning
Failure modes	Slot lag → WAL retention → disk pressure on primary; connector offset loss → re-emission (at-least-once, dedup downstream)
Schema coupling	SMT reads row shape; outbox schema changes require connector reconfiguration + snapshot consideration

WAL retention — the operational non-negotiable

A Debezium replication slot **pins WAL** — Postgres cannot recycle WAL segments that the slot has not consumed. If the connector is down or lagging, WAL accumulates on the primary's disk and can fill it, stalling or crashing the database.

Monitor relentlessly:

```
-- Check slot lag — run every minute, alert on threshold
SELECT slot_name, slot_type, active, restart_lsn, confirmed_flush_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),
restart_lsn)) AS retained_wal
FROM pg_replication_slots WHERE slot_name = 'debezium_outbox_slot';
-- retained_wal > 1 GiB → investigate connector lag
-- restart_lsn far behind confirmed_flush_lsn → connector not acking

-- Heartbeat lag — last heartbeat in Kafka
SELECT now() - max(created_at) AS heartbeat_age FROM
debezium_heartbeat; -- via heartbeat topic consumer
```

Mitigations:

- Set `heartbeat.interval.ms=10000` so the slot advances even when the outbox is idle.
- Alert on `retained_wal > 4 GiB` or `slot active = false` for > 5 min.
- Have a runbook: if the slot is abandoned, `SELECT pg_drop_replication_slot('debezium_outbox_slot')` (after confirming no data loss window) and resnapshot — but understand this loses CDC continuity.
- Never set `max_slot_wal_keep_size = -1` (unlimited) without monitoring — it trades safety for disk exhaustion.

Closing the loop: the inbox and idempotent consumers

The outbox makes **publishing** at-least-once with no ghost writes. To make **consumption** effectively once, the consumer must be idempotent — processing the same `event_id` twice must have the same effect as once. The standard primitive is the **inbox** (consumer-side dedup table).

```
-- Colocated with the consumer's own database (same Postgres that holds
the consumer's business tables)
CREATE TABLE inbox (
    event_id     UUID        PRIMARY KEY,         -- outbox.id
    event_type   TEXT        NOT NULL,
    received_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- Optional: TTL cleanup after 30 days — beyond the max expected
redelivery window
```

```
// Consumer — exactly-once effect via (business write + inbox insert)
in one transaction
// Kafka 3.7, kafka-clients, Postgres 16

public class InventoryConsumer {
    private final DataSource ds;

    // Called for each record from Kafka topic "orders"
    public void handle(ConsumerRecord<String, String> rec) throws SQLEx
ception {
        UUID eventId = UUID.fromString(new String(rec.headers().lastHea
der("event_id").value()));
        String payload = rec.value();
        String aggregateId = rec.key(); // ord_9f3a — for partitioning/
ordering assertion

        try (Connection c = ds.getConnection()) {
            c.setAutoCommit(false);
            try {
                // 1. Dedup check + insert — the inbox IS the lock for
this event
                // INSERT ... ON CONFLICT DO NOTHING returns 0 rows if
already processed
                int inserted;
                try (PreparedStatement ps = c.prepareStatement(

"INSERT INTO inbox(event_id, event_type) VALUES (?, ?) ON CONFLICT DO
NOTHING")) {
                    ps.setObject(1, eventId);
                    ps.setString(2, new
String(rec.headers().lastHeader("event_type").value()));
                    inserted = ps.executeUpdate();
                }
                if (inserted == 0) {
                    c.rollback(); // already processed — idempotent no-
op, but still commit the offset
                    // Still commit the Kafka offset so we do not re-
deliver forever — see offset commit below
                    return;
                }

                // 2. Business logic — guarded by the inbox row, so
redelivery with same event_id will not re-execute
                // Example: reserve inventory for the order
                JsonNode order = objectMapper.readTree(payload);
                try (PreparedStatement ps = c.prepareStatement(
                        "INSERT INTO inventory_reservations(order_id,
sku, qty, status) VALUES (?, ?, ?, 'reserved') "
                      + "ON CONFLICT (order_id, sku) DO NOTHING")) {
                    for (JsonNode item : order.get("items")) {
```

```
                        ps.setString(1, order.get("order_id").asText())
;
                        ps.setString(2, item.get("sku").asText());
                        ps.setInt(3, item.get("qty").asInt());
                        ps.addBatch();
                    }
                    ps.executeBatch();
                }


c.commit(); // business write + inbox insert atomically — the exactly-
once-effect primitive
            } catch (SQLException ex) {
                c.rollback();
                throw ex; // retry via Kafka redelivery (do not commit
offset)
            }
        }
    }
}
```

The consumer's offset commit discipline determines whether the inbox guarantee holds:

- **Commit offset only after the `inbox + business` transaction commits.** If you auto-commit offsets before the transaction, a crash loses the event without processing it — at-most-once.
- **Do not commit offset when dedup short-circuits.** Or rather, do commit — but as a separate step after recognizing the duplicate. The event is already processed; advancing the offset is correct and prevents endless redelivery.

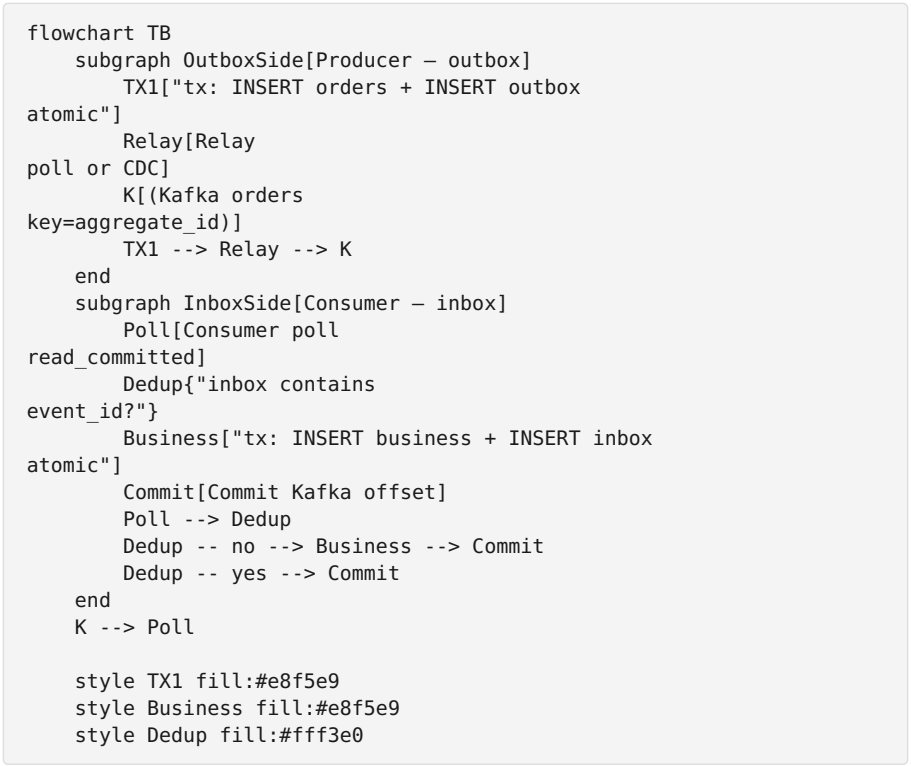


Figure 6-3: End-to-end effectively-once pipeline. The outbox makes publish at-least-once with no ghosts; the inbox makes consume idempotent; offset commit after the transaction prevents loss.

When you can skip the inbox

- The consumer's business operation is **naturally idempotent** (`INSERT ... ON CONFLICT DO NOTHING`, `SET status='shipped'` applied twice is the same). Then the inbox table is still useful for observability but not required for correctness.
- The consumer writes to a system that has its own dedup (e.g., an idempotency-keyed HTTP API) — pass `event_id` as the key and rely on that layer.

Kafka transactions as an alternative for the producer

If the producer is itself a Kafka consumer (stream processor), Kafka transactions (`transactional.id`, `sendOffsetsToTransaction`) can make

the **consume-transform-produce** loop atomic without an outbox — see Ch 3 and Ch 4. The outbox is the right pattern when the source of truth is a relational database, not a Kafka topic.

Ordering: what is preserved and what is not

The outbox preserves **per-aggregate order** if every link respects the key:

1. **Outbox table** — `ORDER BY created_at, id` (polling) or WAL commit order (CDC) gives a deterministic publish order per aggregate when transactions commit sequentially. Concurrent transactions committing in arbitrary order can interleave events for *different* aggregates — this is fine; only per-aggregate order matters.
2. **Kafka publish** — setting `key = aggregate_id` ensures all events for one aggregate land on one partition, which has total order. Without a deterministic key, two events for the same aggregate could land on different partitions and be consumed out of order.
3. **Consumer** — a single consumer instance per partition (one thread per partition) processes in offset order. Increasing `concurrency` beyond partition count does not increase parallelism for one aggregate but does not break order either.

What breaks ordering:

- **Republishing out of order** after a poison row is skipped — ensure the relay does not reorder around failures (process rows strictly in `created_at` order, or use per-aggregate sequence numbers).
- **Changing the partition count** on the destination topic — existing keys hash to different partitions after the change; new events for old aggregates may land elsewhere. Avoid changing partition count on outbox topics, or use custom partitioner with stable mapping.
- **Concurrent transactions** for the *same* aggregate — two concurrent `placeOrder` + `shipOrder` for `ord_9f3a` could commit in either order; the outbox will publish whichever committed first. Prevent with per-aggregate serialization at the application layer (`SELECT ... FOR UPDATE` on the aggregate row, or advisory lock) before the outbox insert.

```
-- Per-aggregate serialization to preserve causal order for one
aggregate's events
-- Acquire before the business + outbox transaction for contended
aggregates
SELECT pg_advisory_xact_lock(hashtext('orders-ord_9f3a'));
-- Now INSERT orders + INSERT outbox atomically — concurrent writers
for the same aggregate serialize here
```

Detecting disorder downstream: include `stream_revision` or `aggregate_version` in the payload and have the consumer assert monotonicity per `aggregate_id` — log or DLQ on gaps/inversions.

Operating the outbox in production

Monitoring

Signal	Query / metric	Alert threshold
Outbox lag (unpublished rows)	`SELECT count(*) FROM outbox WHERE published_at IS NULL`	> 1000 or > 5s p99 publish latency
Oldest unpublished age	`SELECT now() - min(created_at) FROM outbox WHERE published_at IS NULL`	> 30s
WAL retention (CDC)	`pg_replication_slots.retained_wal` (query above)	> 4 GiB
Slot active	`pg_replication_slots.active`	`false` for > 5 min
Connector task failed	Kafka Connect REST `GET /connectors/pg-outbox-connector/status`	`FAILED`
Consumer inbox duplicates	`SELECT count(*) FROM inbox WHERE received_at > now() - '1h'` vs. publish count	divergence > expected at-least-once rate

Signal	Query / metric	Alert threshold
Poison rows	`SELECT id, event_type, created_at FROM outbox WHERE published_at IS NULL AND created_at < now() - '5m'`	any row stuck > 5 min

Poison outbox rows

A row that always fails to publish (serialization bug, oversized payload > `max.request.size`, invalid topic) will stall a single-threaded ordered relay. Handle it:

```
ALTER TABLE outbox ADD COLUMN publish_attempts INT NOT NULL DEFAULT 0;
ALTER TABLE outbox ADD COLUMN last_error TEXT;

-- Relay increments attempts on failure; after N, move to DLQ table and
alert
-- DLQ table has same schema + error metadata, never auto-retried
CREATE TABLE outbox_dead_letter (LIKE outbox INCLUDING ALL);
ALTER TABLE outbox_dead_letter ADD COLUMN failed_at TIMESTAMPTZ NOT NUL
L DEFAULT now();
ALTER TABLE outbox_dead_letter ADD COLUMN error TEXT NOT NULL;
```

Exactly-once at the Kafka layer — when to use transactions

- **Polling relay with idempotent consumer (inbox)** — the default. Relay publishes at-least-once (`acks=all, enable.idempotence=true` prevents broker-side duplicates on retry but not relay-level republishing), consumer dedups — simple, correct, and works with any sink database.
- **Kafka transactions on the relay** — if you need the relay's offset tracking to be transactional (CDC connector already is), or if the producer is also a Kafka consumer in a stream processor, use `transactional.id` and `isolation.level=read_committed` on downstream consumers. Do not add transactions to a polling relay just to avoid an inbox — the inbox is cheaper.
- **Outbox deletion vs. marking.** CDC deletes (or tombstones) after routing; polling marks. Either way, ensure the "published" state is not lost before Kafka ack — otherwise you create ghost writes.

Payload evolution

Outbox payloads are events — apply the same versioning discipline as Ch 5:

- Add optional fields, never remove required ones without a major version and upcaster.
- Include `event_version` in the outbox row so consumers can dispatch correctly.
- For binary payloads (Avro/Protobuf), store `BYTEA` and pass `schema_id` in headers so the consumer can decode with the right schema — but keep `headers` JSONB for observability.

Anti-patterns and common incidents

- **Dual write "with retry"** — wrapping `commit; publish` in a retry loop turns ghost writes into ghost events nondeterministically. The incident is silent divergence that surfaces days later in reconciliation ("why does Postgres say 10,000 orders but the search index says 9,982?").
- **Outbox without inbox and without idempotent consumer** — at-least-once publish with a non-idempotent consumer doubles side effects (double charge, double inventory decrement). The outbox is half the pattern; the inbox completes it.
- **Publishing before commit** (or CDC on uncommitted WAL) — ghost events. Postgres `pgoutput` only decodes committed transactions, so CDC is safe; polling is safe only if it reads committed rows (it does, under `READ COMMITTED`).
- **No `aggregate_id` → no deterministic Kafka key → no per-aggregate ordering** — downstream consumers see `OrderShipped` before `OrderPlaced` for the same order and apply logic out of order.
- **Unmonitored replication slot** — WAL fills disk, primary stalls, all writes stop. The most operationally expensive CDC failure mode and entirely preventable with the queries above.
- **Outbox table as an audit log forever** — unbounded growth without deletion/archival after publish exhausts table bloat and vacuum. Retain 7–30 days, then archive or delete.

Key takeaways

- The dual-write problem is fundamental: a database commit and a broker publish have no shared atomicity — `commit-then-publish` creates ghost writes, `publish-then-commit` creates ghost events, and retries cannot fix it.
- The transactional outbox replaces two distributed writes with one local transaction: business row + outbox row committed atomically; an asynchronous relay publishes to the broker.
- The polling relay is simple and correct (poll `WHERE published_at IS NULL`, publish with `acks=all, enable.idempotence=true`, mark after ack, `FOR UPDATE SKIP LOCKED` for concurrency) with poll-interval latency and `SELECT` load.
- The CDC relay (Debezium 2.5 on `pgoutput` WAL decoding) is near-real-time with negligible hot-path load, using the outbox event router SMT to map rows to Kafka records — but requires operating Kafka Connect, monitoring replication-slot WAL retention, and heartbeats to keep the slot advancing.
- End-to-end effectively-once requires **both sides**: outbox (at-least-once publish, no ghosts) + idempotent consumer with inbox/dedup table (business write + inbox insert atomically, offset commit after) — together they make duplicates invisible.
- Per-aggregate order is preserved by deterministic `aggregate_id` as Kafka key (one partition per aggregate), strictly ordered relay, and per-partition consumer concurrency — concurrent writes to the same aggregate must be serialized (advisory lock) to preserve causal order.
- Operate with lag/age/WAL/slot monitoring, poison-row handling (attempt counter → DLQ), payload versioning, and bounded outbox retention — and never treat the outbox as optional once the system relies on events for correctness.

Further reading

- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 11 — The dual-write problem and the log as the unifying abstraction.
- Debezium documentation 2.5 — Postgres connector, `pgoutput`, outbox event router, slot management. https://debezium.io/documentation/reference/2.5/connectors/postgresql.html and https://

debezium.io/documentation/reference/stable/transformations/outbox-event-router.html

- Postgres documentation 16 — Logical replication, `wal_level=logical`, `pg_replication_slots`, `pgoutput`. https://www.postgresql.org/docs/16/logical-replication.html
- Kafka documentation 3.7 — Producer `acks`, `enable.idempotence`, transactions, `isolation.level`. https://kafka.apache.org/37/documentation.html
- Garcia-Molina, H. & Salem, K. "Sagas." ACM SIGMOD, 1987 — The saga context for long-lived transactions that outbox workflows often participate in.
- Richardson, C. *Microservices Patterns* (Manning, 2018) — Ch. 3 — Transactional outbox pattern (original popularization).
- Confluent / Apache Kafka — Exactly-once semantics, KIP-98 (EOS), KIP-129 — Broker idempotence and transactions.

Chapter 7 — Backpressure and Flow Control

What this chapter covers. Every messaging system is a producer-consumer pipeline where the two sides run at independent rates. When producers outpace consumers — during a traffic spike, a slow downstream dependency, a GC pause, or a poison-message stall — messages accumulate somewhere. Without explicit flow control, that *somewhere* is an unbounded buffer that exhausts memory, spills to disk, and eventually collapses the broker, the consumer, or both. This is the **backpressure problem**: how to signal *upstream* that *downstream* is saturated, and how to react without losing data, violating ordering, or cascading failure. This chapter builds the full backpressure discipline from queueing theory to wire protocol. We cover Little's Law and queueing fundamentals, the taxonomy of backpressure strategies (buffer, drop, throttle, shed, block), push vs. pull and credit-based flow control, and the concrete mechanisms in **Kafka 3.7** (fetch flow control, `pause`/`resume`, quotas), **RabbitMQ 3.13** (prefetch, publisher confirms, global flow control), **gRPC/HTTP/2** (window-based flow control), and reactive frameworks (**Reactive Streams**, **Project Reactor**, **Akka Streams**,

Node.js streams). Every pattern is shown with runnable, version-pinned configuration and code, and viewed through the distributed-systems lens where backpressure propagated across five hops determines whether overload is isolated or becomes a fleet-wide outage.

Learning goals — after this chapter you should be able to:

- State Little's Law and the stability condition (arrival rate vs. service rate) and explain why an unbounded buffer is not a flow-control strategy.
- Enumerate the five backpressure strategies — buffer (bounded), drop, throttle, shed, block — and choose correctly by durability, ordering, and loss-tolerance requirements.
- Contrast push vs. pull and credit-based flow control, and explain how each maps to Kafka, RabbitMQ, gRPC/HTTP/2, and Reactive Streams.
- Configure Kafka consumer flow control: `max.poll.records`, `max.poll.interval.ms`, `fetch.max.bytes`, `pause()` / `resume()`, and broker quotas (`client.quota.callback`, `quota.producer.default`).
- Configure RabbitMQ flow control: `prefetch` (`basic.qos`), publisher confirms, credit-flow, memory/disk alarms, and stream `credit` protocol.
- Implement application-level backpressure in Go (`context` + buffered channels + `semaphore`), Java (Reactor `onBackpressure*`, `OverflowStrategy`), and Node.js (`Readable` / `Writable` highWaterMark, `pipe`).
- Reason about end-to-end backpressure propagation across gateway → service → broker → consumer → database, and detect saturation with queue depth, consumer lag, and `pause` metrics.

> **Boundary note.** This volume's Chapter 1 — Messaging Fundamentals — introduces queues vs. logs vs. pub/sub and where buffers live. Chapter 2 — Delivery Semantics — covers at-least/at-most/effectively-once and how retries interact with backpressure. Volume 5, Chapter 7 (WAL/Recovery) and Volume 3, Chapter 7 (HTTP/2 flow control) provide the storage and transport primitives. Volume 7, Chapter 9 — Rate Limiting, Quotas, and Fairness — covers *admission control at the edge*; this chapter covers *flow control inside the pipeline* after admission. Volume 11, Chapter 2 — Metrics and the Golden Signals — defines the saturation signals (queue depth, lag, p99 processing latency) we alert on here. For the outbox relay's own backpressure (CDC lag vs. polling interval), see Chapter 6.

Why backpressure is not optional

The stability condition

A messaging pipeline is a queueing system. Let:

- λ = mean arrival rate (messages/s) into the buffer
- μ = mean service rate (messages/s) of the consumer
- L = mean number in system, W = mean time in system

Little's Law: `L = λ · W` holds for any stable queueing system — no distribution assumptions. The stability condition is `λ < μ` (arrival slower than service). When `λ > μ` for any sustained interval, `L` grows without bound and `W` (latency) grows linearly with it. A buffer does not fix instability — it only delays the failure and hides the signal.

```
Stable:    λ = 800/s, μ = 1000/s  →  L bounded, W ≈ 1/μ × queue depth
Unstable:  λ = 1200/s, μ = 1000/s  →  L grows at 200/s, W grows without
bound
                                            memory exhausted in L /
capacity seconds
```

In production the rates are not constant. A downstream database slows from 2 ms to 80 ms p99 during a compaction, μ drops by 40×, and a pipeline that was stable at 60% utilization is suddenly unstable at 2400% overload. Without backpressure, the only observable before OOM is *increasing queue depth* — which every system already has but few alert on.

Where messages accumulate

```
flowchart LR
    P[Producers
λ variable] --> B1[Broker buffer
Kafka log / RabbitMQ queue]
    B1 --> C[Consumer process
prefetch / fetch buffer]
    C --> B2[App buffer
channel / queue / reactor buffer]
    B2 --> D[Downstream
DB / HTTP / external API
μ variable]

    style B1 fill:#fff3e0
    style B2 fill:#fff3e0
    style C fill:#e3f2fd
    style D fill:#e8f5e9
```

Buffers exist at four layers: broker (log segments, queue pages), consumer client (fetch/prefetch buffer), application (in-memory channels, Reactor buffers), and downstream (DB connection pool queue, HTTP client queue). Backpressure must be *propagated* across all four — a consumer that applies backpressure to the broker but buffers unboundedly in application memory has only moved the OOM one hop downstream.

The five backpressure strategies

Strategy	Behaviour when saturated	Data loss	Latency effect	Ordering	When to use
Buffer (bounded)	Enqueue up to N; then apply next strategy	None (until next strategy)	Increases with depth	Preserved	Default — but N must be bounded and the overflow strategy explicit

Strategy	Behaviour when saturated	Data loss	Latency effect	Ordering	When to use
Drop (newest/ oldest)	Discard incoming or oldest message	Yes — intentional	Bounded	Preserved for survivors	Telemetry, metrics, best-eńort notifications where loss is cheaper than stall
Throttle	Slow the producer — `sleep`, rate-limit, `pause`	None	Increases upstream	Preserved	Durable pipelines where producers can be slowed (internal services, CDC)
Shed (load shedding)	Reject/NAK with `Busy` / `429` / `Unavailable`	No — caller retries	Caller sees error quickly	Preserved	Edge and synchronous hops — fail fast, let caller back off
Block	Block the producer thread/ coroutine	None	Producer stalls	Preserved	Only inside a single process with bounded concurrency — never across network without timeout

No pipeline uses one strategy everywhere. The correct design is layered: throttle the broker fetch when downstream is slow, shed at the gateway when the service is saturated, and drop only for explicitly loss-tolerant

streams. An *unbounded* buffer is not a sixth strategy — it is the absence of a strategy.

```
flowchart TB
    Saturated{Downstream
saturated?}
    Saturated -->|No| Pass[Forward message
L and W bounded]
    Saturated -->|Yes| Choice{Strategy}
    Choice -->|Durable
producer can wait| Throttle[Throttle / Block
pause fetch, slow producer]
    Choice -->|Caller can retry| Shed[Load shed
429 / NAK / Busy]
    Choice -->|Loss-tolerant
telemetry| Drop[Drop
newest or oldest]
    Choice -->|None set
unbounded buffer| OOM([OOM / disk full
uncontrolled failure])

    style Throttle fill:#e8f5e9
    style Shed fill:#fff3e0
    style Drop fill:#f3e5f5
    style OOM fill:#ffcdd2
```

Choosing by durability and loss tolerance

- **Orders, payments, inventory events (durable, loss-intolerant)**: throttle + shed at the edge. Never drop. Buffer bounded (e.g., 10k in-memory + broker log), then throttle (pause Kafka partition), then shed (return `429` / `503` to caller who retries with backoff).
- **Metrics, traces, clickstream (loss-tolerant, high-volume)**: bounded buffer + drop-oldest. A 1% drop during a spike is cheaper than stalling the pipeline that carries payments on the same broker.
- **Synchronous RPC inside the mesh**: shed. Blocking the caller thread across a network hop holds a connection and a goroutine/thread. Return `UNAVAILABLE` / `429` with `Retry-After` and let the caller back off or hedge (Vol 3, Ch 11).

Push vs. pull vs. credit-based flow control

Model	Who controls rate	How it signals	Example
Push	Producer pushes; consumer must accept or drop	Consumer slow → buffer grows → OOM	Classic RabbitMQ `basic.deliver` without prefetch, naive TCP without windowing
Pull	Consumer pulls when ready	Consumer controls `poll()` / `fetch` cadence	Kafka `poll()`, SQS `ReceiveMessage` long-poll, gRPC server-streaming with `request(n)`
Credit-based	Consumer grants credits; producer sends only within credits	`credit = window - in-flight`; producer stalls at 0	RabbitMQ Streams, AMQP 1.0 `flow`, HTTP/2 `WINDOW_UPDATE`, Reactive Streams `request(n)`

Push without a credit or prefetch limit is the most common source of consumer OOM. Every production consumer must use one of pull or credit-based consumption.

```
sequenceDiagram
    participant P as Producer / Broker
    participant C as Consumer

    Note over P,C: Push without flow control — failure mode
    P->>C: message 1
    P->>C: message 2
    P->>C: message 3 ... burst of 10k
    Note over C: buffer grows unbounded
GC pressure → pause → more buffering → OOM

    Note over P,C: Pull — consumer controls rate
    C->>P: poll(maxRecords=500)
    P-->>C: 500 messages
    Note over C: process 500, commit offsets
    C->>P: poll(maxRecords=500)
    P-->>C: 500 messages

    Note over P,C: Credit-based — window of 1000
    C->>P: credit(1000)
    P->>C: messages 1..600 (in-flight 600)
    P->>C: messages 601..1000 (in-flight 1000 — window exhausted)
    Note over P: stalls — no credit
    C->>P: credit(500) — consumed 500, grant more
    P->>C: messages 1001..1500
```

Kafka flow control

Kafka is a **pull** system: consumers fetch from the log at their own pace. That is the primary backpressure mechanism — a slow consumer simply calls `poll()` less often, and the log retains messages on disk. But three tuning points determine whether that pull is well-behaved.

Consumer-side fetch control

```
// Java — Kafka 3.7 consumer with explicit flow-control tuning
Properties props = new Properties();
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka.prod:9092");
props.put(ConsumerConfig.GROUP_ID_CONFIG, "payments-processor");
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeseriali
zer.class);
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeseriali
zer.class);

// Flow-control knobs — the important ones
props.put(ConsumerConfig.MAX_POLL_RECORDS_CONFIG,
500);          // max records per poll()
props.put(ConsumerConfig.MAX_POLL_INTERVAL_MS_CONFIG,
300_000);  // max time between poll() before rebalance
props.put(ConsumerConfig.FETCH_MAX_BYTES_CONFIG, 52_428_800);     //
max bytes per fetch (50 MiB)
props.put(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, 500);          //
broker waits up to 500ms to fill fetch
props.put(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, 1);              //
return as soon as any data available
props.put(ConsumerConfig.MAX_PARTITION_FETCH_BYTES_CONFIG,
1_048_576); // 1 MiB per partition
props.put(ConsumerConfig.SESSION_TIMEOUT_MS_CONFIG, 45_000);
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);       //
manual commit — essential for backpressure

KafkaConsumer<String, PaymentEvent> consumer = new KafkaConsumer<>(prop
s);
consumer.subscribe(List.of("payments"));

while (true) {
    ConsumerRecords<String, PaymentEvent> records = consumer.poll(Durat
ion.ofMillis(1000));
    for (ConsumerRecord<String, PaymentEvent> rec : records) {
        // Downstream may be slow — check before processing each batch
        if (downstreamSaturated()) {
            // Pause only the saturated partitions — other partitions
continue
            consumer.pause(consumer.assignment());
            // Back off, then resume — cooperative, no rebalance
            Thread.sleep(1000);
            consumer.resume(consumer.paused());
            break;
        }
        process(rec);
    }
    consumer.commitSync(); // commit only after successful processing
}
```

```
// Pause/resume by partition — finer-grained backpressure
// Pause hot partitions (tenant with burst) while continuing to drain
others
Set<TopicPartition> hotPartitions = detectHotPartitions(records);
if (!hotPartitions.isEmpty()) {
    consumer.pause(hotPartitions);
    scheduleResume(hotPartitions, Duration.ofSeconds(5));
}
```

Key invariants:

- **`max.poll.records` + `fetch.max.bytes` bound the per-`poll()` batch.** Without them a single `poll()` can return tens of thousands of large messages and OOM the consumer heap.
- **`max.poll.interval.ms` is the liveness contract.** If processing a batch takes longer than this (because downstream is slow and backpressure is not applied), the group coordinator considers the consumer dead and triggers a rebalance — which reassigns partitions and causes duplicate processing. Either keep batch processing under the interval or call `poll()` heartbeats on a separate thread.
- **`pause()` / `resume()` is cooperative.** It stops fetching from the named partitions without leaving the group, so other consumers do not take over. Use it when downstream (DB, HTTP) is saturated per-partition.

Broker quotas — throttling producers and consumers

```
# server.properties — Kafka 3.7 quotas (KIP-848 client quota overrides)
# Throttle producers that exceed 10 MiB/s per client-id
quota.producer.default=10485760
# Throttle consumers that exceed 20 MiB/s per client-id
quota.consumer.default=20971520
# Per-client override via kafka-configs.sh
```

```
# Set a per-client-id quota: order-service may produce at most 5 MiB/s
kafka-configs.sh --bootstrap-server kafka.prod:9092 \
  --alter --add-config 'producer_byte_rate=5242880,consumer_byte_rate=1
0485760' \
  --entity-type clients --entity-name order-service

# Describe quotas
kafka-configs.sh --bootstrap-server kafka.prod:9092 \
  --describe --entity-type clients --entity-name order-service

# Quota throttling shows as throttle-time-ms in client metrics
# kafka.producer:type=producer-metrics,client-id=order-service ->
throttle-time-avg
```

When a client exceeds its quota, the broker **throttles** — it delays the response by `throttle-time-ms` proportional to the excess. This is true backpressure: the producer's `send()` blocks (or times out) and the client must handle `ThrottleTimeMs`. It prevents a single noisy producer from starving other tenants on the same cluster.

Monitoring consumer backpressure

```
# Consumer lag — the primary saturation signal (per partition)
kafka_consumer_group_lag{group="payments-processor"} > 10000

# Time since last poll — detects stalled consumers before rebalance
time() - kafka_consumer_last_poll_seconds > 60

# Throttle time — quota-induced backpressure
rate(kafka_producer_throttle_time_ms_sum[5m]) > 100

# Pause duration — how long partitions stay paused
histogram_quantile(0.99, rate(consumer_partition_pause_seconds_bucket[5
m])) > 10
```

RabbitMQ flow control

RabbitMQ is a **push** system by default — the broker pushes `basic.deliver` to consumers. Without flow control, a fast queue and a slow consumer reproduce the classic push failure. RabbitMQ provides three layers.

Prefetch — consumer credit

```
// Java — RabbitMQ 3.13 consumer with prefetch (credit-based)
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("rabbitmq.prod");
Connection conn = factory.newConnection();
Channel ch = conn.createChannel();

// QoS: at most 100 unacked messages per consumer, shared across all
queues on this channel
ch.basicQos(100, false);
// Per-queue variant for heterogeneous consumers:
// ch.basicQos(50, false) on channel serving the slow queue, 500 on the
fast queue

DeliverCallback callback = (tag, delivery) -> {
    try {
        process(delivery);
        ch.basicAck(delivery.getEnvelope().getDeliveryTag(), false);
    } catch (DownstreamSaturatedException e) {
        // Reject and requeue — but see Ch 8 for why requeue is
dangerous without backoff
        ch.basicNack(delivery.getEnvelope().getDeliveryTag(), false, tr
ue);
        // Better: pause consumption on this channel until downstream
recovers
        ch.basicQos(0, false); // global prefetch 0 — stops delivery
        scheduleResume(ch, Duration.ofSeconds(5));
    } catch (Exception e) {
        ch.basicNack(delivery.getEnvelope().getDeliveryTag(), false, fa
lse); // to DLX — Ch 8
    }
};
ch.basicConsume("payments", false, callback, tag -> {});
```

```
# Spring AMQP — prefetch + concurrency as flow-control
spring:
  rabbitmq:
    listener:
      simple:
        prefetch: 100              # basic.qos per consumer
        concurrency: 4             # 4 consumers per listener container
        max-concurrency: 8         # scale to 8 under load
        default-requeue-rejected: false  # never silently requeue —
route to DLQ
        acknowledge-mode: manual   # ack only after downstream succeeds
```

Prefetch value	Behaviour	When
0 (unlimited)	Broker pushes as fast as it can	Never in production — unbounded consumer buffer
1	Strict fair dispatch, highest isolation, lowest throughput	Slow, order-sensitive consumers
20–100	Balanced — good throughput, bounded memory	Typical HTTP→DB consumers
500+	High throughput, higher memory, less fair dispatch	Fast consumers, large messages, batch processing

Publisher-side flow control

```
// Publisher confirms — backpressure on the producer side
ch.confirmSelect(); // enable publisher confirms (async ACK from
broker)

ch.addConfirmListener(
    (seqNo, multiple) -> { /* ack — message durable */ },
    (seqNo, multiple) -> { /* nack — broker rejected, retry with
backoff */ }
);

// Wait for confirms with timeout — blocks if broker is saturated
if (!ch.waitForConfirms(5000)) {
    throw new BrokerSaturatedException("publisher confirm timeout");
}

// Spring AMQP — correlated confirms
rabbitTemplate.setConfirmCallback((correlationData, ack, cause) -> {
    if (!ack) log.error("nack: {} cause={}", correlationData, cause);
});
rabbitTemplate.convertAndSend("orders", order, m -> {
    m.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSIS
TENT);
    return m;
}, new CorrelationData(order.id()));
```

```
# RabbitMQ memory and disk alarms — broker-level backpressure
# rabbitmq.conf — RabbitMQ 3.13
vm_memory_high_watermark.relative = 0.6   # block publishers at 60% RAM
vm_memory_high_watermark_paging_ratio = 0.5
disk_free_limit.relative = 1.5            # block at 1.5× queue index
size
credit_flow_default_credit = {400, 200}   # credit-flow window per
connection
```

When the broker hits the memory or disk watermark, it **blocks publishers** — `basic.publish` returns a `ResourceAlarm` and the client must handle it. This is the broker propagating backpressure upstream. Clients that ignore `BlockedListener` will see publish timeouts.

RabbitMQ Streams — explicit credit

```
// RabbitMQ Streams 3.13 — credit-based consumption (like Kafka pull)
Environment env = Environment.builder().uri("rabbitmq-stream://
rabbitmq.prod:5552").build();
Consumer consumer = env.consumerBuilder()
    .stream("payments-stream")
    .offset(OffsetSpecification.first())
    .flow()
        .initialCredits(1000)       // grant 1000 credits (messages)
        .creditBatchSize(500)       // replenish 500 at a time
    .build();
```

---

gRPC / HTTP/2 window-based flow control

HTTP/2 (and thus gRPC) has **connection-level and stream-level flow control** via `WINDOW_UPDATE` frames. Each side advertises a window (default 64 KiB connection, 32 KiB stream) and the sender must not send more than the window. The receiver grants more credit with `WINDOW_UPDATE`.

```
// Go 1.22 — gRPC server streaming with explicit flow control via
context + semaphore
// The gRPC HTTP/2 window is automatic; application-level backpressure
is the semaphore.
func (s *OrderService) StreamOrders(req *pb.StreamRequest, stream pb.Or
derService_StreamOrdersServer) error {
    // Limit in-flight downstream calls to 50 — application
backpressure
    sem := make(chan struct{}, 50)
    for {
        select {
        case <-stream.Context().Done():
            return stream.Context().Err()
        case sem <- struct{}{}:
        }
        order, err := s.nextOrder(stream.Context())
        if err != nil {
            <-sem
            return err
        }
        // Downstream call with timeout — shed if slow
        ctx, cancel := context.WithTimeout(stream.Context(), 2*time.Sec
ond)
        err = s.enrichOrder(ctx, order)
        cancel()
        if err != nil {
            <-sem
            if errors.Is(err, context.DeadlineExceeded) {
                return status.Error(codes.ResourceExhausted, "downstrea
m saturated")
            }
            return err
        }
        if err := stream.Send(order); err != nil {
            <-sem
            return err // client gone or window exhausted
        }
        <-sem
    }
}
```

```
# Envoy — HTTP/2 flow-control window tuning (per cluster)
clusters:
  - name: payments
    http2_protocol_options:
      initial_stream_window_size: 65535      # 64 KiB per stream
      initial_connection_window_size: 1048576 # 1 MiB per connection
    circuit_breakers:
      thresholds:
        - max_pending_requests: 1000
          max_requests: 500
          max_retries: 3
          retry_budget:
            percent: 20
            min_retry_concurrency: 5
```

Envoy circuit breakers are the **load-shedding** layer on top of HTTP/2 flow control: when `max_pending_requests` is exceeded, Envoy returns `503` immediately rather than queueing — that `503` is backpressure propagated to the caller.

Reactive Streams and application-level backpressure

Reactive Streams (`Publisher` / `Subscriber` / `Subscription` with `request(n)`) makes backpressure a first-class protocol signal: a subscriber *requests* `n` items and the publisher must not emit more than requested.

```
// Java 17 — Project Reactor 3.6 — backpressure strategies
Flux<OrderEvent> events = orderEventSource() // fast publisher
    .onBackpressureBuffer(10_000,               // bounded buffer — 10k
        dropped -> log.warn("dropped {}", dropped),
        BufferOverflowStrategy.DROP_LATEST)     // drop newest when
full
    // Alternatives by use case:
    // .onBackpressureDrop(dropped -> metrics.increment("dropped"))
    // .onBackpressureLatest()                  // keep only latest —
for state updates
    // .onBackpressureError()                   // fail fast — no
silent loss
    // .onBackpressureBuffer(1000, false)       // unbounded — never
use in prod
    .publishOn(Schedulers.boundedElastic(), 500) // prefetch 500,
bounded scheduler
    .flatMap(event -> enrichAsync(event)
        .subscribeOn(Schedulers.boundedElastic()), 8); // max 8
concurrent enrichments

events.subscribe(
    event -> persist(event),
    err  -> log.error("stream failed", err),
    ()   -> log.info("stream complete")
);
```

```
// Akka Streams 2.6 — explicit buffer + overflow strategy
Source<OrderEvent, NotUsed> source = orderEventSource();
source
    .buffer(1000, OverflowStrategy.backpressure()) // block upstream
when full — throttle
    // OverflowStrategy.dropHead() — drop oldest
    // OverflowStrategy.dropTail() — drop newest
    // OverflowStrategy.dropBuffer() — drop entire buffer
    // OverflowStrategy.fail() — fail the stream
    .mapAsync(8, event -> enrichAsync(event))      // max 8 in-flight
    .async()                                        // async boundary —
backpressure propagates
    .to(Sink.foreach(this::persist))
    .run(materializer);
```

```
// Node.js 20 — stream backpressure via highWaterMark and pipe
import { Readable, Writable, Transform } from 'node:stream';
import { pipeline } from 'node:stream/promises';

const orderSource = new Readable({
    objectMode: true,
    highWaterMark: 500, // buffer at most 500 objects before push()
returns false
    read() { /* push() orders from Kafka/RabbitMQ */ }
});

const enrich = new Transform({
    objectMode: true,
    highWaterMark: 200,
    async transform(order, _enc, cb) {
        try {
            const enriched = await enrichOrder(order); // may be slow
            cb(null, enriched);
        } catch (err) { cb(err); }
    }
});

const persist = new Writable({
    objectMode: true,
    highWaterMark: 100,
    async write(order, _enc, cb) {
        try { await db.insert(order); cb(); }
        catch (err) { cb(err); }
    }
});

// pipeline() propagates backpressure: persist slow -> enrich pauses ->
orderSource pauses
await pipeline(orderSource, enrich, persist);
// write() returning false / push() returning false is the backpressure
signal
```

```
// Go — channel + semaphore as manual backpressure between Kafka poll
and DB pool
func runConsumer(ctx context.Context, consumer *kafka.Consumer, db
*sql.DB) {
    sem := make(chan struct{}, 50) // at most 50 in-flight DB inserts
    for {
        select {
        case <-ctx.Done():
            return
        default:
        }
        msg, err := consumer.ReadMessage(1 * time.Second)
        if err != nil { continue; }
        sem <- struct{}
{} // blocks when 50 in-flight — backpressure to poll loop
        go func(m *kafka.Message) {
            defer func() { <-sem }()
            ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
            defer cancel()
            if err := persistOrder(ctx, db, m); err != nil {
                log.Printf("persist failed offset=%d err=%v", m.TopicPa
rtition.Offset, err)
                return // Ch 8: retry/DLQ decision
            }
            consumer.CommitMessage(m)
        }(msg)
    }
}
```

End-to-end propagation

Backpressure that stops at one hop is not backpressure — it is a buffer. The full path must propagate:

```
flowchart TB
    Client[Client / Producer] --> GW[API Gateway
shed: 429 + Retry-After]
    GW --> Svc[Service
bounded queue 1000
shed when full]
    Svc --> Broker[(Kafka / RabbitMQ
pause fetch / prefetch limit
quota throttle)]
    Broker --> Consumer[Consumer
pause partitions
semaphore 50]
    Consumer --> DB[(Postgres
pool queue 20
statement timeout 2s)]

    DB -. backpressure .-> Consumer
    Consumer -. pause .-> Broker
    Broker -. throttle / 503 .-> Svc
    Svc -. 503 / 429 .-> GW
    GW -. 429 .-> Client

    style GW fill:#fff3e0
    style Broker fill:#e3f2fd
    style Consumer fill:#e8f5e9
    style DB fill:#f3e5f5
```

The signals and thresholds at each hop:

Hop	Signal	Threshold	Action
DB pool	`pool.wait_queue_depth`, `statement_timeout`	queue > 10, p99 > 500 ms	Consumer stops committing, pauses partitions
Consumer	`consumer_lag`, `in_flight`, `pause_duration`	lag > 10k, in-flight = cap	`pause()` partitions, stop `poll()`
Broker	`quota throttle-time`, `disk watermark`, `memory alarm`	throttle > 100 ms, watermark 60%	Delay produce/fetch, block publishers

Hop	Signal	Threshold	Action
Service	`queue_depth`, `latency p99`, `circuit open`	queue > 80%, p99 > SLO	Return `503` / `429`, open circuit
Gateway	`pending_requests`, `429 rate`	pending > 1000, 429 > 5%	Return `429` to client with `Retry-After`

Without propagation, a slow database causes the consumer buffer to grow, the broker log to retain longer, the service heap to grow, and the gateway to queue — until the first OOM or disk-full kills a process that was otherwise healthy. With propagation, the database slowness surfaces as `429` at the edge within seconds, the caller backs off, and the system degrades gracefully.

Distributed-systems lens

- **Backpressure is a correctness property, not a performance optimization.** An unbounded buffer that OOMs the consumer violates durability (unacked messages lost on crash) and ordering (rebalance reassigns partitions). Bounded buffers with explicit overflow strategies make the failure mode *chosen* rather than *accidental*.
- **Pull and credit-based protocols are the only safe defaults across a network.** Push without a window (RabbitMQ without prefetch, HTTP without `WINDOW_UPDATE`) delegates flow control to the receiver's memory — which is not flow control. Default to `prefetch` / `request(n)` / `WINDOW_UPDATE` and treat unlimited push as a bug.
- **Pause is cooperative, rebalance is not.** Kafka `pause()` keeps partition ownership and resumes without rebalancing; exceeding `max.poll.interval.ms` triggers a rebalance that reassigns partitions and causes duplicate processing. Size `max.poll.interval.ms` above the worst-case batch processing time under backpressure, or heartbeat on a separate thread.
- **Quotas are multi-tenancy isolation.** A single noisy producer or consumer can starve every other tenant on the same Kafka or

RabbitMQ cluster. Per-client quotas turn a noisy-neighbour incident into local throttling rather than a cluster-wide outage.

- **The edge must shed.** Internal backpressure (pause, throttle) protects durability but increases latency for every caller. At the edge (gateway, BFF), shedding with `429` / `503` + `Retry-After` bounds tail latency and gives callers a signal they can act on (backoff, hedge, degrade). A system that only throttles internally and never sheds at the edge will meet its durability SLO and miss its latency SLO on every overload.

Key takeaways

- Little's Law (`L = λ·W`) and the stability condition (`λ < μ`) govern every pipeline: a sustained `λ > μ` makes queue depth and latency grow without bound — a buffer only delays the failure.
- The five strategies are buffer (bounded), drop, throttle, shed, and block — layered by hop: throttle/shed for durable pipelines, drop only for loss-tolerant telemetry, never unbounded.
- Push without a credit/window is unsafe across a network; pull (`poll` / `fetch`) and credit-based (`request(n)` , `WINDOW_UPDATE` , `basic.qos`) are the safe primitives.
- Kafka flow control is pull + `max.poll.records` / `fetch.max.bytes` + `pause()` / `resume()` + broker quotas; RabbitMQ is `basic.qos` prefetch + publisher confirms + memory/disk alarms + stream credits; gRPC/HTTP/2 is `WINDOW_UPDATE` + Envoy circuit breakers.
- Reactive Streams (`request(n)`), Akka `OverflowStrategy` , and Node.js `highWaterMark` / `pipe` make backpressure explicit in application code — bounded buffers with named overflow strategies, never unbounded.
- End-to-end propagation (DB pool → consumer pause → broker throttle → service `503` → gateway `429`) turns a slow downstream into a fast, actionable signal at the edge instead of a cascading OOM.

Further reading

- **Queueing theory:** Kleinrock *Queueing Systems, Volume 1* (1975) — Little's Law and stability. Harchol-Balter *Performance Modeling and*

Design of Computer Systems (2013) — Ch 3 (Little's Law), Ch 11 (scheduling under load).

- **Reactive Streams:** Reactive Streams spec 1.0.4 (https://www.reactive-streams.org/) — `Publisher` / `Subscriber` / `Subscription` / `request(n)` contract. Project Reactor 3.6 reference (https://projectreactor.io/docs/core/release/reference/) — `onBackpressure*` operators. Akka Streams 2.6 docs (https://doc.akka.io/docs/akka/current/stream/index.html) — `OverflowStrategy`, `async` boundaries.
- **Kafka:** Kafka 3.7 docs — Consumer configs (`max.poll.records`, `max.poll.interval.ms`, `fetch.*`), quotas (`quota.producer.default`, `client.quota.callback`), `pause()` / `resume()` (https://kafka.apache.org/documentation/#consumerconfigs). KIP-848 (client quotas).
- **RabbitMQ:** RabbitMQ 3.13 docs — Consumer prefetch (`basic.qos`), publisher confirms, flow control / credit flow, memory/disk alarms (https://www.rabbitmq.com/docs/flow-control), Streams credit protocol (https://www.rabbitmq.com/docs/streams).
- **gRPC / HTTP/2:** RFC 9113 (HTTP/2) §5.2 — flow control (`WINDOW_UPDATE`). gRPC flow-control docs (https://grpc.io/docs/guides/flow-control/). Envoy circuit breakers (https://www.envoyproxy.io/docs/envoy/latest/configuration/upstream/circuit_breaking).
- **Node.js / Go:** Node.js 20 stream docs (https://nodejs.org/api/stream.html) — `highWaterMark`, `pipeline()`, backpressure. Go `context` + `golang.org/x/sync/semaphore` 0.17+ — bounded concurrency.

Chapter 8 — Dead Letters, Retries, Poison Messages

What this chapter covers. In a healthy pipeline every message is consumed, processed, and acknowledged. In production a fraction inevitably is not — the payload is malformed, the schema changed, a downstream dependency is down, or the handler has a bug that throws on one specific input. Without an explicit failure discipline, the system has two bad defaults: retry forever and block the partition, or acknowledge and silently drop the message. The correct discipline is **

retries with backoff and a dead-letter queue (DLQ) **— a durable, observable, replayable sink for messages that cannot be processed after a bounded number of attempts. This chapter builds that discipline end to end: why poison messages stall ordered logs, the retry taxonomy (immediate, delayed, exponential backoff with jitter, non-retryable classification), DLQ design (what to store, how to route, how to operate and replay), poison-message handling (detect, quarantine, skip, fix, replay), and the concrete wiring in** Kafka 3.7 **(Spring Kafka `DeadLetterPublishingRecoverer`, Kafka Streams `ProductionExceptionHandler`, `errors.tolerance` in Kafka Connect) and** RabbitMQ 3.13** (dead-letter exchanges, `x-dead-letter-exchange`, quorum-queue DLX, delayed retry via TTL + DLX or delayed-message plugin/exchange and quorum-queue delayed semantics). Every pattern is shown with runnable, version-pinned configuration and code and viewed through the distributed-systems lens where a single poison message can stall a partition, inflate lag, and invalidate exactly-once-effect guarantees if the failure path is not idempotent and observable.

Learning goals — after this chapter you should be able to:

- Define *poison message*, *dead letter*, and *retry* precisely and explain why `ack-on-failure` (silent drop) and `retry forever` (partition stall) are both unacceptable defaults.
- Classify failures as retryable (transient downstream 503, DB deadlock, timeout) vs. non-retryable (poison — deserialization, schema violation, invariant bug) and route correctly on the first failure.
- Design a retry policy: immediate vs. delayed, fixed vs. exponential backoff with jitter, max attempts, and the per-partition ordering cost of blocking retries.
- Design a DLQ: what to store (original payload + headers + failure metadata + stack trace), how to route (DLX, `DeadLetterPublishingRecoverer`, dead-letter topic naming), and the operational loop (alert, inspect, fix, replay, verify).
- Handle poison messages without stalling the log: detect (deserialization exception, repeated NACK), quarantine (DLQ), skip (commit offset past poison), and replay after a fix — with ordering implications stated.

- Configure Kafka and RabbitMQ DLQ/retry wiring: Spring Kafka `DefaultErrorHandler` + `DeadLetterPublishingRecoverer`, non-blocking retry topics (Spring Kafka `RetryTopicConfiguration`), Kafka Connect `errors.*`, RabbitMQ `x-dead-letter-exchange` + `x-message-ttl` / `x-delayed-message` / quorum-queue considerations.
- Operate DLQs: monitor DLQ depth and retry exhaustion, replay safely (idempotent consumers — see Ch 2 and Vol 6, Ch 9), and prevent DLQ backpressure from re-stalling the main pipeline.

> **Boundary note.** Chapter 2 — Delivery Semantics — defines at-least/at-most/effectively-once and why retries are at-least-once, requiring idempotent consumers (Vol 6, Ch 9). Chapter 6 — The Outbox Pattern — makes the *producer* side idempotent via the outbox; this chapter owns the *consumer* failure path. Chapter 7 — Backpressure and Flow Control — shows how retries amplify load and why retry storms must be throttled; retries without backoff are an anti-pattern. Vol 5, Ch 10 (Sagas) covers compensating transactions that may consume DLQ events. For the formal idempotency-key and exactly-once-effect protocol, see Vol 6, Ch 9; for broker delivery mechanics, see Ch 2.

The failure path nobody drew

The happy-path diagram in every messaging intro shows `produce → broker → consume → ack`. The production diagram has a second path:

```
flowchart TB
    P[Producer] --> B[(Broker
topic / queue)]
    B --> C[Consumer
handler]
    C -->|success| Ack[Ack / commit offset
message done]
    C -->|transient failure
503, timeout, deadlock| R[Retry
backoff + jitter]
    C -->|poison
bad payload, bug, schema| D{Classify}
    R -->|retryable
attempts left| C
    R -->|exhausted| DLQ[(Dead-letter
queue / topic)]
    D -->|retryable| R
    D -->|non-retryable| DLQ
    DLQ --> Op[Operator
inspect, fix, replay]
    Op -->|replay| B

    style R fill:#fff3e0
    style DLQ fill:#ffcdd2
    style Op fill:#e3f2fd
    style Ack fill:#e8f5e9
```

Without explicit wiring for `R` and `DLQ`, the consumer has only two options — both wrong:

Default	What happens	Consequence
Ack on failure (swallow exception, commit offset)	Message acknowledged as if processed	**Silent data loss** — payment never charged, inventory never updated, no signal to alert on
Retry forever (requeue / redeliver immediately)	Same message redelivered in a tight loop	**Partition stall** — Kafka consumer never advances past the poison offset; RabbitMQ queue drains no other messages; CPU/log I/O burned on a message that will never succeed

The DLQ is the third option that makes both failure modes observable and recoverable: after a bounded number of retries, the message is moved to a durable side queue with full failure context, the main

partition/queue advances, and an operator (or automation) can fix the cause and replay.

Poison messages — definition and taxonomy

A **poison message** is any message that will fail deterministically on every retry — the failure is in the message or the handler, not in a transient dependency.

Category	Example	Fails where	Retry helps?
Deserialization poison	`JSON` field `amount` is a string `"12.00"` but handler expects `int64` cents	Deserializer before handler	Never — same bytes every time
Schema poison	Producer upgraded to Avro schema v3, consumer still on v2, new required field `currency` missing	Schema Registry / deserializer	Only after consumer upgrades
Invariant poison	`order.total_cents = -400` violates `CHECK (total_cents > 0)`	Handler validation / DB constraint	Never — message is logically invalid
Handler bug	`nil` dereference on `order.coupon.code` when `coupon` is `null`	Handler code	Only after handler fix + deploy
Oversized poison	Message 12 MiB exceeds `fetch.max.bytes` or DB `max_allowed_packet`	Broker fetch / DB driver	Never without config change

Contrast with **retryable (transient) failures**, where the same message will likely succeed after a delay:

Transient	Example	Retry helps?
Downstream 503 / timeout	`POST /payments/charge` returns `503 Service Unavailable`	Yes — with backoff

Transient	Example	Retry helps?
DB deadlock / connection pool exhaustion	`SQLSTATE 40P01 deadlock detected`	Yes — immediate or short backoff
Broker not reachable	`NotEnoughReplicas`, `LeaderNotAvailable`	Yes — broker recovers
Rate-limited downstream	`429 Too Many Requests` with `Retry-After: 2`	Yes — respect `Retry-After`

Classification must happen **before** the first retry. Retrying a poison message with exponential backoff still stalls the partition for `sum(backoffs)` and still fails — it just fails slowly.

```
// Classification at the handler boundary — the single place that
decides retry vs. DLQ
public enum FailureKind { RETRYABLE, NON_RETRYABLE }

public FailureKind classify(Throwable t) {
    if (t instanceof SerializationException
        || t instanceof SchemaViolationException
        || t instanceof ConstraintViolationException
        || t instanceof IllegalArgumentException) {
        return FailureKind.NON_RETRYABLE; // poison — never retry,
straight to DLQ
    }
    if (t instanceof DataAccessException de) {
        // Postgres deadlock — retryable; constraint violation — poison
        if (de.getMessage().contains("40P01")) return FailureKind.RETRY
ABLE;
        if (de.getMessage().contains("23514")) return FailureKind.NON_R
ETRYABLE; // CHECK violation
    }
    if (t instanceof HttpStatusException he) {
        if (he.status() == 503 || he.status() == 429 || he.status() ==
504) return FailureKind.RETRYABLE;
        if (he.status() >= 400 && he.status() < 500) return
FailureKind.NON_RETRYABLE; // client error — bug or bad data
    }
    if (t instanceof TimeoutException || t instanceof
ConnectException) return FailureKind.RETRYABLE;
    return FailureKind.NON_RETRYABLE; // default — treat unknown as
poison to avoid retry storms
}
```

Why poison stalls ordered logs

In Kafka, offsets are sequential per partition. A consumer that cannot process offset 104 cannot `commit` 104, and the committed offset stays at 103 — even though offsets 105–10000 may be processable. Lag (`end - committed`) grows monotonically and *no other consumer can help* because the partition is owned by one consumer at a time. A single poison message can make lag look like the consumer is down when it is actually livelocked on one offset.

```
sequenceDiagram
    participant Log as Partition log
offsets 103..110
    participant C as Consumer
    participant DLQ as DLQ topic

    Note over Log: committed = 103
    Log->>C: poll — offsets 104..108
    Note over C: offset 104 — poison
deserialization throws
    C->>C: retry 1..3 with backoff — all fail
    C->>DLQ: publish to DLQ topic
with failure headers
    C->>Log: commit 104 — skip poison
    Log->>C: poll — offsets 105..110
    Note over C: offsets 105..110 process normally
ordering of 104 lost — DLQ is the record

    Note over Log: Without DLQ: consumer loops on 104 forever
lag grows, no progress, rebalance on
max.poll.interval.ms exceeded
```

RabbitMQ has the same shape with different mechanics: a poison message that is `basic.nack(requeue=true)` is redelivered immediately to the same consumer, which fails again — a tight loop that starves the queue. `requeue=false` with a DLX moves it aside; without a DLX it is dropped.

Retry design

The retry taxonomy

Strategy	Delay	Ordering cost	When
Immediate retry (blocking)	None — retry in same `poll()` loop	Blocks partition until success or exhaustion	Only for very fast transient (DB deadlock) with very few retries (1–2)
Delayed retry (non-blocking)	Fixed or exponential backoff, message parked elsewhere	Partition advances — retry happens on a retry topic/ queue	Default for HTTP 503/ timeouts — does not stall the main partition
Scheduled retry topic	Message forwarded to `topic.retry` with TTL/delay	Main partition advances immediately	Kafka non-blocking retries (Spring `RetryTopic`), RabbitMQ TTL+DLX or delayed exchange
No retry	—	—	Poison — straight to DLQ

Backoff with jitter

Exponential backoff without jitter creates **retry storms**: 100 consumers failing on the same downstream outage all retry at `1s, 2s, 4s, 8s` in lockstep, thundering-herding the downstream the moment it recovers.

```
delay(n) = min(base * 2^n + jitter, cap)
jitter   = random(0, base * 2^n * jitterFactor)   # full jitter or
decorrelated jitter
cap      = 30s–60s typical; prevents 10-minute stalls on long retry
chains
```

```
// BackOff with jitter — used by Spring Kafka DefaultErrorHandler and
manual retry loops
long backoffWithJitter(int attempt, long baseMs, long capMs, double jit
terFactor) {
    long exp = baseMs * (1L << Math.min(attempt, 20)); // cap exponent
to avoid overflow
    long jitter = (long) (ThreadLocalRandom.current().nextDouble() * ex
p * jitterFactor);
    return Math.min(exp + jitter, capMs);
}
// attempt 0: ~1000ms ± 300ms
// attempt 1: ~2000ms ± 600ms
// attempt 2: ~4000ms ± 1200ms
// attempt 3: ~8000ms ± 2400ms — cap at 30000ms thereafter
```

Ordering and blocking vs. non-blocking retries

- **Blocking retry** (retry inside the `poll` loop before committing) preserves per-partition ordering but *stalls* the partition for `sum(backoffs)` per failing message. Acceptable only when the failure is rare and fast.
- **Non-blocking retry** (publish to a retry topic/queue, commit the original offset, consume the retry topic separately) keeps the main partition moving but *breaks ordering* between the retried message and later messages on the same key. For many workloads (payments, inventory) this is acceptable because the retried message is delayed anyway; for strictly ordered event sourcing (Ch 5), blocking may be required and poison must go to DLQ rather than retry.

Dead-letter queue design

What to store

A DLQ message must be *replayable* and *debuggable* without access to the producer's logs:

Field	Source	Purpose
Original payload (bytes, not deserialized)	Raw `ConsumerRecord.value()` / `delivery.body`	Replay after fix — deserialization poison would be lost if only the exception is stored

Field	Source	Purpose
Original headers + key + topic/ queue + partition + offset	`ConsumerRecord` / `Envelope`	Routing and dedup on replay
Failure metadata	Exception class, message, stack trace (trimmed), handler name	Root-cause without reproducing
Attempt count + backoff history	Retry handler	Distinguish poison (attempt 1) from exhausted retries (attempt 5)
Timestamp + consumer identity	`Instant.now()`, `group.id` / `consumerTag`	Correlation with deploys and downstream incidents

Naming and topology

```
Main:   orders                  →  retry: orders.retry           →
DLQ: orders.dlt
        payments                →         payments.retry
→       payments.dlt
        inventory.events        →         inventory.events.retry
→       inventory.events.dlt

RabbitMQ:
  orders (quorum)  --DLX-->  orders.dlq (quorum)   with x-dead-letter-e
xchange
  orders.retry (quorum, TTL 30s) --DLX--> orders   (delayed retry loop)
  or: orders --x-delayed-message exchange--> orders.retry (delayed plug
in)
```

DLQ topics/queues should be **durable, quorum-replicated, and separately monitored** — a DLQ that pages to disk and is never alerted on is just a slower silent drop.

Kafka — DLQ and retries (Spring Kafka 3.1 / Kafka 3.7)

Blocking retry + DLQ via DefaultErrorHandler

```
// Spring Boot 3.2 + Spring Kafka 3.1 — blocking retries with
exponential backoff + DLQ
@Configuration
class KafkaErrorHandling {

    @Bean
    DefaultErrorHandler errorHandler(KafkaTemplate<String, byte[]> dlqT
emplate) {
        // Backoff: 1s, 2s, 4s, 8s — 4 attempts, then DLQ
        ExponentialBackOffWithMaxRetries backOff = new ExponentialBackO
ffWithMaxRetries(4);
        backOff.setInitialInterval(1_000L);
        backOff.setMultiplier(2.0);
        backOff.setMaxInterval(30_000L);

        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublish
ingRecoverer(
            dlqTemplate,
            // DLQ topic = original topic + ".dlt"
            (record, ex) -> new TopicPartition(record.topic() +
".dlt", record.partition())
        );
        // Enrich DLQ headers with failure context
        recoverer.setHeadersFunction((record, ex) -> {
            Headers h = new RecordHeaders();
            h.add("x-original-topic", record.topic().getBytes(StandardC
harsets.UTF_8));
            h.add("x-original-partition", String.valueOf(record.partiti
on()).getBytes(StandardCharsets.UTF_8));
            h.add("x-original-offset",
String.valueOf(record.offset()).getBytes(StandardCharsets.UTF_8));
            h.add("x-exception-class",
ex.getClass().getName().getBytes(StandardCharsets.UTF_8));
            h.add("x-exception-message", String.valueOf(ex.getMessage()
).getBytes(StandardCharsets.UTF_8));
            h.add("x-stacktrace", trimStacktrace(ex, 4000).getBytes(Sta
ndardCharsets.UTF_8));
            return h;
        });

        DefaultErrorHandler handler = new
DefaultErrorHandler(recoverer, backOff);

        // Poison — never retry, straight to DLQ
        handler.addNotRetryableExceptions(SerializationException.class)
;
        handler.addNotRetryableExceptions(SchemaViolationException.clas
s);

handler.addNotRetryableExceptions(ConstraintViolationException.class);
```

```
        // Transient — retryable (default)
        handler.addRetryableExceptions(TimeoutException.class);
        handler.addRetryableExceptions(HttpStatusException.class); //
filtered by classify() if needed

        // Optional: custom classifier for finer control
        handler.setClassifyFunction((record, ex) ->
            ex instanceof HttpStatusException he && he.status() >= 400
&& he.status() < 500
                ? false // 4xx — poison
                : !(ex instanceof
SerializationException) // everything else retryable except
serialization
        );
        handler.setCommitRecovered(true); // commit offset after DLQ
publish — advances partition
        return handler;
    }

    @Bean
    CommonErrorHandler
deserializationErrorHandler(KafkaTemplate<String, byte[]> dlqTemplate)
{
        // Deserialization failures happen before the listener — handle
at container level
        DeadLetterPublishingRecoverer recoverer = new DeadLetterPublish
ingRecoverer(dlqTemplate,
            (record, ex) -> new TopicPartition(record.topic() +
".dlt", -1));
        return new DefaultErrorHandler(recoverer, new FixedBackOff(0L,
0L)); // no retry for deser poison
    }
}
```

```
// Listener — no try/catch needed; the error handler owns the retry/DLQ
decision
@KafkaListener(topics = "orders", groupId = "order-processor")
void handle(ConsumerRecord<String, OrderEvent> record) {
    // Throwing here enters the error handler above
    orderService.process(record.value());
}
```

Non-blocking retry topics (does not stall the partition)

```
// Spring Kafka 3.1 — non-blocking retries via retry topics (partition
keeps moving)
@Configuration
@EnableKafka
class NonBlockingRetryConfig {

    @Bean
    RetryTopicConfiguration retryTopic(KafkaTemplate<String,
OrderEvent> template) {
        return RetryTopicConfigurationBuilder
            .newInstance()
            // Backoff for retry topics: 1s -> 2s -> 4s, each on its
own topic
            .exponentialBackoff(1_000, 2.0, 30_000)
            .maxAttempts(4)
            .includeTopics(List.of("orders"))
            // DLQ after exhaustion
            .dltHandlerMethod("dlqHandler", "handleDlt")
            // Retry topics are auto-created: orders-retry-1000,
orders-retry-2000, orders-retry-4000, orders-dlt
            .create(template);
    }

    @Component
    static class DlqHandler {
        void handleDlt(ConsumerRecord<String, OrderEvent> record) {
            log.error("DLQ orders-dlt offset={} key={} headers={}
ex={}",
                record.offset(), record.key(), record.headers(),
record.value());
            // Alert, persist to incident table, or forward to
PagerDuty
        }
    }
}
```

Kafka Connect — poison tolerance

```
# Kafka Connect 3.7 — sink connector DLQ for poison records
errors.tolerance=all
errors.deadletterqueue.topic.name=connect-dlq
errors.deadletterqueue.topic.replication.factor=3
errors.deadletterqueue.context.headers.enable=true
errors.log.enable=true
errors.log.include.messages=true
errors.retry.timeout=60000
errors.retry.delay.max.ms=10000
# Without this, a single poison record kills the connector task — with
it, poison goes to DLQ and the task continues
```

Kafka Streams — deserialization poison

```
// Kafka Streams 3.7 — ProductionExceptionHandler for poison on the
produce side
Properties streamsProps = new Properties();
streamsProps.put(StreamsConfig.DEFAULT_PRODUCTION_EXCEPTION_HANDLER_CLA
SS_CONFIG,
    LogAndContinueExceptionHandler.class); // log poison, continue
processing
// Alternative: LogAndFailExceptionHandler — fail the thread (strict,
for must-not-lose)
```

RabbitMQ — DLQ and retries

Dead-letter exchange (DLX)

Every poison or exhausted-retry message should land on a DLX, not be dropped or requeued:

```
// RabbitMQ 3.13 — declare main queue with DLX, DLQ, and quorum queues
Channel ch = conn.createChannel();

// DLQ — durable quorum queue (replicated, safe)
ch.queueDeclare("orders.dlq", true, false, false,
    Map.of("x-queue-type", "quorum"));

// Main queue — quorum, with DLX pointing to the DLQ
ch.exchangeDeclare("orders.dlx", BuiltinExchangeType.DIRECT, true);
ch.queueBind("orders.dlq", "orders.dlx", "orders");

ch.queueDeclare("orders", true, false, false, Map.of(
    "x-queue-type", "quorum",
    "x-dead-letter-exchange", "orders.dlx",
    "x-dead-letter-routing-key", "orders" // lands in orders.dlq
));

// Consumer — poison goes to DLQ, transient can requeue with backoff
(see below)
DeliverCallback cb = (tag, delivery) -> {
    try {
        process(delivery);
        ch.basicAck(delivery.getEnvelope().getDeliveryTag(), false);
    } catch (Exception e) {
        FailureKind kind = classify(e);
        if (kind == FailureKind.NON_RETRYABLE) {
            // Poison — NACK without requeue → routed to orders.dlq via
DLX
            ch.basicNack(delivery.getEnvelope().getDeliveryTag(),
false, false);
        } else {
            // Transient — requeue via delayed retry (not immediate
requeue)
            publishToRetryQueue(delivery, e); // see next section
            ch.basicAck(delivery.getEnvelope().getDeliveryTag(), false)
; // ack original, retry is a new message
        }
    }
};
ch.basicConsume("orders", false, cb, tag -> {});
```

Delayed retry — TTL+DLX vs. delayed-message plugin vs. quorum delayed

Mechanism	How	Ordering	Caveat
TTL + DLX (classic)	Retry queue with `x-message-ttl` + `x-dead-letter-exchange` back to main	Head-of-line blocking — one long-TTL message blocks shorter-TTL ones behind it	Use per-delay retry queues (`orders.retry.1s`, `orders.retry.5s`) not one queue with mixed TTLs
Delayed-message plugin (`x-delayed-message`)	Exchange holds message for `x-delay` header ms	No head-of-line blocking	Plugin required; not available on all managed RabbitMQ
Quorum queue + at-least-once retry publish	Publish to `orders.retry` with application-level `visibleAt` timestamp, consumer polls with delay	No blocking	Most portable; application controls backoff

```
// TTL+DLX delayed retry — one queue per delay level (avoids head-of-
line blocking)
void declareRetryQueues(Channel ch) throws IOException {
    for (int ttl : new int[]{1000, 5000, 30000}) {
        String retryQueue = "orders.retry." + ttl;
        ch.queueDeclare(retryQueue, true, false, false, Map.of(
            "x-queue-type", "quorum",
            "x-message-ttl", ttl,
            "x-dead-letter-exchange", "",              // back to
default exchange
            "x-dead-letter-routing-key", "orders"      // → main queue
        ));
    }
}

void publishToRetryQueue(Delivery delivery, Exception failure) throws I
OException {
    int attempt = attemptCount(delivery); // from x-attempt header
    int ttl = backoffMs(attempt);         // 1000, 5000, 30000
    String retryQueue = "orders.retry." + ttl;
    AMQP.BasicProperties props = new AMQP.BasicProperties.Builder()
        .deliveryMode(2) // persistent
        .headers(Map.of(
            "x-attempt", attempt + 1,
            "x-original-exchange", delivery.getEnvelope().getExchange()
,
            "x-exception", failure.getMessage(),
            "x-first-failure-at", Instant.now().toString()))
        .build();
    ch.basicPublish("", retryQueue, props, delivery.getBody());
    // Max attempts → DLQ instead of retry
    if (attempt + 1 >= 4) {
        ch.basicPublish("orders.dlx", "orders", props,
delivery.getBody());
    }
}
```

```
# RabbitMQ delayed-message plugin alternative (when available)
# Requires: rabbitmq-plugins enable rabbitmq_delayed_message_exchange
# Declare a delayed exchange and publish with x-delay header
# No extra queues — exchange holds the message
```

> **Quorum queues and DLX.** Quorum queues support `x-dead-letter-exchange` but the poison message is enqueued to the DLQ only after `basic.nack(requeue=false)` or `basic.reject`. `requeue=true` never touches the DLX. Always use `requeue=false` for poison/DLQ and `ack + publish to retry` for delayed retries.

Operating the DLQ

Alerting

```
# DLQ depth — any message in DLQ is an incident or a poison source to
investigate
rabbitmq_queue_messages{queue=~".+\\.dlq"} > 0
kafka_topic_partition_current_offset{topic=~".+\\.dlt"} - kafka_topic_p
artition_oldest_offset{topic=~".+\\.dlt"} > 0

# Retry exhaustion rate — how fast are messages hitting the DLQ
rate(spring_kafka_retry_exhausted_total[5m]) > 0.1

# Poison classification rate — early signal that a producer deployed
bad schema
rate(consumer_poison_total{reason="serialization"}[5m]) > 0
```

Every DLQ message should fire an alert or create a ticket. A DLQ that grows silently is a backlog of lost business events.

Replay — safely

Replay must be **idempotent** — the consumer that already processed offsets after the poison must handle the replayed message without double-charging or double-inserting. This requires the idempotent-consumer contract from Vol 6, Ch 9: an inbox/dedup table keyed by `messageId` or `topic+partition+offset`.

```
// Idempotent replay — safe to reprocess a DLQ message after the fix is
deployed
@Transactional
void replayDlqMessage(ConsumerRecord<String, byte[]> dlqRecord) {
    String messageId = new String(dlqRecord.headers().lastHeader("x-
original-offset").value());
    // inbox table: (messageId) PK — insert fails if already processed
    boolean inserted = jdbc.update(

"INSERT INTO inbox(message_id, topic, payload) VALUES (?, ?, ?) ON
CONFLICT DO NOTHING",
        messageId, dlqRecord.topic(), dlqRecord.value()) == 1;
    if (!inserted) {
        log.info("replay deduped messageId={}", messageId);
        return;
    }
    OrderEvent event = deserialize(dlqRecord.value()); // now succeeds
after fix
    orderService.process(event);
}
```

```
# Kafka — replay DLQ topic back to main after a fix (kafka-consumer +
producer loop)
# Use a dedicated replay consumer group so offsets are independent
kafka-console-consumer.sh --bootstrap-server kafka.prod:9092 \
  --topic orders.dlt --group replay-2026-08-20 --from-beginning \
  | kafka-console-producer.sh --bootstrap-server kafka.prod:9092 --
topic orders

# Better — replay with kcat preserving headers and key:
kcat -b kafka.prod:9092 -C -t orders.dlt -o beginning -f '%k|%h|%s\n' \
  | while IFS='|' read -r key headers payload; do
      # filter, transform, then produce back to orders with original
key
      echo "$payload" | kcat -b kafka.prod:9092 -P -t orders -k "$key"
    done
```

```
// Spring Kafka — programmatic replay via KafkaTemplate (preserves
headers)
void replayDlt(KafkaTemplate<String, byte[]> template, String
dltTopic, String mainTopic) {
    // Consume DLT with a throwaway group, produce back to main
    // Run once after the fix is deployed and verified in staging
}
```

Replay checklist:

1. Fix the cause (schema, handler bug, downstream) and deploy — verify in staging that the DLQ payload now deserializes/processes.
2. Replay with idempotent consumer — dedup by `messageId` / `offset` so double-replay is safe.
3. Verify downstream effects — did the replayed `OrderPlaced` create a duplicate search index entry? Idempotent downstream too.
4. Monitor — watch `consumer_lag` and `dlq_depth` during replay; throttle replay rate if downstream is still fragile (Ch 7 backpressure).

Preventing DLQ backpressure

A DLQ that is consumed slowly (or not at all) can itself grow without bound. Treat the DLQ as a queue with its own SLO: alert on depth, set retention (Kafka `retention.ms` on `.dlt` topics, RabbitMQ `x-max-length` + overflow on `.dlq` queues), and never let DLQ consumption share the same consumer group or thread pool as the main pipeline.

```
# Kafka — DLQ topic retention (don't retain poison forever, but long
enough to investigate)
# 30 days, 3 replicas, compact + delete if the DLQ is replayable by key
retention.ms=2592000000
cleanup.policy=delete
min.insync.replicas=2
```

Putting it together — the failure discipline

```
flowchart TB
    C[Consumer handler] --> Classify{Classify
retryable vs poison}
    Classify -->|Poison
serialization, schema, 4xx| DLQ1[DLQ
with failure headers
+ original bytes]
    Classify -->|Transient
503, timeout, deadlock| Retry{Attempts left?}
    Retry -->|Yes| Backoff[Backoff + jitter
cap 30s
non-blocking retry topic]
    Backoff --> C2[Retry on
retry topic / queue]
    C2 -->|success| Ack[Ack / commit]
    C2 -->|fail| Retry
    Retry -->|Exhausted| DLQ2[DLQ
with attempt history]
    DLQ1 & DLQ2 --> Alert[Alert / ticket
DLQ depth > 0]
    Alert --> Fix[Fix cause
deploy]
    Fix --> Replay[Replay
idempotent consumer
throttled]
    Replay --> Verify[Verify downstream
no duplicates]

    style DLQ1 fill:#ffcdd2
    style DLQ2 fill:#ffcdd2
    style Backoff fill:#fff3e0
    style Alert fill:#ffecb3
    style Replay fill:#e3f2fd
```

Rules that survive production:

1. **Never ack on failure.** Every failure either retries (bounded, with backoff) or goes to DLQ. Silent ack is silent loss.
2. **Classify before you retry.** Poison retried is poison delayed — it still stalls the partition for `sum(backoffs)` and still hits DLQ, just slower and with more load.
3. **Non-blocking retries by default.** Blocking (in-loop) retries stall the partition; non-blocking (retry topic/queue) keeps the main pipeline moving. Use blocking only when ordering is strictly required and poison goes to DLQ, not retry.

4. **DLQ is durable, observable, and replayable.** Store original bytes + failure metadata, alert on any DLQ depth, and replay through an idempotent consumer.
5. **Backoff with jitter, cap, and max attempts.** No unbounded retries, no synchronized thundering herds, no 10-minute stalls on a single message.
6. **Replay is a deployment, not a button.** Fix, verify in staging, replay idempotently, verify downstream, throttle if needed.

Distributed-systems lens

- **Ordering vs. availability on poison.** An ordered log cannot both preserve strict per-key ordering and make progress past poison without skipping. The DLQ is the explicit choice to favour availability (partition advances) over ordering (poison message's position is lost). For workloads where ordering is safety-critical (event sourcing Ch 5), the correct response to poison may be to *halt the partition and page* rather than skip — that is a business decision, not a framework default.
- **Retries amplify load — backpressure and retries interact.** A downstream that is slow (Ch 7) and a consumer that retries immediately will amplify the load by `attempts × consumers` and turn a transient slowdown into a retry storm. Every retry policy must include jitter, cap, and a circuit breaker or it becomes a distributed denial-of-service against its own dependency.
- **Exactly-once effect requires idempotent replay.** A DLQ message replayed after a fix is *at-least-once* — the main pipeline may have already processed later messages that assumed the poison was skipped. The consumer must be idempotent (Vol 6, Ch 9 inbox pattern) or replay will double-apply.
- **DLQ is a queue like any other.** It needs retention, replication (quorum / ISR), monitoring, and capacity planning. A DLQ that fills disk or is never consumed reintroduces the same failure it was meant to solve.

Key takeaways

- Two bad defaults — ack-on-failure (silent loss) and retry-forever (partition stall) — are replaced by bounded retries with backoff + DLQ.
- Classify first: poison (serialization, schema, 4xx, invariant) goes straight to DLQ; transient (503, timeout, deadlock) retries with exponential backoff + jitter, cap, and max attempts.
- Poison stalls ordered logs: Kafka cannot commit past the poison offset and RabbitMQ `requeue=true` tight-loops — quarantine to DLQ and commit past the poison.
- Non-blocking retries (retry topic/queue, TTL+DLX or delayed exchange) keep the main partition moving; blocking retries stall it — choose by ordering requirements.
- DLQ stores original bytes + headers + failure metadata + attempt history; it is durable (quorum/ISR), alerted on any depth, and replayed only through an idempotent consumer after the cause is fixed.
- Spring Kafka `DefaultErrorHandler` + `DeadLetterPublishingRecoverer` (+ `RetryTopicConfiguration` for non-blocking), Kafka Connect `errors.deadletterqueue.*`, and RabbitMQ `x-dead-letter-exchange` + per-TTL retry queues (or `x-delayed-message`) are the version-pinned wirings.

Further reading

- **Patterns:** Enterprise Integration Patterns — *Dead Letter Channel*, *Retry*, *Throttler* (https://www.enterpriseintegrationpatterns.com/) — the canonical vocabulary. Nygard *Release It!* (2nd ed) — Ch 4 (stability patterns) — retry, circuit breaker, and bulkhead as they interact with messaging.
- **Kafka:** Spring Kafka 3.1 reference — `DefaultErrorHandler`, `DeadLetterPublishingRecoverer`, `RetryTopicConfiguration` (https://docs.spring.io/spring-kafka/reference/) — blocking vs. non-blocking retries. Kafka 3.7 docs — `errors.tolerance`, `errors.deadletterqueue.*` (Connect), `ProductionExceptionHandler` (Streams) (https://kafka.apache.org/documentation/). Confluent *Kafka retry and DLQ* guide — retry topic topology trade-offs.

- **RabbitMQ:** RabbitMQ 3.13 docs — Dead Letter Exchanges (`x-dead-letter-exchange`), TTL + DLX, delayed-message plugin (`x-delayed-message`), quorum queues (https://www.rabbitmq.com/docs/dlx) (https://www.rabbitmq.com/docs/quorum-queues). RabbitMQ Streams — retry and DLQ considerations.
- **Backoff & reliability:** AWS *Exponential Backoff and Jitter* (https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/) — full, decorrelated, and equal jitter. Google SRE Workbook — Ch 11 (Handling Overload) — retry storms and backpressure.
- **Idempotency for replay:** Vol 6, Ch 9 — Idempotency, Deduplication, and Exactly-Once — the inbox/dedup table that makes DLQ replay safe. Ch 2 and Ch 6 of this volume — broker delivery semantics and outbox idempotency.